



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Escuela Técnica Superior de Ingeniería Informática

Desarrollo de un Asistente Inteligente Basado en LLM para
la Automatización del Análisis y la Visualización de Datos

Trabajo Fin de Grado

Grado en Ciencia de Datos

AUTOR/A: García Mas, Marián

Tutor/a: Monserrat Aranda, Carlos

CURSO ACADÉMICO: 2024/2025

*A mis padres y a mi abuela, por brindarme la oportunidad de formarme y
por su apoyo constante a lo largo de la carrera y de este proyecto.*

*A David, por ser mi compañero en cada momento, por darme la motivación
para seguir adelante, por ayudarme a despejarme cuando lo necesitaba y por
estar siempre a mi lado.*

*Y a mis amigos, que quizá sin saberlo, siempre han sido un pilar
fundamental en este camino.*

Resumen

Este proyecto tiene como objetivo el diseño e implementación de un asistente inteligente conversacional, impulsado por modelos de lenguaje de última generación (LLM), con el fin de mejorar la eficiencia y accesibilidad del análisis de datos. A lo largo del desarrollo se ha construido una aplicación interactiva que permite a los usuarios cargar conjuntos de datos tabulares y escribir consultas en lenguaje natural para obtener resúmenes estadísticos, visualizaciones automáticas o incluso estimaciones de relaciones causales entre variables. El sistema busca democratizar el acceso a la ciencia de datos, permitiendo que incluso perfiles no técnicos puedan realizar exploraciones útiles sobre sus datos.

La herramienta integra componentes de procesamiento de lenguaje natural, técnicas de prompting controlado y un conjunto de herramientas estadísticas y de visualización desarrolladas en Python. Se ha prestado especial atención a la robustez del sistema ante entradas ambiguas, a través de reglas internas que restringen las tareas del agente y guían su comportamiento. Además, se ha desarrollado un módulo específico de análisis causal que permite construir hipótesis, generar grafos causales (DAG), y estimar efectos con respaldo formal mediante la biblioteca DoWhy.

La interfaz, implementada con Streamlit, ha sido diseñada para ofrecer una experiencia clara e intuitiva. Los resultados obtenidos en las pruebas demuestran que el sistema funciona correctamente sobre datasets reducidos, incluso con el uso de LLM gratuitos, abriendo la puerta a futuras mejoras orientadas a tareas más avanzadas.

Palabras clave: LLM, Inteligencia Artificial, Visualización de Datos, Python, Langchain, Agente, Herramientas, Pandas, Prompting, Streamlit, DoWhy

Resum

Aquest projecte té com a objectiu el disseny i la implementació d'un assistent intel·ligent conversacional, impulsat per models de llenguatge de darrera generació (LLM), amb la finalitat de millorar l'eficiència i l'accessibilitat de l'anàlisi de dades. Al llarg del desenvolupament s'ha construït una aplicació interactiva que permet als usuaris carregar conjunts de dades tabulars i escriure consultes en llenguatge natural per obtenir resums estadístics, visualitzacions automàtiques o fins i tot estimacions de relacions causals entre variables. El sistema pretén democratitzar l'accés a la ciència de dades, permetent que fins i tot perfils no tècnics puguin realitzar exploracions útils sobre les seues dades.

L'eina integra components de processament de llenguatge natural, tècniques de prompting controlat i un conjunt d'eines estadístiques i de visualització desenvolupades en Python. S'ha prestat especial atenció a la robustesa del sistema davant d'entrades ambigües, a través de regles internes que restringeixen les tasques de l'agent i en guien el comportament. A més, s'ha desenvolupat un mòdul específic d'anàlisi causal que permet construir hipòtesis, generar grafs causals (DAG) i estimar efectes amb suport formal mitjançant la biblioteca DoWhy.

La interfície, implementada amb Streamlit, ha sigut dissenyada per oferir una experiència clara i intuïtiva. Els resultats obtinguts en les proves demostren que el sistema funciona correctament sobre conjunts de dades reduïts, fins i tot amb l'ús de LLM gratuïts, obrint la porta a futures millores orientades a tasques més avançades.

Paraules clau: LLM, Intel·ligència Artificial, Visualització de Dades, Python, Langchain, Agent, Eines, Pandas, Prompting, Streamlit, DoWhy

Abstract

This project aims to design and implement an intelligent conversational assistant powered by state-of-the-art language models (LLMs), with the goal of improving the efficiency and accessibility of data analysis. Throughout its development, an interactive application has been built that allows users to upload tabular datasets and write natural language queries to obtain statistical summaries, automatic visualizations, or even estimates of causal relationships between variables. The system seeks to democratize access to data science, enabling even non-technical profiles to perform meaningful explorations of their data.

The tool integrates components of natural language processing, controlled prompting techniques, and a set of statistical and visualization tools developed in Python. Special attention has been given to the robustness of the system when facing ambiguous inputs, through internal rules that restrict the agent's tasks and guide its behavior. In addition, a specific causal analysis module has been developed, allowing the construction of hypotheses, generation of causal graphs (DAGs), and estimation of effects with formal support using the DoWhy library.

The interface, implemented with Streamlit, has been designed to provide a clear and intuitive experience. The results obtained during testing demonstrate that the system works properly on small datasets, even when using free LLMs, paving the way for future enhancements aimed at more advanced tasks.

Key words: LLM, Artificial Intelligence, Data Visualization, Python, Langchain, Agent, Tools, Pandas, Prompting, Streamlit, DoWhy

Índice general

Índice general	4
Índice de figuras	7
Índice de tablas	8
1 Introducción	1
1.1 Motivación	1
1.1.1 Motivación personal	1
1.1.2 Motivación profesional	2
1.2 Objetivos	2
1.2.1 Objetivo general	2
1.2.2 Objetivos específicos	2
1.3 Estructura de la memoria	3
2 Estado del Arte	4
2.1 Aplicaciones de Inteligencia Artificial en Análisis de Datos	4
2.1.1 Rol de EDA y limitaciones actuales	4
2.1.2 Avances recientes con IA generativa	5
2.2 Revisión de Herramientas Existentes	5
2.2.1 Herramientas Destacadas	5
2.3 Aportaciones Diferenciales de la Propuesta	6
2.3.1 Comparativa Directa	6
2.3.2 Enfoque educativo y accesible	7
2.3.3 Potencial de Escalabilidad y Futuro	7
3 Fundamentos teóricos	8
3.1 Modelos de Lenguaje a Gran Escala (LLM)	8
3.1.1 Funcionamiento de los LLM	8
3.1.2 Términos Clave	9
3.2 Utilización de Agentes basados en LLM	10
3.3 Técnicas de <i>Prompt Engineering</i> y el enfoque <i>ReAct</i>	11
3.4 Fundamentos del Análisis Exploratorio de Datos (AED)	12
3.5 Fundamentos de la Inferencia Causal	13
3.5.1 Elementos clave en un análisis causal	13
3.5.2 Representación mediante Grafos Causales (<i>DAGs</i>)	14
3.5.3 Criterio de <i>Backdoor</i>	15
3.5.4 Métodos de estimación del Efecto Causal	16
3.5.5 Validación del efecto estimado y evaluación de su robustez	18
4 Metodología	19
4.1 Enfoque general	19
4.2 Fases del Proyecto	20
4.3 Planificación temporal	22
4.4 Herramientas utilizadas	23
5 Desarrollo	25
5.1 Arquitectura general del sistema	25

5.2	Arquitectura del agente	26
5.2.1	Componentes Principales del Agente	27
5.2.2	Flujo de Ejecución Interno del Agente	28
5.2.3	Ciclo de razonamiento ReAct y toma de decisiones autónoma	30
5.3	Navegación y estructura de la aplicación	31
5.3.1	Subida de archivos	31
5.3.2	Resumen Estadístico	32
5.3.3	Generador de gráficos	34
5.3.4	Análisis causal	36
6	Resultados	47
6.1	Comparativa de modelos de lenguaje (Mistral vs Gemini)	47
6.2	Evaluación del rendimiento en cada fase de la aplicación	48
6.2.1	Evaluación de la sección “resumen estadístico”	48
6.2.2	Evaluación de la sección “generador de gráficos”	55
6.2.3	Evaluación del análisis causal	55
7	Conclusiones	63
7.1	Conclusiones generales	63
7.2	Relación con la formación académica	64
8	Líneas futuras	65
	Bibliografía	67
	Anexos	69
I	Relación con los Objetivos de Desarrollo Sostenible (ODS)	70
II	Código fuente del archivo <code>main.py</code>	73
III	Módulo de subida de archivos (<code>upload.py</code>)	77
IV	Funciones auxiliares en <code>usefulservices.py</code>	78
V	Código fuente del agente (<code>causalagent.py</code>)	80
VI	Código fuente del módulo <code>summary.py</code>	87
VII	Código fuente del módulo <code>graphics.py</code>	95
VIII	Prompts y funciones auxiliares de causalidad (<code>prompts.py</code>)	99
IX	Código fuente de <code>causal.py</code>	112
X	Ejemplos de salida de la sección “resumen estadístico”	125
X.1	Detección de nulos	125
X.2	Detección de duplicados	125
XI	Ejemplos de salida de la sección “generador de gráficos”	126
XI.1	Matriz de correlación	126
XI.2	Gráfico de barras	127
XI.3	Gráfico de dispersión	127
XI.4	Gráfico <i>boxplot</i>	128
XI.5	Consultas inapropiadas	128
XII	Ejemplos de salida de la sección “inferencia causal”	129
XII.1	Definición del tratamiento y variable objetivo	129
XII.2	Definición de pregunta causal y selección de hipótesis	129
XII.3	Generación y edición del DAG	131
XII.4	Diagnóstico de identificabilidad del efecto	131
XII.5	Recomendación del método de estimación	132
XII.6	Estimación del efecto causal	132
XII.6.1	Regresión lineal	133

XII.6.2 <i>Propensity score weighting</i>	133
XII.6.3 <i>Distance matching</i>	133
XII.7 Validación de robustez	133
XII.7.1 Regresión lineal	134

Índice de figuras

3.1	Funcionamiento ReAct	12
3.2	Ejemplo grafo causal	15
4.1	Diagrama de Gantt	23
5.1	Arquitectura diseñada	25
5.2	Composición del agente	27
5.3	Flujo de ejecución interno del agente	28
5.4	Interfaz de la sección “Subida de archivo”	32
5.5	Interfaz de la sección “Resumen estadístico”	34
5.6	Interfaz de la sección “Generador de gráficos”	36
5.7	Flujo de inferencia causal desarrollado	37
5.8	Ejemplo de DAG causal generado y editado en la aplicación.	42
6.1	Demostración del funcionamiento de “perfilado general del dataset”	49
6.2	Demostración del funcionamiento de “significado de las columnas”	50
6.3	Ejemplo de detección de nulos con faltantes añadidos.	50
6.4	Ejemplo de detección de duplicados con filas añadidas.	51
6.5	Demostración del cálculo de “correlaciones numéricas”	52
6.6	Demostración de la explicación de “correlaciones numéricas”	52
6.7	Demostración del funcionamiento de “sugerencias de variables”	53
6.8	Demostración del funcionamiento de “sugerencias de variables”	53
6.9	Demostración del funcionamiento de “propuesta de variable objetivo”	54
6.10	Demostración del funcionamiento de “valores atípicos”	55
6.11	Demostración del funcionamiento de “selección de variables”	56
6.12	Demostración del funcionamiento de “variables candidatas”	57
6.13	Demostración del funcionamiento de “generación de hipótesis”	57
6.14	Demostración del funcionamiento de “generación del DAG”	58
6.15	Demostración del funcionamiento de “identificación del efecto”	59
6.16	Demostración del funcionamiento de “recomendación del método estimación”	60
6.17	Demostración del funcionamiento de “estimación del efecto”	61
6.18	Demostración del funcionamiento de “validación de robustez”	61
X.1	Ejemplo de detección de nulos del <i>dataset</i> original.	125
X.2	Ejemplo de detección de duplicados del <i>dataset</i> original.	125
XI.1	Ejemplo de matriz de correlación generada por el agente	126
XI.2	Ejemplo de barplot generado por el agente	127
XI.3	Ejemplo de gráfico de dispersión generado por el agente	127
XI.4	Ejemplo de boxplot generado por el agente	128
XI.5	Ejemplo de consulta errónea	128
XI.6	Ejemplo de consulta errónea	128
XII.1	Indicaciones del sistema al seleccionar tratamiento y resultado.	129

XII.2 Formulación de pregunta causal.	130
XII.3 Hipótesis generadas por el agente.	130
XII.4 Hipótesis generadas por el agente.	130
XII.5 Generación columna treatment.	130
XII.6 Ejemplo de DAG generado automáticamente por el sistema.	131
XII.7 Ejemplo de DAG modificado por usuario.	131
XII.8 Diagnóstico generado por el agente.	131
XII.9 DAG incompleto.	132
XII.10 Diagnóstico generado por el agente tras DAG incompleto.	132
XII.11 Método de estimación recomendado por el agente.	132
XII.12 Estimación con regresión lineal.	133
XII.13 Estimación con <i>propensity score weighting</i>	133
XII.14 Estimación con <i>distance matching</i> (error).	133
XII.15 Validación con regresión lineal.	134
XII.16 Validación con <i>propensity score weighting</i>	134

Índice de tablas

2.1 Revisión de herramientas implementadas por otros usuarios	5
2.2 Comparativa entre herramientas existentes y propuesta TFG	7

CAPÍTULO 1

Introducción

En la actualidad, la Inteligencia Artificial (IA) se ha consolidado como una gran herramienta informática para gran diversidad de sectores. Este campo, en constante evolución, busca emular la capacidad cognitiva humana mediante máquinas, permitiendo el desarrollo de sistemas capaces de aprender, adaptarse y realizar tareas con una eficiencia sin precedentes [1].

Este Trabajo de Fin de Grado (TFG) se centra en la generación de un asistente de Inteligencia Artificial basado en modelos de lenguaje LLM y agentes. En el contexto actual, un agente de Inteligencia Artificial es capaz de realizar tareas de forma autónoma, diseñando su flujo de trabajo y utilizando las herramientas disponibles [2].

El objetivo base del proyecto consiste en optimizar las tareas frecuentes de un analista de datos, especialmente las relacionadas con la generación de resúmenes estadísticos, visualizaciones gráficas y análisis de causalidad. Estas tareas suelen costar tiempo y dedicación, añadiendo el requerimiento de conocimientos técnicos para ejecutar dichas operaciones. Así pues, se sugiere un entorno donde el usuario pueda adjuntar la base de datos que desee. Mediante la activación de un agente inteligente, se ofrecerá un análisis de la fuente. Además, se facilitará la generación de una amplia gama de gráficos y, finalmente, la detección de posibles relaciones causales.

En definitiva, con este trabajo se pretende explorar el potencial que tienen los LLM cuando se integran con agentes especializados en el análisis de datos. Es importante resaltar que, para la realización de este proyecto, se han empleado modelos accesibles mediante APIs públicas y sin costo. Con esta elección, se da a conocer la disponibilidad funcional de este tipo de herramientas sin necesidad de licencias de pago.

1.1 Motivación

El presente proyecto surge del interés por explorar el potencial de las tecnologías de procesamiento de lenguaje natural y de los modelos de lenguaje a gran escala, cuyo desarrollo ha experimentado un crecimiento exponencial en los últimos años. El avance de estas herramientas, junto con la posibilidad de integrarlas con agentes, abre la puerta a aplicaciones con impacto tanto en entornos profesionales como en el uso cotidiano por parte de usuarios sin conocimientos técnicos avanzados.

1.1.1. Motivación personal

El interés por las tecnologías de procesamiento de lenguaje natural (NLP) y los modelos de lenguaje a gran escala (LLM) surgió a raíz del lanzamiento de la plataforma

ChatGPT en noviembre de 2022. La capacidad de este tipo de modelos para generar código de forma efectiva despertó la inquietud de comprender el funcionamiento interno de estas tecnologías y sus fundamentos.

La profundización en este campo, complementada con formación adicional y la experiencia adquirida durante prácticas en empresa, permitió conocer enfoques avanzados como el uso de agentes y su integración con múltiples herramientas, incluyendo la posibilidad de desarrollar herramientas propias que amplíen sus capacidades. Este trabajo se presenta como una oportunidad para combinar intereses en inteligencia artificial, lenguaje natural y análisis de datos, explorando tecnologías emergentes con gran potencial de impacto en distintos sectores.

1.1.2. Motivación profesional

Desde una perspectiva profesional, el proyecto se plantea como una oportunidad para aplicar de forma práctica tecnologías que suponen un cambio de paradigma a nivel tecnológico. La integración de agentes inteligentes con herramientas permite optimizar procesos, mejorar la capacidad de análisis y ofrecer soluciones adaptadas tanto a expertos en datos como a perfiles sin formación técnica.

Asimismo, este enfoque contribuye a reducir barreras en la adopción de la inteligencia artificial, ofreciendo un marco flexible y escalable que puede aplicarse en distintos ámbitos profesionales.

1.2 Objetivos

1.2.1. Objetivo general

Diseñar e implementar un asistente inteligente interactivo, basado en modelos de lenguaje de gran tamaño (LLM) y arquitecturas de agentes orquestadas, capaz de asistir a usuarios no expertos en la exploración y análisis de datos. El sistema integrará funcionalidades de resumen estadístico, generación de visualizaciones y ejecución guiada de flujos de inferencia causal, todo ello accesible desde una interfaz web.

1.2.2. Objetivos específicos

1. Analizar el funcionamiento de los modelos de lenguaje a gran escala (LLM) y su aplicación con agentes inteligentes.
2. Diseñar un agente capaz de interpretar instrucciones en lenguaje natural y ejecutar acciones sobre un conjunto de datos proporcionado.
3. Incorporar técnicas de prompt engineering para optimizar la generación de respuestas válidas y precisas por parte del agente.
4. Desarrollar un entorno interactivo que facilite la interacción usuario-agente de forma intuitiva y accesible.
5. Implementar herramientas para facilitar la obtención de resultados, ofreciendo una visión global del conjunto de datos.
6. Desarrollar nuevas herramientas para el análisis de causalidad que permitan al agente identificar e interpretar relaciones causa-efecto en los datos.

7. Comprobar el funcionamiento del asistente mediante pruebas con bases de datos reales.
8. Comparar el comportamiento y resultados obtenidos por distintos modelos de lenguaje evaluados en la construcción del asistente.
9. Elaborar una documentación completa y estructurada que facilite la implementación, uso y mejora del sistema en el futuro.

1.3 Estructura de la memoria

La memoria presenta ocho capítulos de manera estructurada, los cuales contienen las fases del proyecto desarrollado.

- **Capítulo 1. Introducción:** se presenta la contextualización del tema, la motivación principal, los objetivos propuestos (generales y específicos) y la estructura de la memoria.
- **Capítulo 2. Estado del Arte:** se analiza el panorama actual de las aplicaciones desarrolladas. Se examinan avances recientes con IA generativa y se identifican las carencias comunes en las soluciones revisadas y se destacan las aportaciones diferenciales de la propuesta en este trabajo.
- **Capítulo 3. Fundamentos teóricos:** presenta los conceptos necesarios para contextualizar el trabajo, incluyendo el funcionamiento de los Modelos de Lenguaje a Gran Escala (LLM), el papel de los agentes basados en LLM y las técnicas de prompt engineering con el enfoque ReAct. Asimismo, se introducen los fundamentos del análisis exploratorio de datos (EDA) y de la inferencia causal.
- **Capítulo 4. Metodología:** expone el enfoque seguido para el desarrollo del proyecto, estructurado en distintas fases que abarcan desde la definición inicial hasta la implementación y validación del sistema.
- **Capítulo 5. Desarrollo:** detalla la implementación práctica del sistema, comenzando con la arquitectura general y la estructura interna del agente inteligente. Se describen sus componentes principales, el flujo de ejecución y el ciclo de razonamiento. Además, se expone la navegación y organización de la aplicación desarrollada, explicando cada uno de sus módulos.
- **Capítulo 6. Resultados:** recoge la evaluación experimental del sistema, comenzando con una comparativa de modelos de lenguaje. Se presentan los resultados obtenidos en cada fase de la aplicación, incluyendo la generación de resúmenes estadísticos, las funcionalidades de visualización y el módulo de análisis causal.
- **Capítulo 7. Conclusiones:** resume los principales logros del trabajo, destacando el cumplimiento de los objetivos planteados y el valor del sistema desarrollado.
- **Capítulo 8. Líneas futuras:** plantea posibles extensiones y mejoras del sistema desarrollado.
- **Anexos:** incluyen la relación del trabajo con los Objetivos de Desarrollo Sostenible (ODS), el código fuente completo de los módulos implementados y ejemplos de salida de la aplicación en las secciones de visualización e inferencia causal.

CAPÍTULO 2

Estado del Arte

En este capítulo se realiza un análisis del panorama actual en torno a las tecnologías utilizadas en el presente Trabajo de Fin de Grado (TFG). Se abordan las principales aplicaciones basadas en Inteligencia Artificial (IA) enfocadas en el análisis de datos, con especial énfasis en las herramientas que combinan modelos de lenguaje (LLM) y agentes inteligentes. Asimismo, se examinan diversas soluciones existentes que permiten la exploración de conjuntos de datos, la generación de visualizaciones y, en menor medida, la realización de inferencias causales.

2.1 Aplicaciones de Inteligencia Artificial en Análisis de Datos

El análisis de datos siempre ha conestado como una disciplina orientada a perfiles con experiencia en estadística o programación. Sin embargo, debido al crecimiento de las tecnologías de Inteligencia Artificial, especialmente los modelos generativos, ha comenzado a transformar la forma de uso de este campo. A continuación, se describe el papel actual del análisis exploratorio de datos (EDA) y cómo las nuevas tecnologías basadas en IA están redefiniendo sus posibilidades.

2.1.1. Rol de EDA y limitaciones actuales

El análisis exploratorio de datos (EDA, por sus siglas en inglés) constituye una fase fundamental dentro de cualquier proceso de análisis de datos. Consiste en examinar los datos para identificar patrones, tendencias, inconsistencias e información valiosa. Con ello, podemos familiarizarnos con nuestros datos y realizar cálculos estadísticos y representaciones gráficas [3]. Esta etapa suele apoyarse en herramientas de automatización, como puede ser Pandas Profiling [4], que facilitan la generación de informes.

Sin embargo, estas herramientas presentan limitaciones. En primer lugar, requieren conocimientos técnicos previos para su correcta utilización, lo que limita su accesibilidad a usuarios sin experiencia en programación. En segundo lugar, los resultados que ofrecen tienden a ser genéricos y poco flexibles. Finalmente, estas herramientas no permiten una interacción ni integran mecanismos de razonamiento contextual que faciliten la personalización o interpretación de los resultados. Es aquí donde el uso de modelos de lenguaje (LLM) introduce un nuevo enfoque, más adaptable y centrado en el usuario, permitiendo transformar los resultados en función de las necesidades.

2.1.2. Avances recientes con IA generativa

El auge de los modelos de lenguaje a gran escala (LLM), como GPT-4, ha abierto la puerta a nuevas aplicaciones. Estas tecnologías permiten interpretar instrucciones en lenguaje natural y generar código, gráficos o análisis de forma automática. Este nuevo enfoque se alinea con una tendencia, donde no es necesario tener conocimientos técnicos profundos para obtener información relevante de un dataset.

Entre los avances más destacados, se encuentran los asistentes basados en IA, que combinan modelos de lenguaje con entornos interactivos como Streamlit [5], nuevas herramientas o soluciones open-source desarrolladas en plataformas como HuggingFace Spaces [6].

A pesar de su potencial, estos asistentes aún presentan algunas limitaciones. Su funcionamiento suele depender de APIs externas, que pueden tener restricciones de uso o requerir conexión constante. Además, el rendimiento del análisis puede variar según la calidad del prompt o los datos introducidos. Sin embargo, estos sistemas representan un cambio de paradigma en la forma de interactuar con los datos, ofreciendo una alternativa más intuitiva y accesible.

2.2 Revisión de Herramientas Existentes

Durante los últimos años, se han desarrollado gran cantidad de proyectos que integran modelos de lenguaje con el análisis de datos. En esta sección, se analizan algunas de las propuestas más relevantes que utilizan tecnologías como LangChain [7], Streamlit [5] y modelos LLM, identificando sus funcionalidades, limitaciones y grado de innovación. La revisión permite establecer un punto de referencia para valorar las aportaciones diferenciales del presente Trabajo de Fin de Grado (TFG).

2.2.1. Herramientas Destacadas

Proyecto	Tecnologías	Capacidades Clave	Limitaciones
Build an AI Data Assistant (DigiLab)	Streamlit, LangChain, OpenAI	Carga de CSV, resúmenes estadísticos, visualización, predicción	No incluye análisis causal ni personalización de gráficos
AI Assistant with Vector Store	Streamlit, LangChain, FAISS	Subida de archivos, búsqueda semántica de documentos	Enfocado a documentos, no al análisis estructurado de datos
LangChain GenAI Data Assistant	Streamlit, LangChain, LLaMA3-70B	Extracción de insights, generación de gráficos	Limitado a respuestas predefinidas, sin razonamiento causal

Tabla 2.1: Revisión de herramientas implementadas por otros usuarios

Build and AI Data Assistant (DigiLab) [8]: Este proyecto, difundido a través de tutoriales y contenidos formativos, presenta una guía para construir un asistente de análisis de datos empleando tecnologías de Streamlit [5], LangChain [7] y OpenAI [11]. La aplicación permite la carga de archivos en formato CSV, la generación de resúmenes estadísticos y una visualización gráfica de algunos tipos de datos. Además, ofrece opciones básicas para seleccionar modelos de predicción. A pesar de su utilidad como punto de partida, no incorpora funcionalidades avanzadas como la detección de relaciones causales ni la personalización detallada de los gráficos en base a lenguaje natural.

AI Assistant with Vector Store (GitHub: sharonmordehchai) [9]: Esta solución se centra en la integración de LangChain [7] con una vector store basada en FAISS, lo que permite consultas sobre documentos. Con una interfaz amigable, la aplicación está orientada principalmente a la recuperación de información textual, más que al análisis estadístico o visual de datos tabulares. No incluye capacidades gráficas ni razonamiento causal, por lo que su utilidad en análisis de datos estructurados es limitada.

LangChain GenAI Data Assistant (GitHub: PragyanTiwari) [10]: Esta herramienta combina LangChain [7], Streamlit [5] y el modelo LLaMA3-70B para facilitar el análisis de datos mediante instrucciones en lenguaje natural. Permite generar gráficos y obtener información sobre la base de datos de manera sencilla. Sin embargo, su funcionamiento se basa en prompts predefinidos y carece de capacidades avanzadas como el análisis causal o la interacción con el usuario.

Los proyectos dejan en claro el interés común de combinar agentes basados en LLM con el análisis de datos. En general, todos tienen distintas formas de generación de respuestas, añadiendo que el modelo escogido también influye en la respuesta conseguida. Cabe destacar que ninguno integra el análisis causal, ni tampoco la elección propia para el usuario de hacer el gráfico que desee en lenguaje natural.

2.3 Aportaciones Diferenciales de la Propuesta

Este Trabajo de Fin de Grado (TFG) se ha diseñado con el objetivo de conseguir una experiencia mayormente estructurada y accesible. Se han tenido en cuenta las capacidades de los LLM y las limitaciones que supone utilizar un entorno gratuito.

2.3.1. Comparativa Directa

La propuesta destaca frente a otras soluciones revisadas en el Estado del Arte, ya que incorpora funcionalidades que no se encuentran presentes en los proyectos analizados:

Característica	Propuesta TFG	Herramientas Existentes
Flujo guiado en lenguaje natural	Sí	No
Generación de visualizaciones personalizadas	Sí	Limitada o predefinida
Análisis causal completo con DAG y estimación	Sí	No
Interacción dentro del flujo causal	Sí	No
Uso exclusivo de APIs gratuitas	Sí	Varias requieren pago o es clave privada
Orientación formativa	Sí	No

Tabla 2.2: Comparativa entre herramientas existentes y propuesta TFG

Esta tabla ilustra cómo el enfoque adoptado busca cubrir carencias existentes, especialmente en lo que respecta a la flexibilidad de uso, la interacción y la capacidad de actuar según las intenciones del usuario.

2.3.2. Enfoque educativo y accesible

Una de las prioridades del proyecto ha sido facilitar la comprensión y el uso del sistema incluso a personas sin conocimientos técnicos avanzados. Para ello, se ha diseñado una experiencia guiada que acompaña al usuario desde la carga del dataset hasta la interpretación de los resultados.

Este enfoque educativo se refleja en:

- La simplificación del proceso de análisis de datos mediante instrucciones en lenguaje natural.
- La eliminación de la necesidad de escribir código.
- La posibilidad de experimentar con visualizaciones o hipótesis sin riesgo, fomentando el aprendizaje exploratorio.
- Cada paso del flujo causal está explicado en lenguaje natural, con mensajes de ayuda y validación visual.
- El sistema permite experimentar con diferentes configuraciones de forma segura, fomentando el aprendizaje por exploración.

En este sentido, la herramienta también puede ser útil en contextos docentes, tanto en formación universitaria enfocada en el análisis de datos o la inteligencia artificial.

2.3.3. Potencial de Escalabilidad y Futuro

El asistente construido constituye una base extensible, sobre la que se podrían desarrollar nuevas funcionalidades. Gracias a su arquitectura, el agente desarrollado permite incorporar nuevas herramientas y capacidades especializadas sin necesidad de rediseñar el sistema completo.

CAPÍTULO 3

Fundamentos teóricos

3.1 Modelos de Lenguaje a Gran Escala (LLM)

Los modelos de lenguaje de gran tamaño (LLM) se han convertido en uno de los grandes avances del lenguaje natural. Los LLM son modelos estadísticos de lenguaje preentrenados basados en redes neuronales. Se caracterizan por contener miles de millones de parámetros entrenables y utilizar arquitecturas profundas que les permiten generar y comprender texto de forma contextual y coherente [12].

Estos modelos han sido entrenados sobre enormes corpus textuales para capturar patrones, relaciones semánticas y estructuras lingüísticas complejas. Entre los ejemplos más destacados de LLM se encuentran *GPT-4 (OpenAI)* [13], *Gemini 2.5 (Google DeepMind)* [14] y *LLaMA 4 (Meta AI)* [15].

Su evolución parte del desarrollo de la arquitectura Transformer, presentado en 2017 en el artículo '*Attention is All You Need*', que marcó una diferencia debido a las capas de atención. Esta arquitectura incluye otras innovaciones, como son los *embeddings* y el sistema *encoder-decoder*, que consigue la concentración de las partes más relevantes por parte del decodificador [16].

Seguidamente, aparecieron los modelos *encoder-only*, como *BERT* en el año 2018, orientado a la comprensión del lenguaje. Poco después, surgieron modelos *decoder-only*, donde entran los modelos *GPT*, con altas capacidades para la generación de texto [17]. Los últimos modelos se han configurado como la base para la creación de los asistentes inteligentes actuales.

3.1.1. Funcionamiento de los LLM

El objetivo central del entrenamiento de los LLM se enfoca en predecir la siguiente palabra en una oración. Este objetivo sencillo conduce a que los modelos aprendan estructuras complejas del lenguaje, lo que permite una amplia gama de capacidades emergentes [17]. Algunas de estas capacidades son:

- Generación de texto natural en múltiples idiomas.
- Resumen automático de documentos.
- Respuestas a preguntas abiertas o específicas.
- Traducción automática.
- Extracción de información y búsqueda semántica.

- Redacción de código, análisis de datos o asistencia conversacional.

Gracias a este comportamiento, en el que cada palabra generada se convierte en parte del contexto para predecir la siguiente, los LLM producen texto que es coherente, fluido y contextualmente apropiado. A esto se suma el uso del mecanismo de atención, que permite al modelo detectar relaciones entre palabras independientemente de la distancia que haya entre ellas [17].

3.1.2. Términos Clave

El uso y comprensión de los modelos de lenguaje a gran escala (LLM) requiere familiarizarse con una serie de conceptos fundamentales que forman parte de su funcionamiento interno. Entre los más relevantes se encuentran los siguientes:

Token y tokenización

La *tokenización* es la fase inicial al interactuar con los LLM. Consiste en dividir el texto en piezas más pequeñas conocidas como *tokens*. Estas pueden ser palabras, subpalabras, caracteres o secuencias frecuentes de texto. Según el enfoque del modelo, la tokenización puede hacerse a partir de espacios en blanco, signos de puntuación, o incluso mediante métodos más sofisticados como el *Byte Pair Encoding (BPE)* [17].

Este método permite al modelo comprender palabras complejas como la palabra «*coming*», dividiéndola en «*com*» y «*ing*», y así asociar elementos gramaticales comunes como los tiempos verbales [17].

Contexto y ventana de tokens

El tamaño del contexto o ventana de contexto es un aspecto crucial de los LLM. Se refiere al número máximo de tokens que el modelo puede procesar en una sola solicitud [17]. Es decir, la cantidad de información previa que el modelo es capaz de recordar durante la generación de texto.

Modelos como *GPT-4* admiten contextos de hasta 32.000 tokens, mientras que modelos recientes como *Gemini* pueden manejar 100.000 tokens o más. Esto resulta útil en tareas como el análisis de documentos extensos o respuestas a preguntas complejas basadas en gran cantidad de información.

Embeddings

Una vez tokenizado el texto, cada token se transforma en un *embedding*. Un *embedding* otorga a cada token un vector numérico único (en un espacio multidimensional) que captura su significado. Estos vectores permiten representar relaciones semánticas entre palabras en un espacio multidimensional [17].

Durante el entrenamiento, los *embeddings* se ajustan para que tokens con significados similares se ubiquen cerca entre sí [17]. Este paso es considerado como esencial para que el modelo entienda el lenguaje de forma matemática.

Entrenamiento y ajuste fino

El entrenamiento inicial de los LLM, llamado preentrenamiento, consiste en ajustar los parámetros del modelo para que aprenda a predecir el siguiente token de una secuen-

Este aprendizaje suele basarse en grandes corpus de texto genérico, aunque también se pueden usar datos especializados para mejorar el rendimiento en tareas concretas. En fases posteriores, el modelo puede pasar por un ajuste fino (*fine-tuning*), entrenándose con datos más específicos de un dominio o una tarea [17].

Predicción autoregresiva

El mecanismo más común de generación de texto es autoregresivo. El modelo genera texto tras el entrenamiento prediciendo tokens consecutivos en una secuencia. Es decir, utiliza los tokens anteriores como contexto. Este proceso continúa hasta que se alcanza un token especial de fin de secuencia, o se cumple una condición definida. En cada paso, el modelo asigna una probabilidad a cada posible token del vocabulario y elige el más apropiado [17].

Leyes de Escalado

Las leyes de escalado describen la relación entre el rendimiento del modelo y factores como los parámetros, el tamaño del dataset, el presupuesto computacional y la arquitectura de red [17].

Según el *estudio Chinchilla*, existe una proporción óptima entre el tamaño del modelo (en parámetros) y la cantidad de datos (en tokens) para maximizar su eficiencia. Así, un modelo con 70 mil millones de parámetros puede superar a otro más grande si ha sido entrenado con la cantidad adecuada de tokens [18].

3.2 Utilización de Agentes basados en LLM

Los agentes son sistemas que utilizan LLM para determinar y ordenar un conjunto de acciones. En un flujo de trabajo simple, estas acciones pueden implicar usar una herramienta, examinar su resultado y responder a la solicitud del usuario [17].

Cuando estos agentes se integran con modelos de lenguaje a gran escala (LLM), adquieren la capacidad de procesar instrucciones complejas en lenguaje natural, razonar sobre ellas y ejecutar tareas dinámicas. Esta capacidad ha dado lugar a una nueva generación de agentes potenciados por LLM, capaces de operar sobre entornos abiertos con herramientas externas, tomar decisiones en tiempo real y adaptarse a distintas situaciones [17].

La innovación clave que aportan estos agentes reside en su capacidad de usar herramientas específicas para resolver subtarefas, como usar *Google Search API*, acceder a una base de datos SQL, ejecutar código *Python* o utilizar la calculadora. Con esto, los LLM actúan como el motor de razonamiento, mientras que el sistema que se encarga de gestionar la comunicación entre el modelo y las herramientas es la orquestación del agente [17].

Componentes de un agente

Un agente LLM suele componerse de los siguientes elementos:

- **Modelo de lenguaje (*core*):** actúa como el núcleo que toma decisiones, razona y genera texto [17].

- **Prompt o instrucción inicial:** guía al agente sobre la tarea que debe realizar [17].
- **Herramientas externas:** funciones conectadas que permiten al agente ejecutar acciones especializadas [17].
- **Memoria:** mecanismo opcional para almacenar información relevante de la interacción y mantener coherencia contextual [17].
- **Sistema de orquestación:** controla cuándo invocar herramientas, cómo interpretar los resultados y cómo combinar la información en la respuesta final [17].

Tipos de agentes

Los agentes pueden clasificarse en dos categorías generales. Primero están los agentes de acción (*Action Agents*), que toman decisiones inmediatas y ejecutan una sola acción por paso. Son útiles para tareas simples y bien definidas. En segundo lugar, están los agentes de planificación y ejecución (*Plan-and-Execute Agents*), que primero generan un plan con una secuencia de acciones, y luego ejecutan dichas acciones de forma ordenada o paralela. Pueden reajustar el plan en función de los resultados obtenidos [17].

Flujo de trabajo

El flujo de ejecución más común para agentes de acción sigue estos pasos [17]:

1. El sistema recibe una consulta del usuario.
2. El modelo (LLM) decide si necesita una herramienta, y cuál debe usar.
3. Se ejecuta la herramienta seleccionada con los parámetros definidos.
4. El LLM recibe el resultado, genera una observación, y decide el siguiente paso.
5. El proceso continúa hasta que el modelo puede ofrecer una respuesta final sin usar más herramientas.

Para la creación de este tipo de agentes, se ha empleado *LangChain* [7], un marco de desarrollo orientado a la construcción de aplicaciones basadas en modelos de lenguaje a gran escala (LLM). Esta biblioteca proporciona una arquitectura modular que cubre el ciclo de vida de este tipo de soluciones. Implementa interfaces estándar para modelos de lenguaje, modelos de embedding y vector stores, lo que permite combinar de forma flexible diferentes tecnologías y optimizar su comportamiento en entornos reales. Asimismo, posee herramientas para la orquestación de agentes, gestión de contexto conversacional, persistencia y soporte de *streaming* [19]. Aunque la implementación concreta se detalla en otro capítulo, es importante destacar que este tipo de bibliotecas permiten construir agentes.

3.3 Técnicas de *Prompt Engineering* y el enfoque *ReAct*

El *prompt engineering* es una disciplina que crea y optimiza instrucciones para aprovechar los modelos de lenguaje en diversas aplicaciones y áreas de investigación. Un *prompt* presenta una tarea o instrucción específica al modelo de lenguaje, el cual responde en función de la información contenida en dicho prompt. Un prompt puede ser una pregunta

o una entrada más compleja que incluya contexto o información para guiar al modelo con precisión [17]. Este también puede contener ejemplos, que esto está relacionado con técnicas de *few-shot* y *multi-shot prompting*.

Una de las estrategias más empleadas es el *Chain of Thought (CoT) Prompting*, que permite que los modelos descompongan problemas de múltiples pasos en pasos intermedios, para asignar mayor capacidad de cómputo a aquellos problemas que requieren un mayor número de pasos de razonamiento. Este razonamiento puede utilizarse en tareas como problemas matemáticos, manipulación simbólica o cualquier tarea que los humanos puedan resolver mediante el lenguaje [20].

La evolución de esta técnica es el enfoque *ReAct (Reasoning + Acting)*, empleada por los proyectos anteriormente mencionados. Este tipo de prompting combina razonamiento y acción con modelos de lenguaje. Es capaz de guiar a los LLM para generar tanto trazas de razonamiento verbal como acciones relacionadas con la tarea de manera intercalada. Primero el modelo crea, mantiene y ajusta planes de alto nivel para actuar (razonar para actuar), y a la vez interactúa con entornos externos para conseguir información adicional (actuar para razonar) [21].

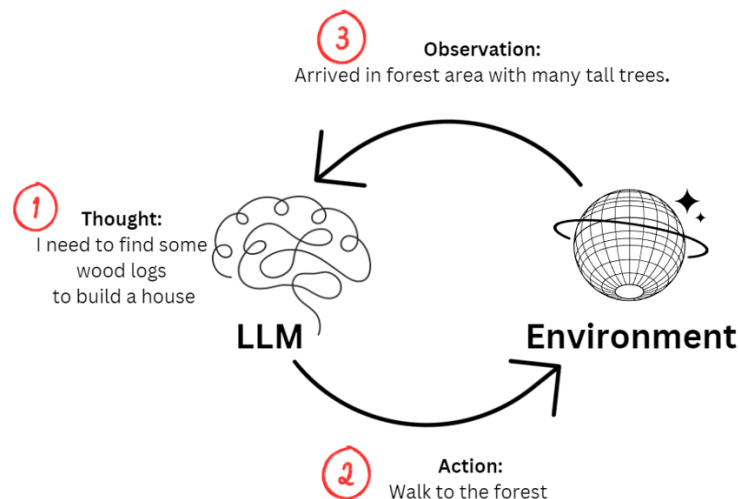


Figura 3.1: Funcionamiento ReAct

Debido a que *ReAct* y otras técnicas forman parte de los proyectos más actuales relacionados con agentes basados en LLM, resulta relevante incorporarlo en el proyecto como metodología clave.

3.4 Fundamentos del Análisis Exploratorio de Datos (AED)

El Análisis Exploratorio de Datos (AED) constituye una etapa inicial esencial en cualquier proceso de ciencia de datos. Su objetivo principal es comprender la estructura general del conjunto de datos, detectar patrones, anomalías, relaciones entre variables y posibles errores. También se define como un enfoque, una actitud frente a los datos, apoyado en un conjunto de herramientas gráficas y síntesis de los datos. [...] En este sentido, el Análisis Exploratorio de Datos puede convertirse en una mini generadora de hipótesis plausibles [22].

El enfoque propuesto por John Tukey pone énfasis en «dejar que los datos hablen», en lugar de partir de hipótesis confirmatorias [23]. Así, mediante el uso de visualizaciones, como por ejemplo boxplots, se pueden identificar características destacadas del dataset.

Los objetivos del Análisis Exploratorio de Datos (AED) pueden resumirse de la siguiente manera [24]:

1. Maximizar la comprensión de la base de datos y entender su estructura.
2. Visualizar las posibles relaciones (dirección y magnitud) entre las variables de exposición y las variables de resultado.
3. Detectar valores atípicos y anomalías (observaciones significativamente diferentes al resto).
4. Desarrollar modelos parsimoniosos (modelos predictivos o explicativos que funcionen con el menor número posible de variables de exposición) o realizar una selección preliminar de modelos apropiados.
5. Extraer y crear variables clínicamente relevantes.

Cabe destacar la importancia de la visualización de datos, ya que es considerada como una herramienta esencial capaz de representar la información de manera accesible, aportando un valor analítico y apoyando a la toma de decisiones.

En el presente proyecto, el AED ha sido implementado como una fase clave previa a la visualización y el análisis causal, permitiendo al agente identificar variables relevantes, detectar errores comunes y orientar al usuario en la comprensión inicial del dataset cargado.

3.5 Fundamentos de la Inferencia Causal

La inferencia causal es la ciencia que se encarga de inferir causalidad a partir de asociaciones y de entender cuándo y por qué difieren entre sí. Tiene como objetivo responder a la pregunta: «¿qué efecto tiene una acción (tratamiento) sobre un resultado?». Esta disciplina se encarga de estudiar el impacto de una intervención sobre un sistema observado, superando las limitaciones de la simple asociación estadística [25].

En este marco, el objetivo se puede reformular como la estimación del efecto que una variable de tratamiento T tiene sobre una variable de resultado Y , reconociendo que nunca es posible observar el mismo individuo en los dos estados (tratado y no tratado). Este dilema se conoce como el «problema fundamental de la inferencia causal» [25].

Una confusión común es interpretar una correlación como evidencia de causalidad. La correlación mide el grado en que dos variables se relacionan estadísticamente, pero no implica que una provoque a la otra. De hecho, pueden estar correlacionadas por azar o por compartir una causa común no observada. Esta advertencia se resume en el principio ampliamente reconocido de que «la correlación no implica causalidad». Por el contrario, la causalidad implica que una variable provoca un cambio en otra [25].

3.5.1. Elementos clave en un análisis causal

El análisis causal comienza identificando la unidad de análisis, es decir, el sujeto o entidad sobre la cual se aplica el tratamiento. En la mayoría de los casos, se trata de personas, pero también puede referirse a productos, regiones geográficas, u otros elementos según el contexto del estudio [25].

A continuación, se van a definir las variables clave:

- Tratamiento (T): Es la intervención o acción cuya influencia se quiere estudiar. Puede ser binario, categórico o continuo [25].
- Resultado (Y): Es la variable dependiente sobre la que se espera observar un cambio como consecuencia del tratamiento [25].
- Variables confusoras (Z): Son aquellas variables que afectan tanto al tratamiento como al resultado. Si no se ajustan adecuadamente, pueden introducir sesgos en la estimación causal. El sesgo es lo que hace que la asociación difiera de la causalidad. El hecho de que lo que se estima a partir de los datos no coincida con la cantidad causal que se quiere recuperar es el verdadero problema [25].
- Variables mediadoras: Actúan como mecanismo intermedio entre el tratamiento y el resultado. Es una ayuda para explicar cómo ocurre el efecto [25].
- Variables de efecto/moderadoras: No afectan directamente el tratamiento ni el resultado, pero modifican la intensidad o dirección del efecto causal [25].

El análisis causal se apoya en modelos que representan relaciones mediante funciones y asignaciones del tipo $\leftarrow$. En estos modelos, se diferencian claramente las variables internas del sistema (aquellas que nos interesan) y las externas (representadas comúnmente por u), sobre las cuales no se modela su mecanismo de generación [25].

Además, se introducen herramientas como el operador $do(\cdot)$, que representa intervenciones activas. Por ejemplo, $do(T = t_0)$ significa analizar lo que sucedería si el tratamiento T fuera fijado de manera externa en el valor t_0 , rompiendo cualquier dependencia natural entre T y sus causas. Este enfoque permite estimar el efecto causal individual como la diferencia entre Y bajo tratamiento y Y sin tratamiento [25].

Hay que tener en cuenta que los datos por sí solos no son suficientes para determinar relaciones causales, todo análisis causal requiere establecer supuestos. Estos supuestos no son observables en los datos, pero son esenciales para la identificación del efecto causal. Por otro lado, se debe tener en cuenta que, en estudios observacionales, donde no existe asignación aleatoria, el ajuste adecuado de las variables confusoras y el uso de conocimiento experto son claves para aproximarse a una estimación válida del efecto causal [25].

3.5.2. Representación mediante Grafos Causales (DAGs)

Una de las principales innovaciones ha sido la representación de supuestos causales mediante DAGs (*Directed Acyclic Graphs* o grafos acíclicos dirigidos). En términos de causalidad, los DAGs sirven para representar las relaciones causales que creemos que existen entre las variables de interés. Al dibujar un diagrama de este tipo, estamos plasmando todo el conocimiento teórico y empírico previo que poseemos sobre un determinado problema [26].

En un DAG, existen una serie de componentes:

- Los nodos representan variables aleatorias [25].
- Las flechas indican relaciones causales. Por ejemplo, si existe una flecha desde la variable X hacia la variable Y , implica que X tiene un efecto causal sobre Y . También es importante mencionar que si no existe una flecha entre dos nodos significa que hay una suposición de independencia condicional en el modelo causal [25].
- Los caminos representan secuencias de nodos [25].

En la siguiente imagen, se muestra un ejemplo de grafo causal, donde se puede asumir que W causa X e Y , y que X causa Y . Además, se puede observar el camino $W \rightarrow X \rightarrow Y$ [27].

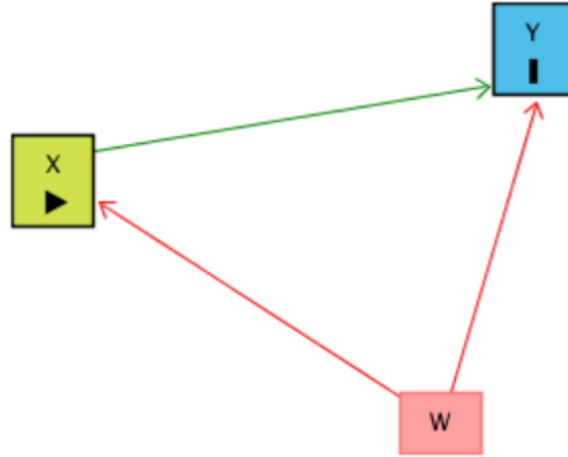


Figura 3.2: Ejemplo grafo causal

3.5.3. Criterio de *Backdoor*

El «criterio de *backdoor*» es una de las herramientas clave en la inferencia causal moderna, y permite determinar si es posible estimar el efecto causal de una variable de tratamiento T sobre un resultado Y , ajustando adecuadamente por variables confusoras [25].

Un conjunto de variables Z cumple el criterio de backdoor si:

- Ninguna de las variables en Z es un descendiente del tratamiento T [25].
- Z bloquea todas las rutas de *backdoor* entre T y Y , es decir, todos los caminos que conectan a T con Y a través de variables anteriores comunes [25].

Cuando se satisface este criterio, el efecto causal de T sobre Y se puede estimar con datos observacionales mediante:

$$P(Y \mid do(T)) = \sum_Z P(Y \mid T, Z) P(Z) \quad (3.1)$$

Las «rutas backdoor» se consideran importantes, ya que crean correlaciones sistemáticas y no causales entre la variable causal de interés y el resultado que se desea estudiar. En términos de regresión, estas rutas introducen sesgo por variables omitidas, y el sesgo puede ser tan grave que incluso invierta el signo del efecto estimado [26].

Evidentemente, para analizar la relación directa causa–efecto entre un tratamiento y un resultado, resulta necesario inhabilitar (cerrar) las rutas alternativas (*backdoor*). Para ello, se deben tener en cuenta principalmente dos aspectos:

1. **Condicionar sobre una confusora:** si una variable está causando tanto el tratamiento como el resultado, se puede cerrar esa ruta controlándola en el análisis. Esto puede hacerse mediante emparejamiento, regresión o incluso estratificación [26].

2. **Evitar condicionar sobre *colliders*:** si una ruta incluye un *collider* (una variable que recibe flechas desde dos direcciones, como $A \rightarrow C \leftarrow B$) entonces esa ruta ya está cerrada. Sin embargo, condicionar sobre un *collider* la abre, introduciendo un sesgo. Por tanto, es importante evitar este tipo de condición [26].

Se sabe que cuando se han cerrado todas las rutas de backdoor, se dice que el diseño satisface el criterio de backdoor. Y si se cumple, entonces se ha aislado un efecto causal. Esto es un enfoque que necesita de gran comprensión, tanto de los datos como del grafo causal, ya que la estimación del efecto podría estar comprometida [26].

3.5.4. Métodos de estimación del Efecto Causal

Para estimar efectos causales en contextos observacionales, este Trabajo de Fin de Grado se apoya en metodologías fundamentadas en regresión, emparejamiento y diseños cuasi-experimentales como variables instrumentales y discontinuidades. Estas familias metodológicas permiten identificar efectos bajo diferentes supuestos, como puede ser el control de confusores observables.

Métodos de Regresión

La regresión es el método más común para estimar la relación entre dos variables mientras se controla por otras. Su utilidad reside en que permite ajustar por variables confusoras al incluirlas como predictores en el modelo, lo que bloquea rutas de *backdoor* y facilita una interpretación causal bajo ciertos supuestos [28].

Este método busca encontrar la mejor línea que relacione una variable independiente X con una dependiente Y , minimizando el error de predicción. Cuando se incluye más de una variable explicativa, cada coeficiente estima la relación entre una variable y el resultado, condicionando por el resto [28].

Aunque originalmente es una técnica de predicción estadística, la regresión puede tener interpretación causal si se han identificado correctamente los confusores y se ha especificado bien el modelo.

Emparejamiento (*Matching*)

El emparejamiento es una técnica ampliamente utilizada para estimar efectos causales, especialmente cuando se dispone de información detallada sobre las variables que influyen tanto en el tratamiento como en el resultado. Su objetivo es construir grupos de comparación que sean similares en función de un conjunto de covariables, con el fin de cerrar caminos de *backdoor* y así aproximar una estimación no sesgada del efecto causal [28].

De manera intuitiva, el emparejamiento busca que las unidades tratadas se comparen únicamente con unidades no tratadas que presenten características observables similares. Esto lo diferencia de la regresión, ya que no depende de supuestos de linealidad para modelar la relación entre variables. Además, ofrece flexibilidad en la estimación de distintos tipos de efectos, como el efecto promedio en la población o el efecto promedio sobre los tratados [28].

Variables instrumentales

Cuando no es posible cerrar todos los caminos de confusión entre el tratamiento y el resultado, las *variables instrumentales* (IV) ofrecen un enfoque alternativo para identificar el efecto causal. En lugar de intentar eliminar toda la variación no deseada usando controles, encuentran una fuente de variación que permite aislar solo el camino causal que nos interesa [28].

Supongamos que queremos estimar el efecto de un tratamiento T sobre un resultado Y , pero existe una variable no observada U que influye en ambos. En este caso, cualquier estimación directa estaría sesgada [28].

La estrategia de IV consiste en encontrar una variable Z que cumpla dos condiciones: (a) afecte al tratamiento T (relevancia) y (b) no tenga un efecto directo sobre el resultado Y (validez). Es decir, todos los caminos abiertos de Z a Y deben pasar necesariamente por T . Si estas condiciones se cumplen, Z puede actuar como un «instrumento» que aísle la parte de T libre de *backdoors*, permitiendo estimar su efecto sobre Y [28].

Tiene una implementación en dos etapas [28]:

- **Primera etapa:** predecir el tratamiento T utilizando el instrumento Z .
- **Segunda etapa:** usar esa predicción de T como variable explicativa de Y .

Además, se basa en dos supuestos clave ya mencionados. El primero es la relevancia, ya que el instrumento debe estar correlacionado con el tratamiento. Si no, la estimación falla o es inestable. El segundo es la validez, este se fundamenta en que el instrumento no debe estar correlacionado con el resultado por ningún camino que no pase por el tratamiento [28].

Este método resulta interesante si se sospechan confusores no observables o hay una fuente de variación que puede considerarse exógena [28].

Regresión Discontinua (RDD)

El «Diseño de Regresión Discontinua (*Regression Discontinuity Design, RDD*)» es una estrategia de inferencia causal que aprovecha contextos en los que la asignación de un tratamiento depende de un umbral (*cutoff*) en una variable continua de asignación (*running variable*). Este enfoque permite comparar unidades muy similares a ambos lados del punto de corte, de modo que las diferencias en el resultado puedan atribuirse con mayor confianza al tratamiento [28].

Los componentes principales de este diseño son:

- **Variable de asignación (*running variable*):** variable continua que determina quién recibe el tratamiento [28].
- **Punto de corte (*cutoff*):** valor de la variable de asignación que establece la elegibilidad para el tratamiento [28].
- **Tratamiento:** generalmente binario, asignado a quienes están por encima (o por debajo) del punto de corte [28].
- **Banda de comparación (*bandwidth*):** rango alrededor del punto de corte en el cual las unidades se consideran comparables [28].

La lógica central del RDD puede resumirse en tres pasos: (a) modelar cómo la variable de asignación influye en el resultado en ausencia del tratamiento, (b) focalizarse en las observaciones próximas al punto de corte, y (c) comparar a quienes estuvieron justo por encima con quienes estuvieron justo por debajo [28].

El RDD ofrece ventajas destacadas: permite estimar efectos causales sin necesidad de controlar directamente por confusores observables y resulta especialmente útil en contextos donde la asignación del tratamiento sigue reglas claras y transparentes [28].

3.5.5. Validación del efecto estimado y evaluación de su robustez

Una vez obtenido un estimador del efecto causal, es fundamental valorar si dicho efecto es robusto, es decir, si se mantiene bajo distintos supuestos o si es altamente sensible a decisiones concretas.

En el presente trabajo, no se implementaron esas técnicas de validación debido a las características del entorno desarrollado. Sin embargo, se incorporó una validación automatizada basada en lenguaje natural, a través de un agente de inferencia causal entrenado para evaluar los resultados obtenidos.

Aunque esta aproximación no reemplaza métodos estadísticos clásicos de robustez, sí ofrece una primera validación cualitativa razonada. Así, permite alertar al usuario sobre posibles debilidades del análisis causal realizado, facilitando una interpretación más crítica de los resultados.

CAPÍTULO 4

Metodología

En este capítulo se describe la metodología seguida para el desarrollo del proyecto, incluyendo la planificación general, las herramientas de trabajo, el enfoque de trabajo aplicado y la organización del proceso. El objetivo es proporcionar una visión estructurada de cómo se abordó la resolución del problema mediante el uso de agentes basados en LLM.

4.1 Enfoque general

El enfoque metodológico seguido en este proyecto combina el desarrollo de software con técnicas de inteligencia artificial centradas en el uso de modelos de lenguaje (LLM) integrados en arquitecturas de agentes. En particular, se ha construido un agente basado en el modelo *Gemini 2.0 Flash* (modelo de *Google Generative AI*) [29].

El sistema se estructura siguiendo un esquema modular y reutilizable, donde el modelo de lenguaje actúa como núcleo de razonamiento, mientras que diversas herramientas complementan sus capacidades. Este enfoque está inspirado en los principios del prompting tipo *ReAct* (*Reasoning + Acting*), permitiendo que el modelo combine razonamiento verbal con acciones concretas, como invocar funciones externas.

El flujo metodológico general se desarrolla en los siguientes pasos:

1. **Inicialización del modelo LLM** Se configura un modelo de lenguaje (*gemini-2.0-flash-001*) mediante la clase `ChatGoogleGenerativeAI`, con un valor configurable de `temperature` para ajustar la creatividad de las respuestas.
2. **Definición de herramientas** Se construyen herramientas personalizadas utilizando la clase `Tool` de *LangChain*. Cada función tiene un propósito y está documentada con descripciones en lenguaje natural, de forma que el agente puede decidir autónomamente cuándo utilizarlas. Cada herramienta representa una acción disponible.
3. **Orquestación del agente** El agente se inicializa mediante el método `initialize_agent`, usando el tipo `chat-zero-shot-react-description`. Esta arquitectura permite al modelo analizar instrucciones complejas, razonar paso a paso y decidir qué herramientas usar para alcanzar una solución.
4. **Interacción iterativa** El sistema sigue un flujo interactivo dividido en fases que permiten al usuario explorar sus datos de manera guiada y flexible. El proceso se desarrolla del siguiente modo:

- a) **Carga de datos:** El usuario comienza subiendo una base de datos en formato CSV o XLSX.
- b) **Selección del análisis:** Una vez cargada la base de datos, el usuario elige el tipo de análisis que desea realizar, seleccionando entre tres secciones disponibles en la interfaz:
 - **Resumen estadístico:** el agente analiza automáticamente el conjunto de datos y genera *insights* relevantes, como medidas descriptivas, presencia de valores nulos o correlaciones destacadas.
 - **Visualizaciones:** el usuario plantea una petición en lenguaje natural (por ejemplo, “muéstrame un histograma de la variable ingresos”), y el agente genera una gráfica adecuada utilizando la herramienta encargada de ejecutar código Python.
 - **Análisis de causalidad:** esta sección permite iniciar un proceso más extenso, en el que el agente guía al usuario en la identificación de relaciones causales potenciales dentro del conjunto de datos.
- c) **Razonamiento y ejecución:** En cada sección, el agente utiliza el modelo LLM para razonar sobre la petición y decidir qué herramientas específicas debe emplear. Estas herramientas se invocan automáticamente, y sus resultados son integrados en el razonamiento general del modelo.
- d) **Respuesta generada:** Finalmente, el sistema devuelve al usuario una salida coherente y estructurada, que puede consistir en texto explicativo, tablas, gráficas o diagnósticos causales, dependiendo de la sección elegida y la complejidad de la consulta.

4.2 Fases del Proyecto

El desarrollo del proyecto se estructuró en varias fases que permiten abordar de forma sistemática el diseño, implementación y validación de un agente LLM, cuyo objetivo principal es actuar como asistente de análisis de datos.

Análisis del problema

En esta etapa inicial se identificó la necesidad de una solución que facilitara el análisis de datos a usuarios no expertos, permitiéndoles interactuar mediante lenguaje natural. El objetivo era reducir la barrera técnica asociada al uso de herramientas de análisis tradicionales.

Durante esta fase se realizó una revisión del estado del arte en tres ámbitos clave:

- Modelos de lenguaje y sus capacidades para interpretar datos estructurados.
- Agentes con herramientas, como los que siguen el enfoque ReAct.
- Frameworks de orquestación como *LangChain*, que permiten combinar razonamiento y ejecución de tareas.

Diseño del enfoque

A partir del análisis anterior, se definió un diseño técnico centrado en tres pilares:

- **Interfaz accesible:** mediante *Streamlit* [5], para que cualquier usuario pudiera subir datos y elegir el tipo de análisis deseado (resumen, visualización o causalidad).
- **Evolución del modelo LLM:** se evaluaron diferentes alternativas:
 - *Mistral-small* (MistralAI) [30]: primera opción, descartada por su lentitud y bajo rendimiento.
 - Modelos *Hugging Face* vía API [31]: presentaban limitaciones por cuota y disponibilidad.
 - *Gemini 2.0 Flash* (Google) [29]: opción final adoptada por su rapidez, precisión y buena integración con herramientas externas.

Preparación del entorno

Se configuró el entorno de desarrollo con los elementos necesarios para la ejecución del sistema:

- Creación de un entorno virtual con dependencias gestionadas mediante `requirements.txt`.
- Configuración de credenciales para el acceso al modelo *Gemini*.
- Instalación de bibliotecas clave: *LangChain*, *dotenv*, *pandas*, *DoWhy*, etc.
- Incorporación de bases de datos de prueba para verificar los análisis generados por el agente.

Implementación iterativa

La construcción del agente se realizó de forma incremental:

- Se integró el modelo LLM y se configuró la arquitectura del agente reactivo.
- Se añadió como herramienta principal la ejecución de código Python.
- Posteriormente se incorporó soporte para inferencia causal, permitiendo analizar relaciones entre variables. Para ello, se diseñaron varias herramientas que debía usar el agente.
- Tras cada iteración se realizaron pruebas funcionales para validar el comportamiento del sistema y mejorar la formulación de *prompts* o la lógica de uso de herramientas.
- Se ajustaron parámetros del modelo (como `temperature = 0`) para maximizar la fiabilidad de las respuestas.

Validación funcional

Finalmente, se realizaron pruebas manuales con datos reales para evaluar el rendimiento del agente. En particular, se verificó:

- La capacidad del sistema para interpretar las instrucciones dadas en los *prompts* y seleccionar la herramienta adecuada.

- La generación correcta de análisis estadísticos, visualizaciones y diagnósticos causales.
- La identificación de limitaciones del modelo (como posibles alucinaciones o errores de interpretación) y problemas técnicos (latencias o errores en el procesamiento de datos).

4.3 Planificación temporal

El desarrollo del proyecto se extendió durante aproximadamente 18 semanas, desde el 27 de marzo de 2025 hasta el 1 de agosto de 2025, siguiendo una planificación flexible pero estructurada en etapas sucesivas.

La distribución temporal aproximada puede dividirse en las siguientes fases:

Fase 1: Investigación y exploración (Semanas 1-3)

Durante las primeras semanas se llevó a cabo una revisión intensiva de soluciones ya existentes relacionadas con asistentes inteligentes basados en LLM. Se analizaron enfoques implementados por otros proyectos, se exploraron ideas aplicables a la creación de agentes interactivos, y se evaluaron diferentes modelos de lenguaje.

En esta etapa también se realizaron pruebas comparativas entre distintos LLM (como Mistral y modelos de Hugging Face), con el objetivo de identificar el modelo más adecuado para las tareas previstas.

Fase 2: Pruebas de viabilidad e implementación inicial (Semanas 4-10)

Una vez identificado el modelo más eficaz, se comenzaron a realizar pruebas funcionales sobre su integración mediante LangChain. Al obtener buenos resultados de rendimiento y compatibilidad con herramientas externas, se dio paso a la construcción inicial de la aplicación.

La primera funcionalidad desarrollada fue la sección de resumen estadístico, que permitía al agente generar descripciones y diagnósticos automáticos sobre un conjunto de datos.

En esta fase también se añadió una segunda funcionalidad: la generación automática de gráficos personalizados a partir de peticiones en lenguaje natural.

Fase 3: Implementación del análisis causal (Semanas 11-14)

Esta fue una de las fases más complejas, que comenzó con la investigación de las librerías, pasos metodológicos y técnicas necesarias para aplicar inferencia causal de forma automatizada.

Tras diseñar un flujo lógico viable, se construyeron herramientas específicas para cada parte del proceso causal: selección de tratamiento, generación del DAG, selección del método de estimación, estimación del efecto y validación.

En esta fase ya se comenzó a redactar la memoria, sobre todo los primeros puntos.

Fase 4: Validación, correcciones y redacción (Semanas 15-18)

Finalmente, se dedicó un bloque de semanas a probar el comportamiento general del agente en distintos escenarios. Se identificaron y corrigieron errores funcionales, se refinaron los prompts de interacción y se ajustaron detalles de la interfaz. Con ello, se llevó a cabo la mayor parte de la redacción del proyecto.

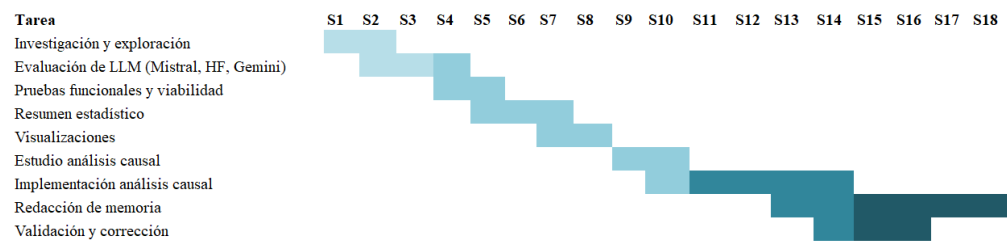


Figura 4.1: Diagrama de Gantt

4.4 Herramientas utilizadas

El desarrollo del proyecto ha requerido una combinación de tecnologías para la implementación del agente, la interfaz de usuario, el análisis causal y la ejecución de código Python desde lenguaje natural. Las herramientas pueden agruparse en las siguientes categorías:

Modelos de lenguaje y orquestación

- **Google Gemini 2.0 Flash:** modelo LLM empleado como núcleo de razonamiento del agente. Se accede mediante la API de Google Generative AI a través de la clase ChatGoogleGenerativeAI.
- **LangChain:** biblioteca utilizada para la orquestación del agente, generación de herramientas, gestión de memoria del agente y control de flujo entre instrucciones, razonamientos y acciones.

Personalización del agente

- **Herramientas LangChain (Tool):** se implementaron funciones encapsuladas como herramientas, accesibles desde el agente para tareas como ejecución de código Python o tareas más enfocadas a la inferencia causal, como la generación de DAGs, estimación de efectos y validación estadística.
- **DoWhy:** biblioteca de Python utilizada para fomentar el razonamiento y análisis causal. Ofrece métodos para estimar efectos, realizar intervenciones y analizar contrafactuales, además de herramientas para validar supuestos y hacer la inferencia más robusta [32].
- **Scikit-learn, pandas, numpy, seaborn, matplotlib:** bibliotecas auxiliares para cálculo estadístico, visualización y soporte a los análisis realizados por el agente.

Interfaz de usuario

- **Streamlit**: biblioteca para el desarrollo de la interfaz web interactiva que permite a los usuarios subir sus propios datasets, navegar entre secciones y obtener respuestas directamente en el navegador.
- **CSS personalizado**: se utilizó código CSS dentro de Streamlit para personalizar visualmente la aplicación, mejorando la experiencia de usuario.

Gestión del entorno y configuración

- **Entorno virtual** (venv): se creó un entorno virtual para aislar las dependencias del proyecto.
- **Gestión de dependencias**: se emplearon pip y el archivo `requirements.txt`.
- **Manejo de variables sensibles** (dotenv): para la gestión segura de claves API.
- **Reintentos controlados y manejo de errores HTTP**: especialmente en la interacción con la API de Gemini, gestionando límites de uso y respuestas de error.

Ecosistema de trabajo

El desarrollo del proyecto se ha realizado en *Visual Studio Code* como entorno de desarrollo integrado (IDE), aprovechando su soporte para Python [33]. El entorno de ejecución se ha configurado con Python 3.10.16 en un entorno virtual (venv), gestionando las dependencias a través de pip y el archivo `requirements.txt`.

Para la gestión segura de credenciales y variables de configuración se ha empleado la biblioteca `python-dotenv` [34], y la ejecución de pruebas y depuración se ha realizado directamente desde el IDE.

CAPÍTULO 5

Desarrollo

En este capítulo se detalla el desarrollo completo del sistema construido: un asistente inteligente de análisis de datos que utiliza modelos de lenguaje natural para interactuar con el usuario y operar sobre conjuntos de datos subidos por este.

Se describe a continuación la arquitectura general, el funcionamiento interno del agente inteligente, las herramientas desarrolladas y el flujo de interacción del usuario a través de la interfaz web. También se explican los principales retos técnicos abordados, así como las soluciones implementadas durante el proceso.

5.1 Arquitectura general del sistema

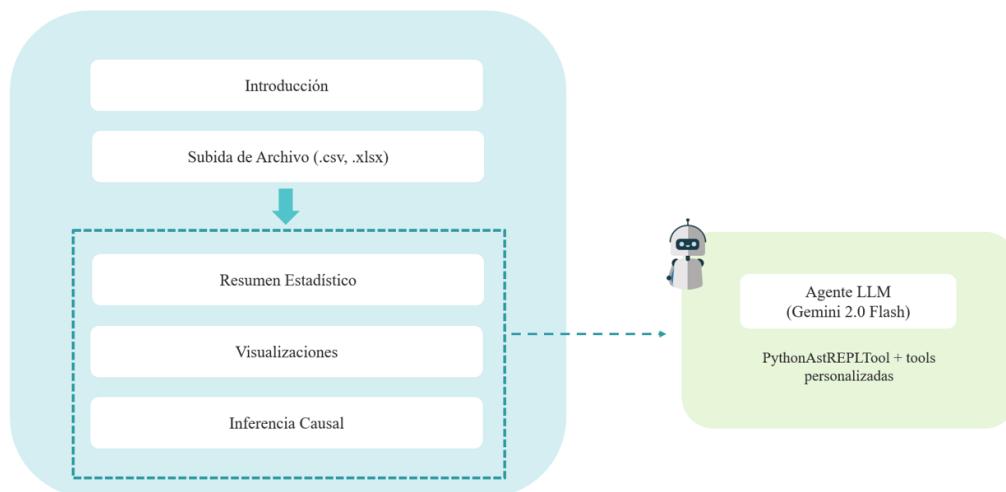


Figura 5.1: Arquitectura diseñada

La solución desarrollada se estructura como una aplicación web interactiva orientada a facilitar el análisis exploratorio, visual y causal de datos mediante un agente inteligente basado en un LLM de última generación. La arquitectura general, como se puede observar en la Figura 5.1, la solución desarrollada se organiza en torno a dos bloques principales: la interfaz de usuario, donde se ejecuta el flujo de interacción, y el agente LLM, responsable del razonamiento de las respuestas.

Interfaz en Streamlit

La aplicación se articula a través de un archivo central `main.py`, que gestiona la navegación entre cinco secciones o páginas principales:

- **Introducción:** Página inicial de bienvenida con explicación del propósito del proyecto.
- **Subida de archivos:** Permite al usuario cargar archivos `.csv` o `.xlsx` que serán analizados posteriormente.
- **Resumen estadístico:** Al seleccionar esta sección, se activa el agente inteligente, que genera descripciones detalladas del conjunto de datos, incluyendo perfilado, significados de columnas, valores nulos, duplicados, correlaciones, variables sugeridas y posibles variables objetivo.
- **Visualizaciones:** El usuario puede formular peticiones en lenguaje natural para crear gráficos. El agente interpreta la solicitud y genera código Python válido con `matplotlib` o `seaborn`, que se ejecuta automáticamente.
- **Inferencia causal:** Esta sección contiene todo el flujo causal guiado: desde la selección de tratamiento y variable objetivo, pasando por la generación automática del DAG, estimación del efecto causal con diferentes métodos, hasta la validación del resultado.

Agente LLM (bloque de procesamiento)

El razonamiento avanzado es realizado por un agente LLM basado en el modelo *Gemini 2.0 Flash* de Google. Este agente está configurado para recibir instrucciones, interpretar datos, generar respuestas y crear código Python válido, integrando además herramientas personalizadas para tareas específicas.

El agente es invocado desde las secciones donde se requiere razonamiento automatizado (resumen, visualización e inferencia causal), mientras que las demás secciones funcionan de manera estática.

Flujo de funcionamiento

El flujo completo comienza cuando el usuario carga un dataset en la aplicación. A continuación, puede navegar por las diferentes secciones y realizar peticiones relacionadas con el resumen, la visualización o el análisis causal. Las instrucciones introducidas se envían al agente LLM, que interpreta el contexto y responde en consecuencia: bien generando texto explicativo, bien seleccionando y ejecutando la herramienta más adecuada, o razonando directamente mediante lenguaje natural. Finalmente, la aplicación muestra los resultados al usuario de manera interactiva. En el caso de las visualizaciones, se ofrece además la opción de exportarlas en formato PNG.

Finalmente, cabe destacar que en el Anexo II se incluye el código fuente de la interfaz principal, que permiten profundizar en los detalles técnicos del desarrollo.

5.2 Arquitectura del agente

El agente construido es el núcleo lógico del sistema, encargado de ejecutar los razonamientos sobre los datos y generar respuestas en lenguaje natural. Para ello, combina

un modelo de lenguaje (LLM), un conjunto de herramientas especializadas, mecanismos de memoria contextual y acceso directo a un DataFrame cargado.

A continuación, se muestra un esquema en la Figura 5.2 que resume la arquitectura funcional del agente:

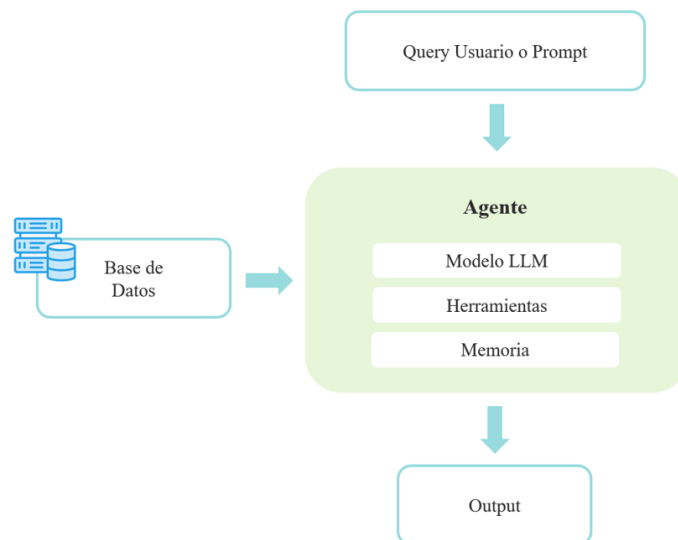


Figura 5.2: Composición del agente

5.2.1. Componentes Principales del Agente

Entradas al Agente

El agente recibe dos fuentes principales de información. Por un lado, el **prompt o consulta del usuario**, especificado en lenguaje natural desde la interfaz web o en base a los *prompts* predefinidos. Estas instrucciones pueden abarcar desde peticiones de hipótesis causales hasta solicitudes de generación de gráficos.

Por otro lado, la **base de datos** cargada por el usuario (en formato `.csv` o `.xlsx`), que se transforma internamente en un objeto `DataFrame` de *pandas* [35]. Este se almacena en la sesión de Streamlit mediante la variable `st.session_state["df"]`, lo que permite su acceso en todo momento durante la ejecución del agente.

Núcleo del Agente

Una vez recibidas las entradas, el agente activa su lógica de procesamiento, construida sobre una arquitectura modular con distintos componentes.

El **modelo LLM (BaseLLM / CausalLLM)** constituye el núcleo de razonamiento. La clase `BaseLLM` centraliza parámetros comunes como `temperature = 0.0`, que garantiza salidas deterministas, y `chunk_size = 1`, que anticipa el manejo de entradas extensas mediante fragmentación. Sobre esta base se construye la clase `CausalLLM`, que implementa la interacción con el modelo generativo *Gemini 2.0 Flash* de Google, accedido mediante la biblioteca `langchain_google_genai` [36].

Para la generación de respuestas, esta clase combina el uso de `PromptTemplate`, que estructura y parametriza las peticiones al modelo [37], con `LLMChain`, encargado de conectar el *prompt* con el modelo y producir la salida final [38].

El **CausalAgentFactory** actúa como factoría de agentes. Su función es preparar el entorno de ejecución del agente: reinicia el índice del DataFrame cargado, inicializa la memoria conversacional, configura y registra las herramientas especializadas, y finalmente crea la instancia del agente mediante `initialize_agent` con el tipo `ZERO_SHOT_REACT_DESCRIPTION`.

En paralelo, el **CausalAgentExecutor** encapsula la instancia final del agente y expone un método `run(prompt)`, que simplifica la ejecución de instrucciones y permite integrar fácilmente el agente con la interfaz de usuario y con otros módulos del sistema.

El agente dispone de un conjunto de **herramientas especializadas** que se invocan de manera automática cuando la tarea lo requiere. Estas incluyen la identificación de efectos causales, la generación de gráficos DAG en formato DOT, la estimación de efectos con métodos de *DoWhy*, la validación de robustez, la ejecución de análisis Python sobre el DataFrame, así como diagnósticos de desequilibrios y *propensity scores*. El tipo de agente `ZERO_SHOT_REACT_DESCRIPTION` permite al modelo decidir de forma autónoma qué herramienta es la más adecuada en cada momento.

Para asegurar coherencia entre las interacciones, se implementa una **memoria conversacional** mediante `ConversationBufferMemory`. Esta conserva de forma íntegra el historial de la sesión, lo que facilita que el agente recupere selecciones anteriores, hipótesis formuladas o variables tratadas sin necesidad de reprocesar información [new23].

Salida del Agente

La ejecución concluye con una salida generada según el contexto y la naturaleza de la petición. Esta salida puede consistir en texto explicativo, en fragmentos de código Python ejecutados en tiempo real, en visualizaciones gráficas (como histogramas, diagramas de caja o DAGs), o bien en estructuras JSON con resultados cuantitativos, tales como estimaciones de efectos o validaciones estadísticas.

Cuando la salida es de tipo JSON, el agente la transforma automáticamente en un formato más comprensible para el usuario, integrando la explicación con la interfaz de la aplicación.

5.2.2. Flujo de Ejecución Interno del Agente

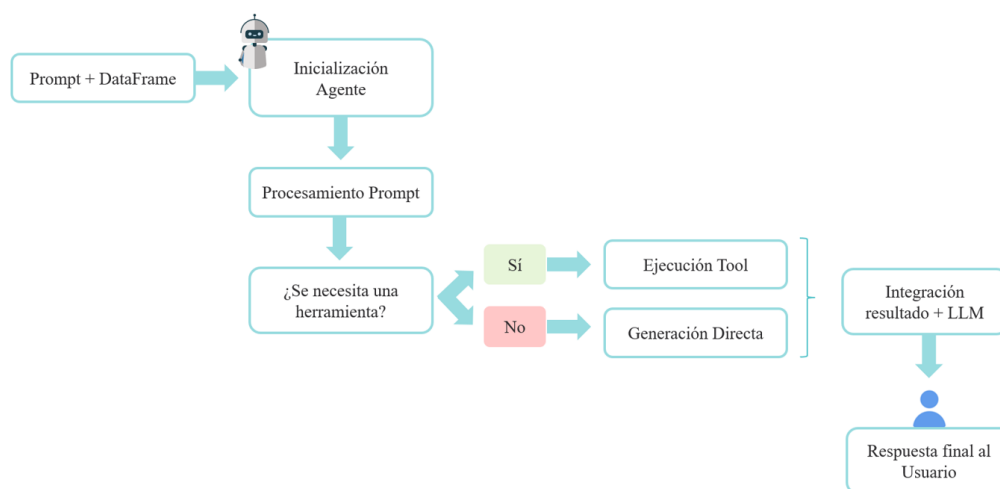


Figura 5.3: Flujo de ejecución interno del agente

El funcionamiento interno del agente sigue un flujo estructurado, mostrado en la Figura 5.3. Este mismo va desde la entrada del usuario hasta la generación de una respuesta precisa. Este recorrido puede dividirse en los siguientes pasos secuenciales:

Inicialización del agente

Cuando el agente es creado a través de `CausalAgentFactory.create_agent(df)`, se produce la siguiente secuencia:

- Se carga el modelo LLM mediante la clase `CausalLLM`.
- Se genera una instancia de memoria conversacional (`ConversationBufferMemory`) para registrar el contexto de interacción.
- Se agregan todas las herramientas disponibles (`causal_tools`) junto con las herramientas dinámicas.

Toda esta inicialización queda integrada mediante la función `initialize_agent()`.

Recepción del Prompt

Cuando el agente recibe un *prompt* a través del método `agent.run(prompt)`, se activa un flujo de procesamiento que sigue la lógica siguiente:

- El agente analiza el texto recibido en lenguaje natural.
- Según el tipo de entrada (pregunta, instrucción o análisis), decide si basta con generar texto o si debe invocar alguna herramienta.
- Si la tarea lo requiere (por ejemplo, estimar un efecto, generar un DAG o ejecutar código), selecciona automáticamente la herramienta más adecuada.

Transformación y ejecución del Prompt

En esta etapa, el *prompt* recibido se transforma y ejecuta mediante los siguientes pasos:

- Se combina con sus variables dinámicas (si existen) a través de un `PromptTemplate`.
- Se ejecuta un `LLMChain`, que convierte el *prompt* parametrizado en una consulta estructurada para el modelo.
- Cuando es necesario el uso de herramientas, `LangChain` invoca la función correspondiente, la ejecuta y devuelve el resultado al LLM.
- La ejecución puede repetirse en ciclos hasta que el agente considera que ha producido una respuesta final satisfactoria.

Generación de salida

Finalmente, el agente produce una respuesta en función de la petición original. Esta salida puede adoptar distintas formas (texto explicativo, código, visualizaciones o resultados cuantitativos), y es devuelta al sistema para mostrarse al usuario a través de la interfaz de *Streamlit*.

5.2.3. Ciclo de razonamiento ReAct y toma de decisiones autónoma

El núcleo del comportamiento del agente está basado en el paradigma *ReAct* (*Reasoning + Acting*). Este patrón, implementado a través del tipo `ZERO_SHOT_REACT_DESCRIPTION`, permite que el agente resuelva tareas siguiendo una secuencia de pasos lógicos.

A diferencia de los sistemas tradicionales de pregunta-respuesta, donde el modelo responde con una respuesta única, el enfoque ReAct simula un ciclo humano de pensamiento. El agente recibe una consulta del usuario, reflexiona sobre la tarea, decide si debe actuar, ejecuta acciones externas si lo considera necesario y, finalmente, emite una respuesta.

Cada iteración de este proceso está compuesta por bloques estructurados de la siguiente forma:

- **Thought:** el modelo expone su razonamiento sobre cómo abordar el problema.
- **Action:** selecciona una herramienta concreta para ejecutar la tarea.
- **Action Input:** define los parámetros o datos que se pasarán a la herramienta elegida.
- **Observation:** recibe el resultado de la herramienta invocada.
- **Final Answer:** produce la respuesta final, integrando el contexto acumulado.

Este ciclo puede repetirse varias veces (alternando entre *Thought*, *Action* y *Observation*) hasta que el agente decide que ha alcanzado una conclusión adecuada.

Un ejemplo ilustrativo se obtiene al ejecutar el cálculo de filas duplicadas en un `DataFrame`:

```
> Entering new AgentExecutor chain...
Thought: Okay, I need to analyze the DataFrame `df` for duplicate rows...
Action: python_repl_ast
Action Input:
```python
duplicates = df.duplicated().sum()
total_rows = len(df)
...
print(f"Número de filas duplicadas: {duplicates}")
print(f"Porcentaje de filas duplicadas: {percentage_duplicates:.2f}%")
...
Número de filas duplicadas: 0
Porcentaje de filas duplicadas: 0.00%

Observation: No se encontraron filas duplicadas.
Thought: The analysis revealed that no duplicate rows were found.
Final Answer: El análisis del conjunto de datos no reveló la presencia de
→ filas duplicadas.

Finished chain.
```

La implementación de este patrón ofrece múltiples ventajas: permite descomponer peticiones complejas en subtareas manejables, dota de flexibilidad al agente al apoyarse

en herramientas externas y facilita la adaptabilidad al contexto, ya que el modelo puede ajustar sus pasos en función de lo que reciba durante la interacción.

En términos internos, la funcionalidad ReAct se basa en tres principios fundamentales:

- Razonamiento sin ejemplos previos (*zero-shot*).
- Uso de las descripciones de herramientas como referencia para decidir cuál aplicar.
- Alternancia entre la generación de texto y la ejecución de funciones.

En el Anexo V se presenta el código fuente completo del agente desarrollado, incluyendo las clases principales, la definición de herramientas y la lógica de orquestación.

## 5.3 Navegación y estructura de la aplicación

---

En este punto, se describen las distintas páginas que componen la aplicación y su relación con el flujo general del sistema.

### 5.3.1. Subida de archivos

Una de las primeras interacciones que debe realizar el usuario en la aplicación es la carga de un conjunto de datos en formato `.csv` o `.xlsx`. Esta parte resulta indispensable, ya que el agente necesita disponer de los datos para poder operar.

La implementación de este módulo se lleva a cabo mediante la interfaz de *Streamlit*, utilizando su componente `st.file_uploader`, que permite seleccionar un archivo desde el sistema local. Además, se ofrece una funcionalidad adicional para seleccionar archivos previamente subidos durante sesiones anteriores.

El comportamiento se resume en dos flujos complementarios:

- **Selección de archivo previo:** si existen archivos en la carpeta de trabajo, se muestran mediante un desplegable (`st.selectbox`). La función `list_all_datasets()` (véase Anexo IV) busca archivos `.csv` o `.xlsx` disponibles y los presenta como opciones. Si el usuario selecciona uno de ellos, su ruta se guarda en la variable de sesión `st.session_state["file_path"]`.
- **Carga de un nuevo archivo:** si se desea subir un archivo nuevo, el usuario puede hacerlo a través del botón `st.file_uploader`. Una vez elegido el archivo, al pulsar el botón *Siguiente*, se ejecuta la función `save_uploaded_file()` (véase Anexo IV), que guarda el archivo en el directorio de trabajo y registra su ruta en la sesión para que pueda ser utilizada por las demás secciones de la aplicación.

A nivel técnico, el archivo cargado se convierte más adelante en un objeto `pandas.DataFrame`, accesible a través del estado de la sesión. Este diseño permite al usuario reutilizar archivos anteriores y cargar nuevos, asegurando que el *dataset* siempre esté disponible para el agente en el resto del flujo.

Finalmente, se muestra una captura de pantalla de la sección de “Subida de archivo”:



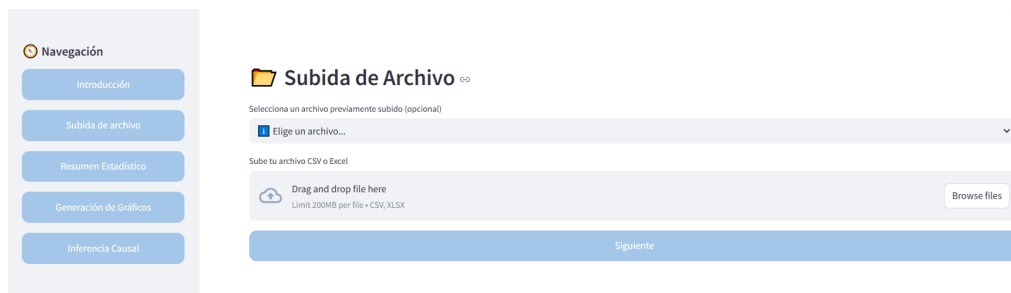


Figura 5.4: Interfaz de la sección “Subida de archivo”

Para una revisión más detallada de la implementación de este módulo, se recomienda consultar el Anexo III, donde se incluye el código fuente correspondiente.

### 5.3.2. Resumen Estadístico

La sección “Resumen Estadístico” representa el primer punto de contacto entre el usuario y el agente dentro del flujo de análisis. Su objetivo es realizar una exploración automatizada del conjunto de datos cargado, proporcionando un diagnóstico en lenguaje natural. Detrás de esta funcionalidad existe una lógica que combina la ingeniería de *prompts*, el uso de la herramienta de ejecución de código Python, la gestión de errores y el control del contexto conversacional.

#### Inicialización del agente y flujo

Una vez que el usuario carga un archivo válido (.csv o .xlsx), este se transforma en un objeto `pandas.DataFrame` y se almacena en la variable `st.session_state["df"]`. Primero se realiza una toma de contacto mostrando las cinco primeras filas del *dataset*. Tras esta previsualización, el usuario puede inicializar el agente mediante el botón correspondiente, lo que ejecuta la clase `CausalAgentFactory`. Una vez inicializado, el agente puede comenzar a responder consultas de análisis estadístico a través de *prompts* predefinidos.

#### Diseño y estructura de los *prompts*

Cada subanálisis se activa mediante una plantilla de *prompt* que se envía al agente. Estos *prompts* están diseñados para guiar al LLM en la generación de respuestas estructuradas. Sus características clave son:

- **Contexto explícito:** cada *prompt* define claramente el “rol” del agente, lo que guía el tipo de razonamiento esperado.
- **Objetivo claro:** se establece qué tipo de análisis se debe realizar (por ejemplo, detectar duplicados o proponer una variable objetivo).
- **Reglas estrictas:** los *prompts* incluyen restricciones detalladas para evitar que el modelo genere código innecesario, mencione términos técnicos (como `DataFrame`, `NaN`, etc.) o invente datos.
- **Formato esperado:** algunos *prompts* especifican el formato exacto de salida, lo que mejora la consistencia en las respuestas.
- **Ejemplos:** en ciertos casos se utiliza *few-shot prompting*, proporcionando ejemplos de salida que sirven de guía al agente.

### Uso de herramientas y razonamiento guiado

Aunque el agente puede comportarse como un generador de texto, una parte clave de su inteligencia reside en su capacidad para decidir cuándo debe ejecutar código Python sobre la base de datos cargada por el usuario. Esta ejecución no la realiza directamente el modelo, sino a través de una herramienta auxiliar que le permite interactuar con código Python.

La herramienta empleada en este proyecto es `PythonAstREPLTool`, un componente proporcionado por `LangChain` que permite al agente analizar, interpretar y ejecutar código Python sobre variables del entorno (en este caso, el `df`). Dicha herramienta transforma el código en un árbol de sintaxis abstracta (AST), lo que asegura su validación y control antes de la ejecución [39]. Gracias a ello, el agente puede calcular matrices de correlación, detectar valores nulos, identificar valores atípicos, entre otros análisis.

El usuario no necesita ver ni escribir código, todo ocurre de forma interna, gestionado por el agente, y el resultado se devuelve en un lenguaje comprensible para el receptor.

### Persistencia y eficiencia: función `consulta_segura`

Para evitar reenvíos redundantes al modelo y manejar posibles errores, como límites de uso (HTTP 429), se utiliza una función denominada `consulta_segura()` (véase Anexo VI). Esta función guarda la respuesta generada en `st.session_state`, lo que permite que subsecuentes cargas de la página no repitan la misma consulta. Además, en caso de errores temporales, realiza reintentos automáticos con espera progresiva, ofreciendo una experiencia robusta al usuario.

### Análisis de las subsecciones

La sección de “Resumen Estadístico” se organiza mediante componentes expandibles (*expanders*), que permiten al usuario explorar diferentes aspectos del *dataset*. Cada uno de estos *expanders* está vinculado a un *prompt* específico que activa al agente, el cual interpreta la tarea, genera una respuesta y, en algunos casos, ejecuta código Python. El código correspondiente se muestra en el Anexo VI. A continuación, se describe con mayor detalle cómo funcionan estos apartados.

- **Perfilado general del dataset:** el agente describe cuántas filas y columnas contiene el archivo cargado, y clasifica cada columna según su tipo de variable (numérica, categórica, booleana, etc.). Aunque la respuesta final es completamente textual para facilitar la comprensión, el modelo consulta directamente el `DataFrame` mediante la herramienta de ejecución de código. De esta forma, se garantiza la fidelidad de la información sin intervención humana.
- **Significado de las columnas:** el enfoque es más interpretativo, ya que el agente razona sobre los nombres de las columnas para inferir su propósito. Aquí no se ejecuta código, sino que la tarea recae directamente en la capacidad lingüística del LLM.
- **Valores nulos:** el agente calcula cuántos valores faltantes existen en cada columna y resalta aquellas con proporciones elevadas. Esta tarea requiere la herramienta `PythonAstREPLTool`, que permite obtener un informe comprensible para el usuario sin mostrar código explícito.

- **Valores duplicados:** el agente analiza si existen registros idénticos y evalúa si su presencia podría distorsionar los análisis posteriores. La herramienta de ejecución de código realiza el conteo de duplicados con precisión, devolviendo un resultado expresado en lenguaje natural.
- **Correlaciones numéricas:** en este apartado el agente calcula la matriz de correlación entre variables numéricas y además interpreta sus resultados, identificando relaciones fuertes, débiles o inesperadas. Este proceso combina la ejecución de código con razonamiento del modelo.
- **Sugerencia de nuevas variables:** el enfoque es completamente basado en razonamiento. El modelo examina los nombres de las columnas y propone posibles variables derivadas. No se ejecuta código, pero los *prompts* incluyen restricciones estrictas para evitar invenciones o ejemplos irreales.
- **Propuesta de variable objetivo:** se priorizan aquellas variables que representen un estado final o resultado predecible, evaluando también su relación con otras variables. En ocasiones el agente puede apoyarse en herramientas para revisar distribuciones. El resultado es una propuesta razonada de variable objetivo.
- **Valores atípicos:** el agente aplica métodos estadísticos como IQR y *z-score* para identificar observaciones extremas en las columnas numéricas. La herramienta Python AstREPLTool resulta esencial en este análisis, permitiendo al modelo redactar un informe explicativo y comprensible.

Gracias a esta integración, el sistema ofrece un análisis estadístico accesible a cualquier tipo de usuario, ocultando la complejidad técnica sin perder rigor.

A modo ilustrativo, en la Figura 5.5 se presenta la vista inicial de esta sección. El diseño busca una experiencia clara e intuitiva para los usuarios que la utilicen.

**Resumen Estadístico del Dataset**

Vista previa del dataset

	Brand	Model	AccelSec	TopSpeed_KmH	Range_Km	Efficiency_WtKm	FastCharge_KmH	RapidCharge	PowerTrain	PlugType	BodyStyle	Segment
0	Tesla	Model 3 Long Range Dual Motor	4.6	233	450	161	940	Yes	AWD	Type 2 CCS	Sedan	D
1	Volkswagen	ID.3 Pure	10	160	270	167	250	Yes	RWD	Type 2 CCS	Hatchback	C
2	Polestar	2	4.7	210	400	181	620	Yes	AWD	Type 2 CCS	Liftback	D
3	BMW	iX3	6.8	180	360	206	560	Yes	RWD	Type 2 CCS	SUV	D
4	Honda	e	9.5	145	170	168	190	Yes	RWD	Type 2 CCS	Hatchback	B

Iniciar agente

Agente inicializado correctamente

Perfilado general del dataset

Significado de las columnas

Valores nulos

Figura 5.5: Interfaz de la sección “Resumen estadístico”

### 5.3.3. Generador de gráficos

La sección de visualización de datos proporciona una interfaz que permite al usuario generar gráficos de forma automática, simplemente formulando una instrucción en lenguaje natural. Esta capacidad está impulsada por el agente LLM, que transforma las peticiones del usuario en código Python, específicamente para construir gráficos utilizando las bibliotecas *matplotlib* o *seaborn*.

### Flujo general de funcionamiento

Una vez cargado un archivo de datos y almacenado en `st.session_state["file_path"]`, el usuario accede a esta sección y puede inicializar el agente si aún no lo ha hecho. Este proceso invoca la clase `CausalAgentFactory`, que construye el agente con acceso al `DataFrame` y las herramientas necesarias.

La interfaz guía al usuario a introducir una instrucción textual, como por ejemplo: “Haz un histograma de la columna edad”. Esta instrucción se concatena con un bloque de instrucciones internas que actúan como un *prompt* extenso. El objetivo de este *prompt* no es solo describir el rol del agente (en este caso, analista de datos especializado en visualización), sino también imponer reglas para que el código generado:

- Esté escrito en inglés y respete exactamente los nombres de variables proporcionados por el usuario (no los renombre), para evitar errores en el ciclo de ejecución.
- Utilice únicamente `matplotlib` o `seaborn`.
- No invoque funciones de renderizado directo como `st.pyplot()` o `plt.show()`, para mantener el control desde Streamlit. Es decir, la acción de mostrar los gráficos recae en la interfaz y no en el agente.
- Utilice un color por defecto unificado, salvo que el usuario especifique otro. Esto permite mantener una estética homogénea en toda la aplicación.
- Devuelva siempre el código en un bloque Markdown, con el fin de poder extraerlo y procesarlo con mayor facilidad.

### Procesamiento del código generado

El resultado devuelto por el agente debe ser un bloque Markdown con código para obtener el gráfico solicitado. El sistema utiliza expresiones regulares (`re.search(...)`), con el fin de localizar y extraer dicho bloque. Si no se detecta un bloque de código válido, se notifica al usuario que la salida no es ejecutable, pero se le muestra igualmente el texto generado por el agente.

Una vez validado, el código es ejecutado dinámicamente mediante `exec(...)` en un entorno controlado. Toda la operación se encapsula en un bloque `try-except`, lo que permite capturar y reportar posibles errores de ejecución.

Además, se ha implementado la opción de descargar los gráficos generados por el agente. En este caso, si el gráfico es válido, se utiliza `plt.savefig()` para exportarlo a un objeto `BytesIO`, que actúa como buffer de memoria en formato PNG. Este buffer se visualiza mediante `st.pyplot(plt.gcf())` y se ofrece como descarga al usuario a través de `st.download_button(...)`, con el nombre `grafico_generado.png`.

Es necesario destacar que este comportamiento robusto se logra gracias al diseño del *prompt* y a las restricciones impuestas. De este modo, el modelo se mantiene enfocado exclusivamente en la generación de visualizaciones, ignorando cualquier petición ajena a este propósito. Asimismo, el uso de `plt.close("all")` antes y después de cada operación garantiza que no se acumulen figuras activas en memoria.

### Ejemplo ilustrativo

En la Figura 5.6 se muestra una captura de la sección de generación de gráficos. En ella se observa cómo el usuario introduce una petición en lenguaje natural (por ejemplo, “Haz

un gráfico de correlación de color rosa”), y el agente responde generando automáticamente una visualización válida, que se renderiza en la interfaz.

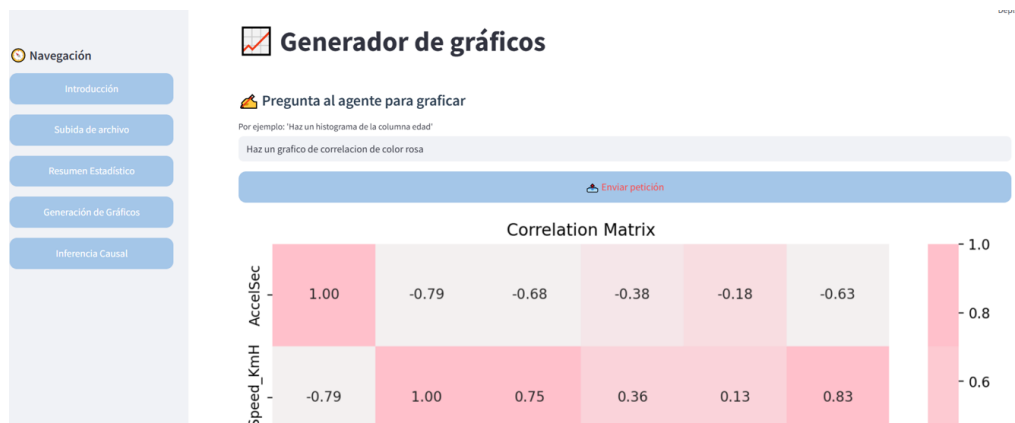


Figura 5.6: Interfaz de la sección “Generador de gráficos”

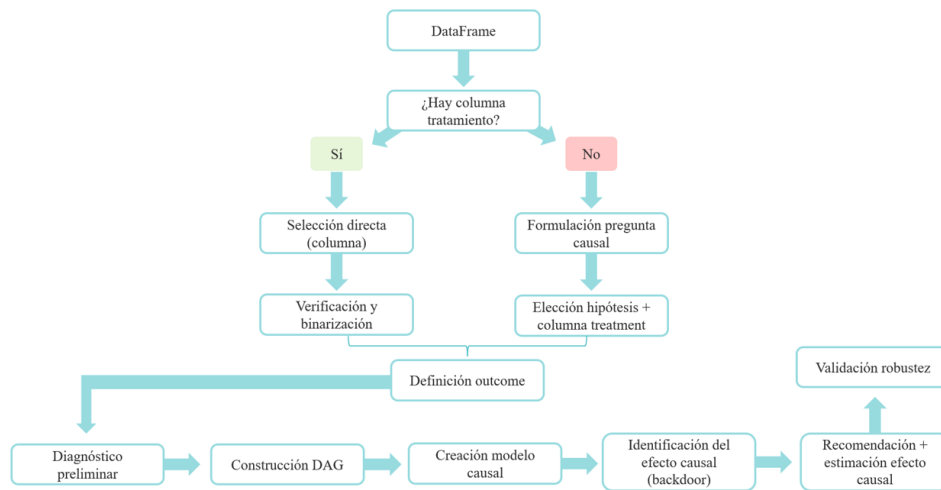
El código fuente completo correspondiente a esta sección se encuentra en el Anexo VII.

#### 5.3.4. Análisis causal

Esta sección presenta un sistema diseñado para democratizar el uso de técnicas de inferencia causal en ciencia de datos, permitiendo que incluso usuarios no expertos puedan llegar a estimar efectos causales a partir de datos observacionales. El enfoque adoptado combina herramientas de lenguaje natural con modelos de inferencia formal, creando un flujo conversacional guiado por un agente LLM, que interactúa con la biblioteca DoWhy y diversos componentes desarrollados *ad hoc* para esta aplicación.

A lo largo del proceso, se han diseñado múltiples herramientas que permiten automatizar, estructurar y explicar cada fase del análisis causal. Estas herramientas se integran con el agente mediante *prompts* construidos, que definen con precisión tanto el rol esperado del modelo como las instrucciones que debe seguir.

El flujo causal implementado abarca desde la definición del tratamiento y el resultado, pasando por la construcción del grafo causal y la identificación del efecto, hasta llegar a su estimación cuantitativa y la posterior validación de robustez. En la Figura 5.7 se observa de manera visual el flujo mencionado.



**Figura 5.7:** Flujo de inferencia causal desarrollado

A continuación, se describen en detalle las distintas fases del análisis y los componentes principales que lo hacen posible. Para consultar la implementación completa del código fuente asociado, véase el Anexo IX. Para consultar la construcción de los prompts mencionados durante esta sección, véase el Anexo VIII.

#### a) Selección del tratamiento o inferencia desde una pregunta

El módulo de inferencia causal de la aplicación ha sido diseñado para cubrir dos escenarios: contar con una columna de tratamiento previamente definida o partir de una pregunta causal formulada en lenguaje natural. Ambas modalidades convergen en el mismo flujo general de análisis, pero se diferencian en su punto de partida. A continuación, se describen ambas opciones:

##### Opción 1: Selección directa de la columna de tratamiento

En este enfoque, el usuario parte de la existencia de una columna en el conjunto de datos que representa, o puede representar, una condición de tratamiento. Esta columna puede haber sido generada previamente o simplemente contener información relevante que el usuario considera útil para definir un grupo tratado y un grupo de control.

Desde la interfaz, el usuario selecciona esta columna, y el sistema realiza una verificación de su formato. Si la variable ya es binaria (es decir, contiene únicamente dos valores distintos, como 0 y 1 o “Sí” y “No”), puede utilizarse directamente como variable de tratamiento. No obstante, si la columna no está codificada como binaria, se transforma automáticamente para cumplir con los requisitos del análisis causal.

Esta transformación binaria se realiza siguiendo una lógica sistemática:

- Si la columna es categórica y presenta un valor dominante (es decir, con mayor frecuencia relativa), este será codificado como el valor 1 de la nueva columna `treatment`. El resto de los valores se codificarán como 0.
- Si contiene valores numéricos no binarios, se aplicará una transformación lógica consistente, aunque este caso es menos habitual cuando el usuario elige directamente la columna.

El proceso de conversión es transparente para el usuario, pero se presenta un resumen de la codificación aplicada por pantalla. Se indica explícitamente cuál fue el valor

mapeado como 1 y cuál como 0, junto con una advertencia si la columna original tenía más de dos categorías distintas.

Tras la selección y binarización del tratamiento, el sistema solicita la definición de la variable objetivo (*outcome*), que será el resultado sobre el cual se evaluará el posible efecto causal. Esta variable debe cumplir ciertos requisitos básicos: ser de tipo numérico (continua o discreta) o binaria. Si se detecta que la variable es binaria pero no está codificada como 0 y 1, también se procede a su recodificación automática, utilizando el valor de mayor frecuencia como 1 por consistencia interpretativa.

Una vez definidas ambas columnas (tratamiento y resultado), el sistema considera que existe una relación causa-efecto bien especificada, y se habilitan las siguientes fases del análisis.

### Opción 2: Formulación de una pregunta causal

Si el *dataset* no contiene explícitamente una columna de tratamiento, el sistema ofrece una vía alternativa basada en lenguaje natural. El usuario puede formular libremente una pregunta causal, como por ejemplo: “¿Qué factores influyen en el precio?”. A partir de esta entrada, el agente identifica automáticamente la variable objetivo (*outcome*) a la que se refiere la consulta.

Este comportamiento se logra gracias al uso del prompt `generar_prompt_variables_relacionadas(...)` que instruye al agente para:

- Inferir la variable objetivo desde la pregunta del usuario.
- Calcular la correlación de todas las columnas numéricas con respecto a dicha variable.
- Seleccionar como variables candidatas solo aquellas que presentan una correlación fuerte (mayor a 0.7 o menor a -0.7).

La correlación se utiliza aquí como un filtro que permite identificar relaciones significativas entre variables, sin asumir causalidad directa. Con esto se consigue reducir el espacio de búsqueda a variables con potencial relevancia causal.

El prompt genera una instrucción precisa para que el agente devuelva la información en un formato JSON estructurado, sin explicaciones adicionales, como se muestra en este ejemplo:

```
{"objetivo": "Precio",
 "candidatas": ["Tamaño", "Potencia", "Consumo"]}
```

Esta salida puede ser interpretada y procesada automáticamente por el sistema, permitiendo al usuario elegir entre las variables candidatas propuestas o escoger manualmente otra de su interés.

Una vez definida la variable candidata, el agente LLM genera automáticamente varias hipótesis causales, con el objetivo de representar posibles relaciones causa-efecto entre dicha variable y el *outcome*. Estas hipótesis siguen una lógica estructurada que las hace fácilmente traducibles a código Python. Si ninguna de las hipótesis sugeridas resulta adecuada, el usuario puede introducir manualmente su propia propuesta.

Este comportamiento se articula mediante el uso del prompt `generar_prompt_hipotesis(...)`, que construye una instrucción detallada basada en tres elementos:

- La variable objetivo y la candidata.

- Una descripción resumida del dataset, generada dinámicamente por la función `generar_descripcion_dataset(...)`. Esta descripción distingue entre variables numéricas (con estadísticas como media, mínimo y máximo) y categóricas (mostrando sus valores únicos), lo que aporta contexto al modelo.
- Un conjunto de reglas estrictas de formulación, que obliga al agente a proponer hipótesis claras y concisas, con nombres exactos de columnas entre paréntesis (por ejemplo, (Precio), (Peso\_kg)). Estas hipótesis deben ser cuantificables, de forma que puedan traducirse a expresiones booleanas binarias.

Por ejemplo, una hipótesis generada podría ser: *“Los vehículos con una velocidad máxima superior a 200 km/h (TopSpeed\_KmH) presentan precios más altos (Price).”*

A partir de la hipótesis seleccionada, el sistema procede a crear la columna de tratamiento (`treatment`) utilizando la herramienta de generación de código Python. Esta funcionalidad se activa mediante un prompt que solicita al agente LLM dos elementos clave:

- Una explicación textual del razonamiento causal que justifica la hipótesis. Esta breve explicación se muestra por pantalla al usuario.
- El código Python necesario para generar la columna `treatment` en función de la variable implicada.

El prompt que se utiliza tiene una estructura cuidadosamente diseñada que le indica al modelo que debe devolver la respuesta en formato JSON, con claves explícitas.

El contenido generado se evalúa después de forma controlada mediante `exec()` para integrarse directamente en el `DataFrame`.

Un ejemplo típico de respuesta del agente sería:

```
{"explicacion": "La hipótesis se basa en que valores altos de `X` (> 100) tienen un efecto positivo sobre el precio.",
"codigo": "df['treatment'] = (df['X'] > 100).astype(int)"}
```

Una vez creada, la columna original que se usó para derivar el `treatment` (en el ejemplo, `X`) es eliminada automáticamente del `DataFrame`. Esta medida previene problemas de colinealidad, asegurando así una interpretación más robusta del efecto del tratamiento.

## b) Diagnóstico previo al análisis causal

Como paso siguiente ante las dos opciones mostradas anteriormente, el sistema presenta un conjunto de análisis exploratorios y estadísticos diseñados para evaluar la viabilidad del estudio causal antes de aplicar cualquier método.

Este bloque de diagnóstico tiene un enfoque doble: garantizar que los grupos definidos (tratados y control) sean comparables, y ofrecer al usuario una visión preliminar del comportamiento de la variable objetivo según el tratamiento.

El proceso incluye los siguientes pasos, implementados directamente en código Python con las bibliotecas `pandas`, `matplotlib`, `seaborn` y `scipy`:

1. **Distribución de la columna `treatment`:** Se genera un gráfico de barras que muestra el número de observaciones en cada grupo (`treatment = 0` y `treatment = 1`). Esto permite comprobar visualmente si existe un desequilibrio en el número de observaciones por grupo.



2. **Cálculo del balance de grupo:** A partir de la frecuencia relativa de cada grupo, se calcula el porcentaje de observaciones y se advierte al usuario si algún grupo tiene menos de 10 observaciones o si hay un desbalance excesivo (por ejemplo, una diferencia superior al 40 % entre grupos). Estas condiciones podrían invalidar o sesgar los resultados estimados.
3. **Estadísticas descriptivas por grupo:** Se agrupa el dataset por la variable `treatment` y se computan estadísticas clave de la variable objetivo (media y desviación estándar). Esto permite obtener una primera aproximación de si el tratamiento podría estar generando un cambio observable en el resultado.
4. **Visualización mediante boxplot:** Se genera un gráfico de cajas (*boxplot*) que muestra la distribución de la variable objetivo en ambos grupos. Esta visualización permite observar asimetrías, presencia de valores extremos (*outliers*) o diferencias en la dispersión de los datos.
5. **Test estadístico de diferencia entre medias:** Finalmente, se ejecuta un test de diferencia de medias (*t-test* independiente con varianzas no iguales) entre los grupos tratados y no tratados. Se muestran el estadístico *t* y el valor *p* asociado. Si  $p < 0,05$ , se interpreta que la diferencia observada es estadísticamente significativa bajo el umbral clásico de confianza del 95 %.

Esta fase se considera un filtro de calidad, ya que permite detectar con antelación si existen problemas en el diseño del tratamiento (por ejemplo, grupos desequilibrados), lo cual puede ahorrar tiempo y evitar interpretaciones erróneas en etapas posteriores del flujo causal.

### c) Generación y edición del DAG causal

Una vez definidas las variables clave del estudio (tratamiento y resultado), el sistema procede a construir un *Directed Acyclic Graph* (DAG) o grafo causal. Este diagrama representa visualmente las hipótesis del modelo, codificando las posibles relaciones de causalidad entre las variables del conjunto de datos.

El DAG inicial se genera de forma automática a través del agente, empleando un *prompt* diseñado para resumir la información más relevante del dataset. Este resumen incluye:

- La variable de tratamiento (`treatment`) y la variable de resultado (`outcome`).
- Las cinco variables numéricas con mayor correlación absoluta con el resultado (posibles mediadores).
- Las variables categóricas asociadas estadísticamente al tratamiento (confusores), identificadas mediante un test chi-cuadrado con  $p < 0,05$ .

La información se organiza en un bloque JSON y se entrega al agente, que no construye el grafo manualmente, sino que invoca una herramienta específica denominada `generate_dag` (véase Anexo V). Dicha herramienta transforma el resumen causal en un bloque de código DOT, el formato estándar de la librería *Graphviz* utilizada para renderizar grafos en la interfaz.

Los nodos del grafo se colorean de acuerdo con su rol causal:

- Verde claro para el tratamiento.

- Gris para el resultado.
- Naranja para los confusores.
- Azul claro para los mediadores.

El funcionamiento interno de la herramienta puede resumirse en una secuencia lógica. En primer lugar, el bloque de información estructurado en formato JSON es preprocesado y validado para asegurar que la estructura sea correcta. A continuación, se extraen los elementos relevantes, concretamente la variable de tratamiento, la variable de resultado, los posibles mediadores identificados a través de la correlación y los confusores derivados del test de asociación estadística.

Con esta información se construyen los nodos del grafo, asignando a cada uno un estilo visual diferenciado en función de su papel causal. Una vez definidos los nodos, se establecen las relaciones dirigidas que reflejan las hipótesis causales más relevantes. Así, se incluye la influencia del tratamiento sobre el resultado, la posible conexión de los confusores tanto con el tratamiento como con el resultado, y el papel de los mediadores en la cadena causal desde el tratamiento hasta la variable de interés. Estas relaciones se diseñan siguiendo el criterio de *backdoor*, ampliamente utilizado en la literatura de inferencia causal para garantizar la validez de la identificación.

Finalmente, la herramienta genera como salida un bloque de código en formato DOT, que puede ser representado de manera inmediata en la aplicación mediante la función `st.graphviz_chart()`. De esta forma, el usuario dispone de una visualización clara y editable del grafo causal propuesto.

Una vez generado el DAG automático, el sistema lo muestra al usuario y ofrece dos opciones:

- Confirmar la estructura sugerida como válida.
- Editar el grafo manualmente introduciendo código DOT, con previsualización interactiva en tiempo real.

Esta flexibilidad permite al usuario ajustar el modelo según su conocimiento o criterio experto. El DAG final, ya sea automático o modificado, se almacena en el estado de la aplicación y constituye el modelo estructural sobre el que se realizarán las siguientes fases del análisis causal.

A modo ilustrativo, en la Figura 5.8 se presenta un ejemplo de grafo causal generado y posteriormente editado con la herramienta. Para su construcción se planteó una hipótesis de relaciones entre variables a partir de un dataset, que será descrito en detalle en el capítulo de Resultados. Este ejemplo permite visualizar cómo el sistema representa de manera estructurada el papel de las distintas variables (tratamiento, confusores, mediadores y resultado) dentro del modelo causal.

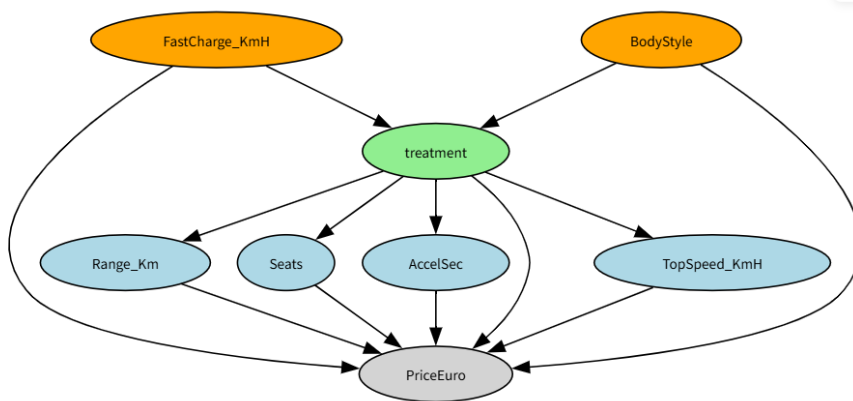


Figura 5.8: Ejemplo de DAG causal generado y editado en la aplicación.

#### d) Construcción del modelo con DoWhy

Una vez confirmado el DAG causal, el sistema procede a la creación formal del modelo de inferencia utilizando la biblioteca DoWhy. En este caso, el modelo no es generado por el agente, sino construido directamente por el sistema, garantizando así su estabilidad y persistencia a lo largo de toda la sesión de análisis. Esta decisión de diseño permite mantener un objeto estructurado (`CausalModel`) que puede ser reutilizado en las etapas posteriores del flujo.

La construcción del modelo causal requiere los siguientes elementos fundamentales: el dataset tratado (`df`), que ya contiene la columna de tratamiento en formato binario junto con la variable de resultado, el nombre de la variable de tratamiento, la variable objetivo del estudio causal, y el grafo causal, representado como una estructura `networkx.DiGraph` [40] derivada del DAG confirmado por el usuario.

Dado que el grafo validado por el usuario está inicialmente en formato DOT (propio del lenguaje de grafos de Graphviz), resulta necesario transformarlo antes de que pueda ser interpretado por DoWhy. Este proceso se desarrolla en dos fases. En primer lugar, el código DOT se analiza mediante la librería `pydot` [41], que lo interpreta y construye un objeto intermedio de tipo `pydot.Graph`. En segundo lugar, se realiza la conversión a `NetworkX`, extrayendo manualmente los nodos y aristas para conformar un grafo dirigido de tipo `networkx.DiGraph`.

Una vez completada esta conversión, se crea el modelo causal mediante la clase `CausalModel` de DoWhy, el cual queda almacenado en la sesión de Streamlit (`st.session_state["causal_model"]`) para su uso en fases posteriores.

Adicionalmente, se proporciona al usuario un resumen visual que recoge los elementos principales del modelo, incluyendo la variable de tratamiento, la variable de resultado y el número total de observaciones disponibles en el dataset.

#### e) Identificación del efecto causal

Una vez construido el modelo con DoWhy, el siguiente paso consiste en evaluar si el efecto causal puede ser identificado a partir de los datos disponibles. Este proceso de identificación se lleva a cabo de forma híbrida: por un lado, se emplea una herramienta interna basada en el método `model.identify_effect()` de la propia biblioteca, y por otro, se recurre al agente LLM para interpretar los resultados y explicarlos en lenguaje natural. Esta interacción se gestiona mediante un *prompt* específico (`generar_prompt_identificacion_efecto_seguro`) (véase Anexo VIII), que instruye al agente para invo-

car la herramienta `identify_causal_effect` (véase Anexo V) y devolver la salida en un bloque estructurado de tipo JSON.

La herramienta `identify_causal_effect` desarrolla varias acciones encadenadas. En primer lugar, invoca el método `identify_effect()` sobre el modelo causal previamente construido. Este método implementa algoritmos formales de teoría causal que permiten determinar si el efecto del tratamiento sobre el resultado es identificable bajo el DAG proporcionado. En términos prácticos, el procedimiento consiste en verificar si el efecto puede estimarse utilizando únicamente la distribución de los datos observados [42].

En segundo lugar, el sistema almacena el estimando causal (`identified_estimand`) en la sesión de Streamlit, de manera que pueda ser reutilizado posteriormente en la fase de estimación. Además, se extraen las variables de ajuste necesarias para bloquear posibles caminos de confusión (*backdoors*), mediante el método `get_backdoor_variables()`.

Una vez ejecutada la herramienta, el sistema muestra al usuario la información más relevante: el criterio de identificación utilizado (por ejemplo, el backdoor), las variables que deben incluirse como covariables en el modelo y una justificación en lenguaje natural elaborada a partir de los resultados obtenidos. En caso de que el modelo no sea identificable (por ejemplo, si no existen suficientes variables observadas para bloquear todos los caminos de confusión), el agente comunica esta limitación de manera explícita e incluye recomendaciones para rediseñar el DAG o recopilar más datos.

Con este enfoque, el sistema garantiza que el usuario disponga de una guía clara sobre qué variables deben considerarse en la estimación del efecto, favoreciendo un análisis robusto y libre de sesgos.

#### f) Elección del método de estimación

Una vez construido e identificado el modelo causal, el siguiente paso consiste en seleccionar el método más adecuado para estimar el efecto del tratamiento sobre la variable de resultado. Esta elección resulta fundamental, ya que un método inapropiado puede introducir sesgos en el análisis.

Con el objetivo de asistir al usuario en esta tarea, el sistema cuenta con el agente LLM, que evalúa automáticamente las condiciones del conjunto de datos y recomienda el método de estimación más adecuado entre los disponibles en la librería DoWhy.

El proceso comienza con la ejecución de una herramienta interna denominada `diagnostico_causal_df_tool` (véase Anexo V), que actúa como un filtro preliminar. Esta herramienta analiza el dataset teniendo en cuenta la variable de tratamiento y el conjunto de covariables identificadas en fases previas, y realiza las siguientes operaciones:

- Evalúa el balance entre grupos tratados y no tratados, aplicando métricas específicas como la diferencia de medias estandarizada (SMD) para variables numéricas y Cramér's V para variables categóricas. Estas métricas permiten estimar si los grupos son comparables.
- Estima el *score* de propensión mediante un modelo logístico, analizando su calidad a través del AUC (área bajo la curva ROC), la presencia de valores extremos (scores cercanos a 0 o 1) y el nivel de solapamiento (*overlap*) entre los grupos.
- Genera una serie de recomendaciones metodológicas automatizadas, basadas en reglas empíricas que indican la viabilidad de aplicar métodos como regresión ajustada, emparejamiento o ponderación.

La salida de esta herramienta se organiza en un bloque estructurado de tipo JSON, que el agente utiliza como base para su razonamiento. Un ejemplo simplificado es el siguiente:

```
{
 "propensity_score": {
 "auc": 0.63,
 "extremos": false,
 "overlap_visual": "moderado"
 },
 "recomendaciones_metodologicas": {
 "usar_matching": true,
 "usar_ipw": true,
 "usar_regresion": true
 }
}
```

Este razonamiento automatizado se apoya en un componente clave: el *prompt* `generar_prompt_recomendar_metodo_critico(...)` (véase Anexo VIII). Dicho *prompt* instruye al agente paso a paso, definiendo su rol como experto en inferencia causal y el objetivo de recomendar un método válido de estimación. En su estructura se establece el contexto del análisis (dataset, tratamiento, covariables), se revisan críticamente los métodos disponibles en DoWhy y se fijan criterios claros para decidir cuándo aplicar o descartar cada uno. Asimismo, se especifica cómo debe invocarse la herramienta de diagnóstico y cómo interpretar sus resultados.

La salida exigida al agente sigue un formato controlado: un bloque de código Python que imprime un JSON con dos elementos principales:

- `"metodo_recomendado"`: el nombre del método sugerido (por ejemplo, `"backdoor.propensity_score_matching"`).
- `"justificacion"`: una explicación profesional en lenguaje natural, basada en los resultados del diagnóstico y redactada en español.

A partir de esta información, el agente evalúa cuál de los métodos disponibles resulta más coherente con las características del dataset. Por ejemplo, si el solapamiento entre grupos es alto y no se detectan *scores* extremos, pueden recomendarse técnicas como el emparejamiento por *score* de propensión (*propensity score matching*) o la ponderación inversa (IPW). En cambio, si los grupos ya están razonablemente balanceados, puede ser suficiente una regresión lineal ajustada. Cada recomendación viene acompañada de una justificación en lenguaje natural que explica al usuario por qué dicho método es apropiado en su caso particular. No obstante, el usuario no está obligado a seguir la sugerencia: el sistema también ofrece la posibilidad de seleccionar manualmente otro método de la lista, de acuerdo con su propio criterio experto.

Los métodos disponibles para esta etapa han sido seleccionados por su respaldo teórico y por su implementación en la librería DoWhy [43]. A continuación, se enumeran junto con sus características principales:

- `backdoor.linear_regression`: simple y fácil de interpretar. Se utiliza cuando se asume relación lineal entre variables.
- `backdoor.propensity_score_matching`: útil para estimar el ATT. Mejora el balance emparejando unidades con *scores* de propensión similares.

- `backdoor.propensity_score_weighting` (IPW): emplea toda la muestra, ponderando por el *score* de propensión para generar una población sintética balanceada. Es adecuado para estimar el ATE.
- `backdoor.propensity_score_stratification`: divide la muestra en estratos homogéneos según el *score* de propensión. Es una técnica robusta.
- `backdoor.distance_matching`: empareja observaciones según la distancia entre co-variables, evitando supuestos fuertes. Resulta útil cuando el número de variables es reducido.
- `iv.instrumental_variable`: corrige sesgos debidos a confusores no observados, siempre que exista un instrumento válido, lo cual no siempre es fácil de garantizar.
- `iv.regression_discontinuity`: permite estimaciones causales cerca de un umbral de asignación del tratamiento, aplicable cuando la variable de tratamiento depende de un criterio claro.

El usuario puede optar por aceptar la recomendación del agente o seleccionar manualmente otro método entre los disponibles. La elección final queda almacenada en la sesión y será empleada en la siguiente fase del flujo.

#### g) Estimación del efecto causal

Una vez seleccionado el método más adecuado para la estimación del efecto causal, el sistema procede a ejecutar dicha estimación utilizando el modelo causal previamente identificado. Esta fase tiene como objetivo cuantificar el impacto que ejerce el tratamiento sobre la variable de resultado.

El procedimiento es llevado a cabo por el agente, el cual ejecuta internamente una herramienta diseñada específicamente para este propósito (`estimate_causal_effect_tool`, véase Anexo V). Esta herramienta combina el modelo causal ya generado, el estimando identificado y el método elegido por el usuario o recomendado por el sistema. Su función es realizar una llamada a la librería de inferencia causal, construyendo los parámetros adecuados para cada tipo de método (por ejemplo, ponderaciones, métricas de distancia o instrumentos válidos, según corresponda).

El resultado de esta estimación se expresa en dos componentes fundamentales:

- **La magnitud del efecto estimado**, es decir, en cuánto se espera que cambie la variable de resultado como consecuencia directa del tratamiento, en promedio, y ajustado por los confusores considerados.
- **Un intervalo de confianza del 95 %**, que cuantifica el grado de incertidumbre estadística asociado a esa estimación. Además, permite al analista juzgar si el efecto observado es estadísticamente significativo.

Internamente, esta herramienta también controla errores comunes y garantiza la trazabilidad del análisis. Por ejemplo, si el modelo causal no ha sido generado correctamente, o si el estimando no está definido, la herramienta detiene el proceso y devuelve un mensaje de error informativo. Asimismo, si el resultado estimado es anómalo o el intervalo no puede calcularse, el sistema también lo detecta y lo comunica.

Una vez completada la estimación, el agente produce una interpretación del resultado en lenguaje natural. Esta interpretación es generada mediante un *prompt* (véase Anexo VIII, que guía al agente en la elaboración de una respuesta estructurada y accesible.

Para contextualizar su respuesta, el *prompt* proporciona al agente la información clave del análisis: el nombre de la variable de tratamiento, el nombre de la variable de resultado y el método causal utilizado. A continuación, se le instruye para que ejecute la herramienta interna previamente explicada que devuelve los resultados numéricos de la estimación y que, a partir de estos, redacte una explicación.

Con esta dinámica, se solicita que exponga la magnitud del efecto junto con el intervalo de confianza. Para ello, se crea una sección de interpretación y conclusiones, en la que el agente sintetiza los resultados, señala las implicaciones prácticas del resultado y, si corresponde, advierte sobre posibles limitaciones del análisis. Además, el *prompt* impone restricciones importantes para garantizar la calidad comunicativa de la respuesta. Por ejemplo, se le indica cómo actuar si los resultados no son confiables.

#### h) Validación de robustez

Como paso final del flujo, el sistema realiza una evaluación de la robustez del efecto causal estimado. Esta etapa tiene como objetivo comprobar si el resultado obtenido se mantiene estable frente a la posible presencia de variables omitidas o perturbaciones en los datos.

La validación es llevada a cabo por el agente, que ejecuta una herramienta específica basada en los mecanismos de refutación causal disponibles en la librería DoWhy (*refute\_estimate\_tool*, véase Anexo V). En este caso, se utiliza la técnica conocida como *random\_common\_cause*, que comprueba si la estimación del efecto cambia tras añadir al conjunto de datos una variable aleatoria independiente como causa común. Idealmente, la estimación no debería verse afectada [44].

La herramienta interna encargada de esta evaluación recupera automáticamente el modelo causal, el estimando y el efecto estimado previamente guardados en sesión, y aplica la técnica de refutación configurada. El resultado incluye cuatro elementos clave: el valor del efecto causal original, el nuevo efecto obtenido tras la refutación, el valor *p* asociado a la prueba (si está disponible) y una descripción resumida del procedimiento aplicado.

A partir de esta información, el agente genera una interpretación en lenguaje natural. Esta interpretación está guiada por un *prompt*, que le indica que analice cuidadosamente la magnitud del cambio entre el efecto original y el nuevo. La respuesta producida se organiza en dos secciones principales:

- **Resultado de la refutación.** Se resume de forma comprensible el análisis realizado. Se informa si el efecto estimado se mantuvo similar o si sufrió cambios notables, e incluye la interpretación del valor *p* en términos de significancia estadística.
- **Conclusión de robustez.** El agente ofrece una evaluación cualitativa del resultado. Si el efecto permanece estable y el valor *p* es elevado (superior a 0.05), se concluye que el estimado es robusto ante la inclusión de posibles variables omitidas. Por el contrario, si se detectan cambios importantes o una disminución en la significancia, se advierte que el resultado podría no ser del todo confiable y se recomienda considerar esta limitación al interpretar los hallazgos.

---

## CAPÍTULO 6

# Resultados

---

### 6.1 Comparativa de modelos de lenguaje (Mistral vs Gemini)

---

Antes de integrar de forma definitiva el modelo de lenguaje en la aplicación, se realizaron pruebas experimentales con dos LLMs para evaluar su rendimiento en tareas básicas de interacción con un `DataFrame`. El objetivo no era realizar una comparativa exhaustiva, sino identificar cuál de los modelos disponibles ofrecía un comportamiento más estable y preciso dentro del contexto de trabajo.

Las pruebas se llevaron a cabo utilizando un *notebook* con un dataset de prueba. Se formularon preguntas básicas como “¿Cuántas filas tiene el `DataFrame`?” o “¿Cuál es el valor medio de la columna X?”, con el propósito de comprobar si el modelo era capaz de comprender el contexto, razonar correctamente sobre el contenido del `DataFrame` y generar código funcional en pandas. Para ello se empleó la función `create_pandas_dataframe_agent` de LangChain [45], que permite crear un agente conversacional basado en un LLM y vinculado a un `DataFrame`, con acceso a una herramienta de ejecución de código en Python (`python_repl_ast_tool`).

En el caso del modelo Mistral [46], la configuración se realizó mediante variables de entorno contenidas en un archivo `mistral.env`, donde se especificaban el modelo `mistral-medium` y la clave de acceso a la API oficial. Pese a que la integración técnica fue exitosa, los resultados obtenidos fueron en general insatisfactorios. El tiempo de respuesta del modelo era significativamente más alto que el de Gemini, incluso para preguntas muy simples. Además, la precisión de las respuestas era baja: en varias ocasiones, Mistral generó afirmaciones incorrectas, como señalar que el `DataFrame` contenía 1.000 filas cuando en realidad solo había unas 100. En otras situaciones, el modelo devolvía respuestas distintas ante la misma pregunta o directamente no convergía a una solución coherente.

Otro problema recurrente con Mistral fue la generación de código erróneo o el uso de parámetros no soportados. Por ejemplo, en ciertos casos el modelo intentó utilizar el parámetro `stop`, que no estaba implementado en su API en el momento de las pruebas, lo que provocaba errores de ejecución. También se detectaron dificultades en la interpretación de instrucciones estructuradas: comandos sencillos como `df.shape` eran en ocasiones malformulados o ignorados. Estos errores ponen de manifiesto limitaciones importantes en la capacidad de Mistral para operar de manera fiable con herramientas externas y estructuras de datos tabulares.

Frente a estas dificultades, se optó por repetir las mismas pruebas utilizando el modelo Gemini, de Google, bajo condiciones idénticas. La diferencia en el rendimiento fue clara. Gemini ofreció respuestas más rápidas, consistentes y técnicamente correctas. Mos-



tró una comprensión sólida del contexto del DataFrame, ejecutó correctamente las operaciones solicitadas y respondió con un lenguaje claro y ajustado a la pregunta. Además, fue más estable: las respuestas no solo eran correctas, sino también reproducibles y no mostraban el tipo de “alucinaciones” observadas en Mistral.

Estas diferencias fueron determinantes para que la versión final de la aplicación se desarrollara exclusivamente con Gemini, descartando el uso de Mistral en fases posteriores del proyecto.

## 6.2 Evaluación del rendimiento en cada fase de la aplicación

Para evaluar el comportamiento del sistema desarrollado en condiciones reales, se utilizó un conjunto de datos sobre automóviles eléctricos. Este conjunto, denominado `ElectricCarData_Clean.csv`, extraído de Kaggle [47], contiene un total de 103 registros y 14 variables, incluyendo tanto atributos técnicos como comerciales de distintos modelos de vehículos eléctricos disponibles en el mercado europeo.

Las variables del dataset abarcan características como la aceleración (`AccelSec`), velocidad máxima (`TopSpeed_KmH`), autonomía (`Range_Km`), eficiencia energética (`Efficiency_WhKm`), tipo de carga rápida (`FastCharge_KmH`), tren de potencia (`PowerTrain`), tipo de enchufe (`PlugType`), carrocería (`BodyStyle`), segmento comercial (`Segment`), número de plazas (`Seats`) y precio (`PriceEuro`), entre otras.

Este dataset fue elegido por su reducido tamaño y por incluir una combinación de variables numéricas, categóricas y binarias que permiten poner a prueba todas las funcionalidades de la aplicación. A lo largo de este apartado, se analizará el rendimiento del sistema en cada una de las fases del trabajo utilizando este conjunto de datos, valorando la precisión de los resultados obtenidos.

### 6.2.1. Evaluación de la sección “resumen estadístico”

Esta sección evalúa el rendimiento de la funcionalidad de “Resumen Estadístico” de la aplicación, aplicada al dataset real utilizado para el desarrollo y validación del sistema. El análisis fue realizado íntegramente por el agente, que combinó razonamiento guiado, ejecución de código y generación de lenguaje natural para describir los principales aspectos del conjunto de datos.

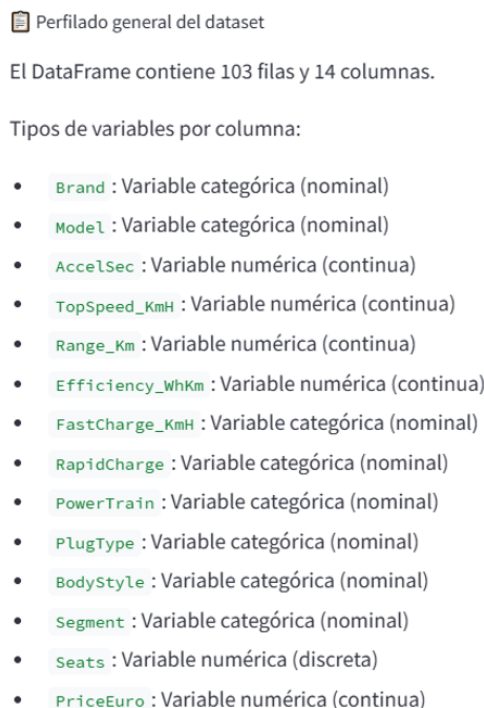
#### Perfilado general del dataset

En esta primera fase del análisis, el agente debía identificar la estructura general del conjunto de datos cargado: número de filas y columnas, así como el tipo de variable correspondiente a cada columna. Los resultados obtenidos en esta etapa fueron completamente satisfactorios.

El agente identificó correctamente el número total de filas y columnas, mostrando una capacidad precisa para consultar directamente la información del DataFrame a través de la herramienta de ejecución. Con esto, se garantiza que el archivo ha sido leído de forma correcta y que el contenido está en condiciones de ser interpretado.

En cuanto a la clasificación de las variables, los tipos asignados por el agente a cada columna fueron coherentes. Detectó adecuadamente las variables numéricas continuas (como aquellas relacionadas con velocidad, aceleración o consumo energético), las variables discretas (por ejemplo, el número de asientos), y las categóricas, respetando la distinción entre aquellas con valores de texto y otras con categorías claramente definidas.

Además, toda la respuesta se entregó en un formato claro y ordenado. El modelo cumplió correctamente las instrucciones del *prompt*: no tradujo los nombres de las columnas, no mencionó estructuras técnicas (como `DataFrame` o funciones de código) y redactó la información con un lenguaje adecuado para usuarios no técnicos.



Perfilado general del dataset

El DataFrame contiene 103 filas y 14 columnas.

Tipos de variables por columna:

- `Brand` : Variable categórica (nominal)
- `Model` : Variable categórica (nominal)
- `AccelSec` : Variable numérica (continua)
- `TopSpeed_KmH` : Variable numérica (continua)
- `Range_Km` : Variable numérica (continua)
- `Efficiency_WhKm` : Variable numérica (continua)
- `FastCharge_KmH` : Variable categórica (nominal)
- `RapidCharge` : Variable categórica (nominal)
- `PowerTrain` : Variable categórica (nominal)
- `PlugType` : Variable categórica (nominal)
- `BodyStyle` : Variable categórica (nominal)
- `Segment` : Variable categórica (nominal)
- `Seats` : Variable numérica (discreta)
- `PriceEuro` : Variable numérica (continua)

**Figura 6.1:** Demostración del funcionamiento de “perfilado general del dataset”

## Significado de las columnas

En esta segunda etapa del análisis, el agente tenía como objetivo interpretar y describir el propósito de cada una de las columnas presentes en el conjunto de datos. A diferencia del perfilado general, que se apoya en la ejecución de código, esta funcionalidad se basa exclusivamente en el razonamiento del modelo de lenguaje, guiado por un *prompt* estructurado que le indica cómo proceder.

Los resultados obtenidos fueron correctos y útiles. El agente fue capaz de generar una descripción clara y contextualizada para cada columna del dataset, respetando siempre el nombre original de las variables, sin traducciones ni alteraciones. Las explicaciones ofrecidas mostraron una comprensión razonable del dominio, interpretando adecuadamente conceptos como aceleración, autonomía, tipo de enchufe o eficiencia energética.

Cabe destacar que, a pesar de no apoyarse en ejecución de código, esta etapa resulta muy valiosa para usuarios sin conocimiento del dataset, ya que traduce los nombres de columnas en descripciones comprensibles.

### Significado de las columnas

Aquí tienes la descripción de cada columna en el dataset:

- **Brand** : Marca del vehículo eléctrico.
- **Model** : Modelo específico del vehículo eléctrico.
- **AccelSec** : Tiempo en segundos que tarda el vehículo en acelerar de 0 a 100 km/h.
- **TopSpeed\_KmH** : Velocidad máxima que puede alcanzar el vehículo en kilómetros por hora.
- **Range\_Km** : Autonomía del vehículo en kilómetros con una carga completa.
- **Efficiency\_WhKm** : Consumo de energía del vehículo en vatios-hora por kilómetro.
- **FastCharge\_KmH** : Velocidad de carga rápida del vehículo en kilómetros de autonomía añadidos por hora de carga.
- **RapidCharge** : Indica si el vehículo tiene capacidad de carga rápida (Yes/No).
- **PowerTrain** : Tipo de tren motriz del vehículo (ej. tracción delantera, tracción trasera, tracción total).
- **PlugType** : Tipo de enchufe utilizado para cargar el vehículo.
- **BodyStyle** : Estilo de carrocería del vehículo (ej. sedán, SUV, hatchback).
- **Segment** : Segmento del mercado al que pertenece el vehículo (ej. A, B, C, SUV).
- **Seats** : Número de asientos en el vehículo.
- **PriceEuro** : Precio del vehículo en euros.

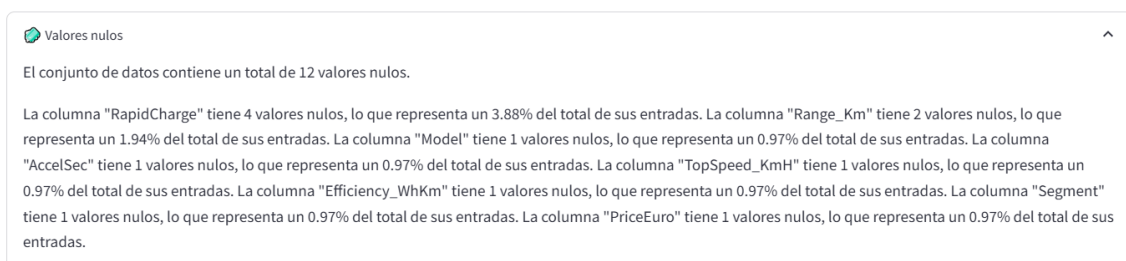
**Figura 6.2:** Demostración del funcionamiento de “significado de las columnas”

## Valores nulos

En esta etapa del análisis, el sistema evalúa la calidad del dataset en relación con la presencia de valores nulos. A diferencia del punto anterior, esta funcionalidad activa la herramienta de ejecución de código, permitiendo al agente consultar directamente el contenido del *DataFrame* para detectar valores faltantes en cada columna.

En el caso concreto del dataset original, el resultado fue claro: no se encontraron valores nulos en ninguna de las columnas. Esta información fue correctamente identificada y comunicada por el agente. El informe no incluyó detalles innecesarios ni menciones a funciones de código, cumpliendo con las restricciones impuestas en el *prompt* correspondiente.

Para comprobar la capacidad de detección del sistema, se añadieron de manera artificial un total de 12 valores nulos distribuidos en distintas columnas del dataset. El agente identificó correctamente estas ausencias y las comunicó en el informe siguiendo las reglas establecidas, resaltando en **negrita** aquellas variables con una proporción relativamente alta de datos faltantes.



**Figura 6.3:** Ejemplo de detección de nulos con faltantes añadidos.

La demostración visual del *dataset* original sin valores nulos se incluye en el Anexo X.1.

## Duplicados

El análisis de registros duplicados constituye una etapa esencial dentro del diagnóstico inicial del dataset, ya que la presencia de filas idénticas puede introducir sesgos o redundancias que afecten la calidad de los análisis posteriores. En esta fase, el agente accede directamente al contenido del *DataFrame* mediante la herramienta de ejecución de código, con el fin de identificar cuántas, si las hay, filas duplicadas existen en el conjunto de datos.

En las pruebas realizadas con el dataset de vehículos eléctricos, el agente detectó correctamente que no existían registros duplicados. La información fue presentada en un único párrafo, redactado de forma clara y sin tecnicismos, tal como se establecía en las restricciones del *prompt*.

Para verificar la capacidad de detección, se añadieron de forma artificial dos filas duplicadas al dataset. El agente reconoció estos registros y generó un informe que incluía tanto la descripción de las filas repetidas como una advertencia sobre su posible impacto en el análisis.



Brand	Model	AccelSec	TopSpeed_KmH	Range_Km	Efficiency_WhKm	FastCharge_KmH	RapidCharge	PowerTrain	PlugType
Audi	e-tron 50 quattro	6.8	190	280	231	450	Yes	AWD	Type 2 CCS
Tesla	Cybertruck Dual Motor	5	190	460	261	710	Yes	AWD	Type 2 CCS

La presencia de registros duplicados puede sesgar los análisis, inflar métricas y llevar a conclusiones erróneas. Es importante investigar la causa de estos duplicados y tomar medidas para corregirlos antes de realizar análisis más profundos.

**Figura 6.4:** Ejemplo de detección de duplicados con filas añadidas.

Al igual que en el caso de los valores nulos, la demostración detallada del *dataset* original se presenta en el Anexo X.2.

## Correlaciones numéricas

La sección dedicada al análisis de correlaciones numéricas es una de las más complejas dentro del módulo de “resumen estadístico”, ya que combina la ejecución de código con la interpretación contextual por parte del modelo. En esta fase, el agente calcula una matriz de correlación entre todas las variables numéricas del dataset, utilizando herramientas de análisis estadísticas integradas, y posteriormente interpreta los resultados en lenguaje natural.

En las pruebas realizadas, el sistema ofreció un rendimiento satisfactorio. El agente fue capaz de identificar correctamente las columnas numéricas del dataset, calcular la matriz de correlación real y presentarla en forma de tabla legible. Este primer paso técnico se ejecutó sin errores, lo que valida el correcto funcionamiento de la herramienta *PythonAstREPLTool*.

Pero el aspecto más destacable se observó en la fase interpretativa. El agente fue capaz de redactar un análisis claro, estructurado y con contenido relevante, identificando de forma precisa las relaciones fuertes y débiles entre variables. Por ejemplo, señaló con acierto la alta correlación positiva entre velocidad máxima y precio, así como la relación inversa entre aceleración y velocidad, que tiene una interpretación lógica desde el punto

de vista físico. También se detectaron señales de posible multicolinealidad, explicadas de forma comprensible para usuarios no técnicos.

En caso de haber habido relaciones inesperadas o inconsistentes (por ejemplo, correlaciones fuertes entre variables que no deberían estar conectadas), el agente habría sido capaz de advertirlo, ya que el *prompt* le instruye para detectar y comentar este tipo de anomalías.



Figura 6.5: Demostración del cálculo de “correlaciones numéricas”

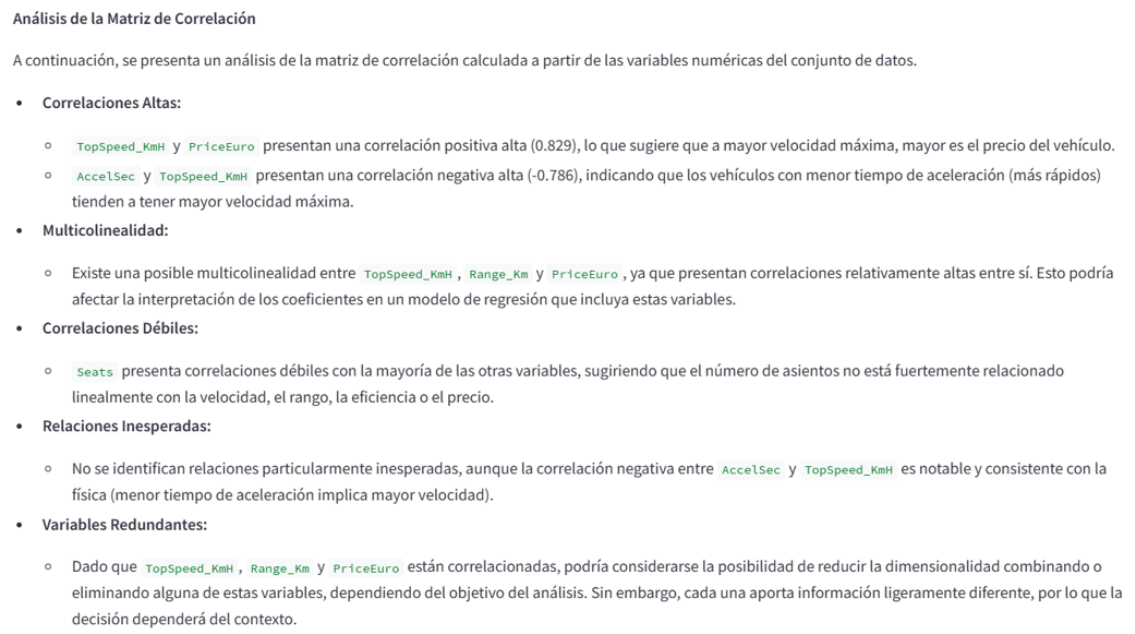


Figura 6.6: Demostración de la explicación de “correlaciones numéricas”

Sugerencias de nuevas variables

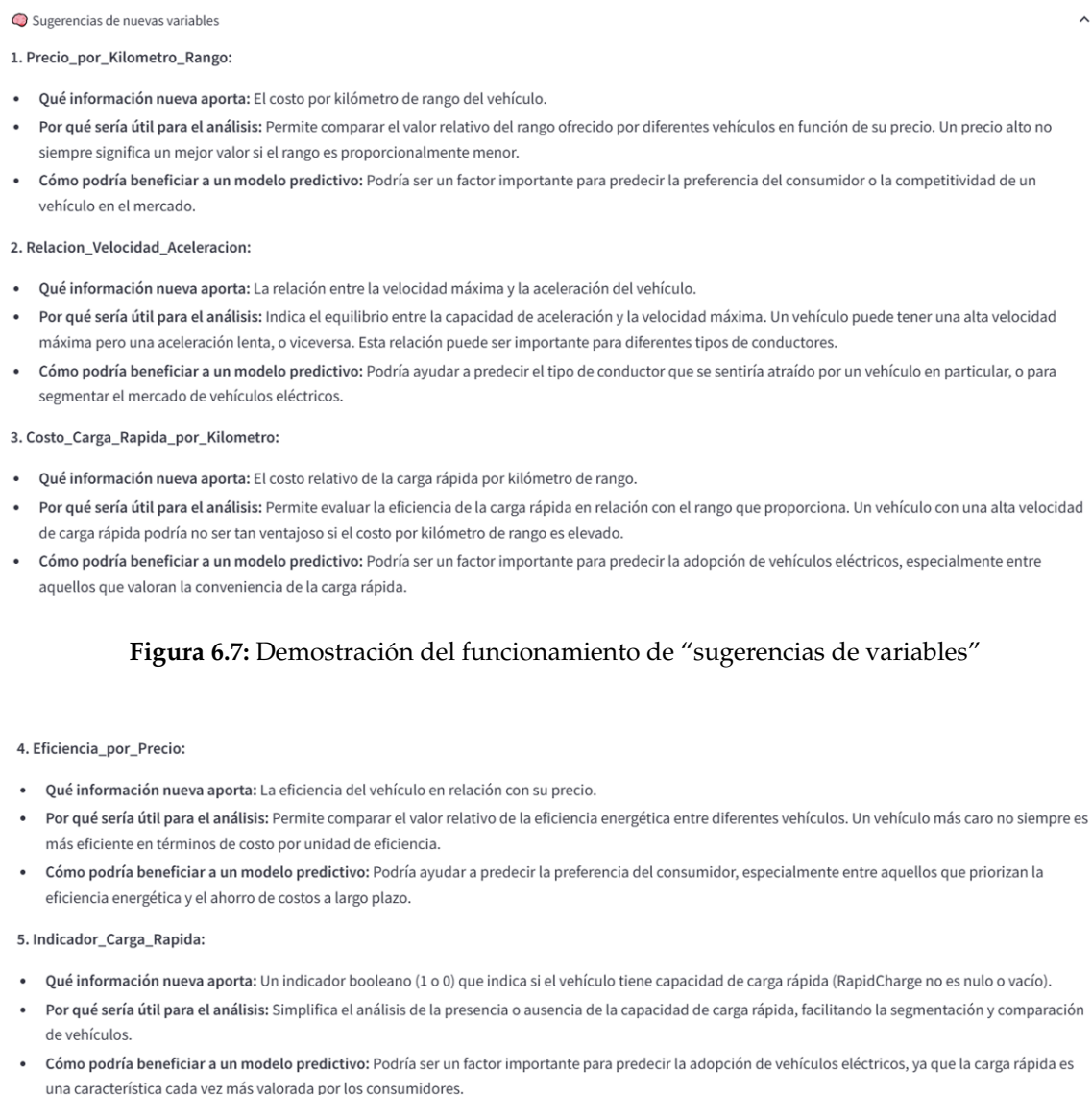
En esta etapa del resumen estadístico, el agente no ejecuta código, sino que activa su capacidad de razonamiento sobre el contenido del dataset para proponer variables derivadas que puedan aportar valor al análisis posterior o al desarrollo de modelos predictivos. Esta tarea está guiada por un *prompt*, que obliga al modelo a trabajar exclusivamente con las columnas existentes y justificar con detalle cada sugerencia.

Los resultados obtenidos en esta sección fueron especialmente positivos. El agente propuso variables nuevas lógicas, relevantes y directamente derivables del contenido

del dataset. Por ejemplo, planteó combinaciones razonadas entre precio y autonomía, eficiencia por coste o indicadores booleanos sobre características clave como la carga rápida. Estas propuestas no solo eran factibles desde el punto de vista técnico, sino que además fueron explicadas con claridad en términos de su utilidad para el análisis o para el desarrollo de modelos.

Lo más destacable fue que el modelo cumplió estrictamente las limitaciones impuestas por el *prompt*. No inventó columnas inexistentes, no propuso ejemplos genéricos ni introdujo lenguaje técnico innecesario. Todas las variables sugeridas se basaban en combinaciones comprensibles entre atributos existentes, como velocidad, aceleración, precio o eficiencia. Además, la redacción de cada sugerencia seguía una estructura profesional: se explicaba qué aporta la nueva variable, por qué es útil y cómo podría beneficiar al análisis predictivo.

En caso de que el dataset hubiese estado mal estructurado o no ofreciera suficientes variables combinables, el *prompt* estaba preparado para que el agente devolviera un mensaje indicando que no era posible proponer nuevas *features* sin conocer el contenido real.



**Figura 6.8:** Demostración del funcionamiento de “sugerencias de variables”



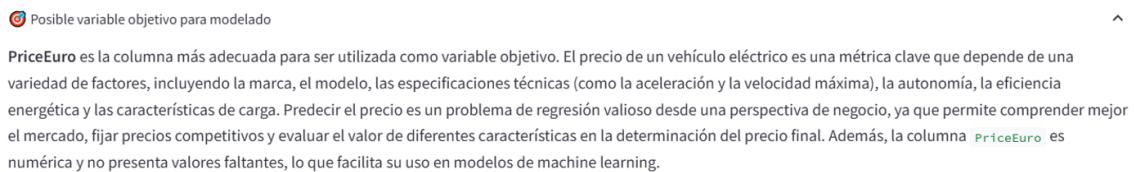
### Propuesta de variable objetivo

Una de las tareas más relevantes dentro del flujo de análisis es la identificación de una posible variable objetivo, es decir, aquella que podría predecirse a partir del resto en un modelo supervisado. Esta funcionalidad se basa principalmente en el razonamiento semántico del modelo, aunque también puede recurrir a la ejecución de código para explorar la estructura del dataset si lo considera necesario.

En el caso evaluado, el agente propuso como variable objetivo la columna relacionada con el precio del vehículo. La variable seleccionada es numérica, no presenta valores faltantes y representa un resultado económico. Además, su elección fue acompañada por una justificación coherente, ya que se argumentó que el precio depende de otros atributos clave como autonomía, eficiencia, velocidad, marca o tipo de carga, y que modelar esta relación resulta valioso para análisis de mercado y fijación de precios.

Lo más destacable de esta sección fue cómo reaccionó el agente: este no se limitó a identificar una variable “numérica” cualquiera, sino que priorizó aquella que cumple criterios de interpretabilidad, relevancia práctica y potencial de predicción. También respetó las restricciones impuestas por el *prompt*, como evitar variables con datos faltantes o sin relación clara con el resto del dataset.

En caso de no encontrar una variable idónea, el *prompt* le habría instruido para devolver una advertencia razonada, en lugar de forzar una elección incorrecta.



Posible variable objetivo para modelado

PriceEuro es la columna más adecuada para ser utilizada como variable objetivo. El precio de un vehículo eléctrico es una métrica clave que depende de una variedad de factores, incluyendo la marca, el modelo, las especificaciones técnicas (como la aceleración y la velocidad máxima), la autonomía, la eficiencia energética y las características de carga. Predecir el precio es un problema de regresión valioso desde una perspectiva de negocio, ya que permite comprender mejor el mercado, fijar precios competitivos y evaluar el valor de diferentes características en la determinación del precio final. Además, la columna PriceEuro es numérica y no presenta valores faltantes, lo que facilita su uso en modelos de machine learning.

Figura 6.9: Demostración del funcionamiento de “propuesta de variable objetivo”

### Valores atípicos (outliers)

El análisis de valores atípicos representa uno de los puntos más técnicos del resumen estadístico, ya que requiere la aplicación de criterios estadísticos para identificar observaciones extremas. En esta fase, el agente activa la herramienta de ejecución de código para calcular métricas como el rango intercuartílico (IQR) o el *z-score*, y luego redacta una interpretación textual sobre los posibles valores atípicos detectados.

En las pruebas realizadas, el agente identificó correctamente columnas numéricas que contenían valores especialmente altos o bajos en relación con el resto de la muestra, y los presentó en una lista ordenada, cumpliendo el formato requerido. El modelo explicó de forma clara por qué ciertos valores podían considerarse atípicos y cuál sería su posible impacto en el análisis.

El lenguaje utilizado fue variado y profesional, evitando repeticiones o explicaciones genéricas. Por ejemplo, distinguió entre vehículos con velocidades anormalmente altas, precios extremos o rangos de autonomía poco habituales, y contextualizó cómo estos casos podrían alterar las medias generales o inducir sesgos en modelos predictivos.

En caso de que no se hubiesen detectado *outliers*, el sistema estaba preparado para devolver una respuesta controlada, indicando explícitamente que no se observaban valores extremos relevantes, evitando así falsos positivos o interpretaciones innecesarias.

- ▲ Valores atípicos (outliers) ^
- **AccelSec** : valor atípico detectado: 22.4 (alto). Una aceleración tan lenta podría indicar un vehículo con rendimiento muy inferior al promedio, afectando el análisis de desempeño general.
  - **AccelSec** : valor atípico detectado: 2.1 (bajo). Una aceleración tan rápida podría representar un vehículo de alto rendimiento inusual, sesgando las métricas de velocidad promedio.
  - **TopSpeed\_KmH** : valor atípico detectado: 410 (alto). Esta velocidad máxima es excepcionalmente alta y podría distorsionar el análisis de las capacidades de velocidad típicas.
  - **TopSpeed\_KmH** : valor atípico detectado: 123 (bajo). Una velocidad máxima tan baja podría indicar un vehículo con limitaciones significativas, afectando las conclusiones sobre rendimiento.
  - **Range\_Km** : valor atípico detectado: 970 (alto). Un rango tan extenso podría representar una tecnología de batería avanzada inusual, sesgando las métricas de autonomía promedio.
  - **Range\_Km** : valor atípico detectado: 95 (bajo). Un rango tan limitado podría indicar una batería con problemas o un vehículo diseñado para distancias muy cortas, afectando las conclusiones sobre autonomía.
  - **Efficiency\_WhKm** : valor atípico detectado: 273 (alto). Un consumo de energía tan alto podría indicar ineficiencia o un vehículo con características especiales, distorsionando el análisis de eficiencia energética.
  - **Efficiency\_WhKm** : valor atípico detectado: 104 (bajo). Un valor tan bajo de consumo podría representar una medición inusual o un vehículo atípico, alterando las conclusiones sobre eficiencia general.
  - **Seats** : valor atípico detectado: 7 (alto). Un número tan alto de asientos podría indicar un vehículo diseñado para transporte de pasajeros, afectando el análisis de vehículos personales.
  - **Seats** : valor atípico detectado: 2 (bajo). Un número tan bajo de asientos podría indicar un vehículo deportivo o biplaza, sesgando las métricas de capacidad promedio.
  - **PriceEuro** : valor atípico detectado: 215000 (alto). Este precio es considerablemente más alto que el resto y puede influir de forma desproporcionada en los análisis de costos promedio.
  - **PriceEuro** : valor atípico detectado: 20129 (bajo). Un precio tan bajo podría indicar un modelo básico o una oferta inusual, afectando el análisis de precios promedio.

Figura 6.10: Demostración del funcionamiento de “valores atípicos”

### 6.2.2. Evaluación de la sección “generador de gráficos”

Para la evaluación de la capacidad de generación de gráficos por parte del agente, se realizaron múltiples pruebas ejecutando peticiones gráficas variadas y comparando sus resultados con representaciones manuales generadas directamente en un *notebook*. El objetivo era comprobar que el código producido por el agente correspondiera con lo que se espera de una visualización estadísticamente válida.

Los tipos de gráficos evaluados incluyeron diagramas de barras, gráficos de pastel, gráficos de dispersión, histogramas y conteos categóricos, entre otros. En todos los casos, el agente fue capaz de generar correctamente el código en Python utilizando las bibliotecas `matplotlib` o `seaborn`, respetando las instrucciones impuestas a través del *prompt*. Las visualizaciones producidas fueron comparadas con versiones ejecutadas manualmente, confirmando que los valores representados eran correctos y coherentes con los datos del dataset.

El comportamiento del sistema ante entradas mal formuladas también fue objeto de evaluación. Si el usuario introduce una petición poco clara, ambigua o irrelevante (por ejemplo: “haz lo que quieras” o “dibuja cualquier cosa”), el agente está específicamente instruido para no generar código alguno, sino devolver un mensaje informativo indicando que no puede procesar la solicitud y que se requiere una petición más específica. Esta lógica también se aplica en casos donde se intenta desviar al agente hacia tareas para las que no está autorizado (por ejemplo, análisis estadístico o modelado), reforzando así la seguridad funcional del sistema.

Las imágenes generadas por el sistema pueden consultarse en el Anexo XI, donde se recopilan ejemplos de las salidas gráficas producidas por el agente a partir de distintas instrucciones.

### 6.2.3. Evaluación del análisis causal

Esta sección presenta los resultados obtenidos al evaluar el funcionamiento del módulo de inferencia causal implementado en la aplicación. Para cada fase, se analiza si el comportamiento del agente fue adecuado, si los resultados producidos fueron correctos



y comprensibles, y si el proceso global puede considerarse fiable y útil desde una perspectiva práctica.

Las salidas visuales generadas durante el análisis, incluyendo gráficos, estructuras causales y explicaciones del agente, se recogen en el Anexo B, al que se hará referencia a lo largo del presente apartado.

### Definición del tratamiento y variable objetivo

Durante la fase de definición del tratamiento y la variable de resultado, se realizaron múltiples pruebas sobre el dataset utilizando ambos modos disponibles en la aplicación: selección directa de columnas y formulación de preguntas causales en lenguaje natural. En todos los casos, el sistema respondió de manera satisfactoria, generando resultados correctos y coherentes.

En la opción de selección manual de columnas, se verificó que el sistema detecta adecuadamente si la variable elegida como tratamiento necesita ser recodificada. Por ejemplo, al seleccionar columnas categóricas con más de dos valores, se aplicó correctamente la codificación binaria basada en la frecuencia, y se mostró al usuario un resumen comprensible del mapeo aplicado. Esta lógica se mantuvo de forma consistente en todas las pruebas, sin errores de codificación ni fallos en la validación de formatos.

También se validó que, al elegir la variable de resultado, el sistema verifica correctamente que sea de tipo numérico o binario, y aplica transformaciones adecuadas en los casos necesarios. Se hicieron pruebas seleccionando variables en distintos formatos y, en todos los casos, la validación y recodificación se comportaron de forma precisa.

¿Quieres seleccionar una columna como tratamiento?

☒ Sí, seleccionar tratamiento

☐ No, formular una pregunta causal

Selecciona la columna de tratamiento:

RapidCharge

Columna seleccionada como tratamiento: **RapidCharge** (convertida a binaria)

Codificación aplicada a RapidCharge:

- Valor 1 → Yes
- Valor 0 → No

Selecciona la variable objetivo (resultado):

PriceEuro

Variable objetivo seleccionada: **PriceEuro**

☒ Confirmar tratamiento y variable objetivo

**Figura 6.11:** Demostración del funcionamiento de “selección de variables”

Se incluyen en el Anexo XII.1 capturas y ejemplos de otros casos de prueba.

### Formulación causal asistida

En esta fase se evalúa la capacidad del sistema para asistir al usuario en la generación de hipótesis causales a partir de una pregunta formulada en lenguaje natural. Para ello, se llevaron a cabo múltiples pruebas en las que se plantearon preguntas abiertas como

“¿Qué influye en la velocidad?” o “¿Qué variables afectan a la autonomía del vehículo?”, con el objetivo de observar el comportamiento del agente ante consultas vagas o ambiguas.

Los resultados fueron altamente satisfactorios. En todos los casos, el sistema fue capaz de identificar correctamente la variable objetivo mencionada en la pregunta, incluso si su nombre exacto no coincidía con el del *dataset* (por ejemplo, interpretando “velocidad” como `TopSpeed_KmH`, como en la Figura 6.12). Además, aplicó correctamente el criterio de selección de variables candidatas basándose en correlaciones fuertes, devolviendo un conjunto reducido de atributos con alta relevancia potencial para el análisis causal.

Deploy

## Selección del Tratamiento o Pregunta Causal

¿Quieres seleccionar una columna como tratamiento?

- ☐ Sí, seleccionar tratamiento  
☒ No, formular una pregunta causal

Formula tu pregunta causal (por ejemplo: ¿Qué influye en el precio?)

que variables influyen en la velocidad

### Variables candidatas encontradas

Selecciona cómo deseas elegir la variable para el análisis causal:

- ☒ Seleccionar una de las variables candidatas sugeridas  
☐ Quiero seleccionar otra variable manualmente

Selecciona una de las variables candidatas:

- ☐ `AccelSec`  
☐ `Range_Km`  
☒ `PriceEuro`

Has seleccionado: `PriceEuro`. Pulsa el botón para continuar.

**Figura 6.12:** Demostración del funcionamiento de “variables candidatas”

A partir de estas variables candidatas, el agente generó varias hipótesis causales escritas en lenguaje natural. Estas hipótesis fueron, en general, claras y cuantificables, lo que permitió su traducción a código Python para la construcción de la variable `treatment`. Todas las hipótesis sugeridas cumplían los criterios de formulación impuestos: uso explícito de columnas existentes, presencia de umbrales numéricos o valores categóricos específicos, y compatibilidad con una condición booleana interpretable.

Como ejemplo ilustrativo, se utilizan las variables `AccelSec` y `TopSpeed_KmH` en la Figura 6.13.

### Sugerencias de hipótesis causales ↔

Selecciona una hipótesis sugerida o escribe la tuya propia:

- ☒ "Los vehículos que tardan más de 8 segundos en acelerar de 0 a 100 km/h (`AccelSec`) tienden a tener una velocidad máxima menor (`TopSpeed_KmH`)."  
☐ "Los vehículos que tardan menos de 6 segundos en acelerar de 0 a 100 km/h (`AccelSec`) tienden a alcanzar una velocidad máxima superior a 200 km/h (`TopSpeed_KmH`)."  
☐ Quiero escribir mi propia hipótesis

☒ Hipótesis seleccionada: "Los vehículos que tardan más de 8 segundos en acelerar de 0 a 100 km/h (`AccelSec`) tienden a tener una velocidad máxima menor (`TopSpeed_KmH`)."

**Figura 6.13:** Demostración del funcionamiento de “generación de hipótesis”

En las pruebas realizadas, el código generado por el sistema para construir la columna `treatment` fue correcto en todos los casos. Asimismo, el sistema aplicó correctamente

la eliminación de la variable original utilizada en la condición, evitando problemas de colinealidad en fases posteriores del análisis.

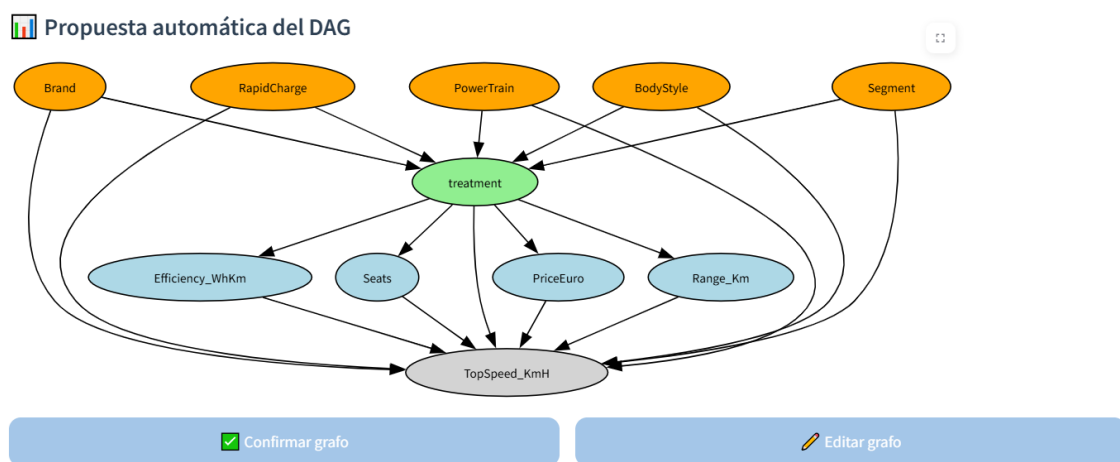
Para una mayor transparencia, se incluyen en el Anexo XII.2 otros ejemplos detallados de entradas realizadas, hipótesis generadas, código devuelto y estructura final del *DataFrame*.

### Evaluación de la generación del DAG

Durante esta fase se evaluó el rendimiento del sistema en la generación automática del grafo causal (DAG). El agente fue capaz de construir correctamente un DAG inicial a partir de los elementos clave del análisis (tratamiento, *outcome*, mediadores y confusores) basándose en un resumen estructurado del *dataset*.

Cabe destacar que, en todos los casos, el agente hizo un uso correcto de la herramienta específica diseñada para la generación del DAG. En lugar de construir el grafo manualmente, invocó adecuadamente la herramienta `generate_dag`, tal como estaba previsto en el diseño de los *prompts*.

Los resultados fueron en general muy positivos. El grafo generado fue coherente con la hipótesis causal seleccionada y reflejaba adecuadamente las relaciones esperadas según el conocimiento previo del dominio. Las flechas entre nodos estaban bien orientadas y el modelo cumplía con la restricción de ser acíclico. Además, las relaciones incorporadas respetaban las reglas del criterio de *backdoor*, lo cual garantizaba que la estructura resultante fuera apta para su posterior identificación formal.



**Figura 6.14:** Demostración del funcionamiento de “generación del DAG”

En los casos en los que el usuario quiso realizar ajustes manuales sobre el grafo, el sistema ofreció una funcionalidad de edición controlada a través de un campo de texto en el que se podía modificar directamente el código DOT. Esta opción fue probada en varios escenarios y demostró ser estable. El sistema renderizaba correctamente las ediciones, ofrecía previsualizaciones instantáneas, y permitía confirmar el DAG modificado como versión final para el análisis.

En el Anexo XII.3 se incluyen capturas y ejemplos de DAGs generados en distintos escenarios como evidencia visual del correcto funcionamiento de esta funcionalidad.

### Diagnóstico de identificabilidad del efecto

Durante esta etapa se evaluó si el sistema era capaz de diagnosticar correctamente la identificabilidad del efecto causal a partir del DAG generado. El análisis se realizó empleando la herramienta `identify_causal_effect`, la cual fue invocada automáticamente por el agente mediante el *prompt* correspondiente.

Los resultados fueron satisfactorios en todos los casos evaluados. El agente identificó de forma correcta el criterio de identificación (principalmente el criterio de *backdoor*) y determinó el conjunto de variables de ajuste necesarias para garantizar un análisis sin sesgo. Estas variables coincidían con las relaciones establecidas en el grafo causal y mostraban una lógica coherente.

Además, el agente generó una justificación bien enfocada, en la que explicaba por qué el efecto era identificable, qué criterio se había seguido y cuáles eran las variables clave para el ajuste.

#### Diagnóstico de identificabilidad

Criterio usado: `backdoor`

Variables de ajuste: `['Segment', 'BodyStyle', 'PowerTrain', 'RapidCharge', 'Brand']`

Justificación: El efecto causal fue identificado utilizando el criterio de backdoor. Esto significa que, al ajustar por las variables 'Segment', 'BodyStyle', 'PowerTrain', 'RapidCharge' y 'Brand', se bloquean todos los caminos de puerta trasera entre el tratamiento y el resultado, permitiendo estimar el efecto causal directo del tratamiento sobre el resultado. Estas variables son suficientes para eliminar el sesgo de confusión y obtener una estimación causal válida.

**Figura 6.15:** Demostración del funcionamiento de “identificación del efecto”

En los casos en que la estructura del DAG no permite la identificación de un efecto (por ejemplo, cuando no existe ninguna ruta causal entre la variable de tratamiento y la de resultado), el sistema reacciona indicando que el efecto es nulo y que no se requieren variables de ajuste.

En el Anexo XII.4 se incluyen ejemplos distintos de salidas generadas por el agente junto con sus respectivas justificaciones.

### Recomendación del método de estimación

Durante esta fase se evaluó si el sistema era capaz de analizar las condiciones del *dataset* y recomendar un método de estimación causal adecuado. Para ello, se apoyaba en una herramienta interna que analizaba automáticamente aspectos clave como el *score* de propensión, el balance entre grupos, el valor del AUC, la presencia de *scores* extremos y el grado de solapamiento (*overlap*) entre las distribuciones.

En general, el comportamiento del agente fue correcto. Logró interpretar los resultados del diagnóstico y emitió recomendaciones coherentes con lo que cabría esperar desde una perspectiva metodológica. La justificación textual proporcionada por el modelo fue clara y argumentada, vinculando explícitamente las métricas observadas con las características de cada método.

Método recomendado por el agente:

 **backdoor.linear\_regression**

**Justificación:** El score de propensión muestra un AUC de 0.817, lo que indica un solapamiento moderado y la presencia de scores extremos. Esto desaconseja el uso de propensity score matching o weighting. La herramienta recomienda usar regresión. Dado que las variables de ajuste son categóricas, se deben codificar adecuadamente (one-hot encoding) antes de aplicar la regresión lineal. La variable 'Brand' tiene un Cramer's V alto de 0.831, lo que sugiere una fuerte asociación con el tratamiento, por lo que es importante incluirla en el modelo de regresión.

**Figura 6.16:** Demostración del funcionamiento de “recomendación del método estimación”

Sin embargo, es importante señalar que la recomendación se basa en reglas heurísticas y no implica necesariamente que el método sugerido sea el óptimo en todos los casos. Existen múltiples factores que pueden influir en la calidad final de la estimación, como la heterogeneidad del tratamiento, que no son evaluados de forma exhaustiva en esta etapa. Por tanto, aunque el método propuesto por el agente suele ser una elección válida, no puede garantizarse que sea la mejor posible.

Por último, se comprobó que el sistema permite al usuario rechazar la recomendación y seleccionar manualmente otro método dentro de una lista cerrada, lo que aporta flexibilidad al flujo. Esta opción es especialmente relevante si el usuario posee conocimiento experto o desea contrastar diferentes estrategias de estimación.

En el Anexo XII.5 se incluyen otros ejemplos representativos del diagnóstico general, de las recomendaciones emitidas y de las justificaciones proporcionadas por el agente para distintos escenarios.

### Estimación del efecto causal

Durante esta etapa del análisis se procedió a la estimación cuantitativa del efecto causal, utilizando el método previamente seleccionado. La herramienta integrada ejecutó correctamente el procedimiento de estimación sobre el modelo causal ya construido, devolviendo dos resultados clave: el valor numérico del efecto causal medio y su correspondiente intervalo de confianza al 95 %.

El agente, a partir de estos resultados, generó una interpretación que incluía tres apartados: magnitud del efecto, intervalo de confianza y conclusiones interpretativas. En términos generales, la redacción fue clara y correcta para un usuario final sin conocimientos avanzados en inferencia causal, lo que cumple con el objetivo de accesibilidad del sistema. Además, se indicaba si el resultado era estadísticamente significativo, en función de si el intervalo de confianza incluía o no el valor nulo.

En las pruebas realizadas con diferentes configuraciones de tratamiento y resultado, el sistema produjo estimaciones razonables con la hipótesis causal. No obstante, la significancia estadística no estuvo siempre garantizada, lo cual fue reflejado en la interpretación generada por el agente.

 **Estimación e interpretación del agente:****Magnitud del efecto**

En promedio, el tratamiento parece estar asociado con una disminución de aproximadamente 3.6 unidades en la variable de resultado.

**Intervalo de confianza y significancia**

El intervalo de confianza del 95% para esta estimación se extiende desde -11.9 hasta 4.7. Este amplio intervalo sugiere una considerable incertidumbre en la estimación del efecto.

**Interpretación y conclusiones**

Este resultado sugiere que el tratamiento podría tener un efecto negativo en la variable de resultado, aunque con cierta incertidumbre. Dado que el intervalo de confianza incluye el cero, no se puede descartar que el tratamiento no tenga ningún efecto en absoluto. Es importante considerar que este análisis se basa en el método seleccionado y los datos disponibles, y que otros factores no considerados podrían estar influyendo en la relación observada.

**Figura 6.17:** Demostración del funcionamiento de “estimación del efecto”

Cabe destacar que, en aquellos casos en los que los resultados eran poco concluyentes, el modelo señaló posibles limitaciones o recomendó considerar otras estrategias de estimación.

En el Anexo XII.6 se incluyen más ejemplos textuales de interpretaciones generadas por el agente, junto con sus correspondientes valores estimados e intervalos de confianza.

### Validación de robustez del estimado

Como fase final del análisis causal, se procedió a la evaluación de la robustez del efecto estimado, utilizando la técnica de refutación `random_common_cause`. En todos los casos evaluados, el agente fue capaz de ejecutar correctamente la herramienta de validación, recuperar los resultados numéricos y elaborar una buena interpretación. El mensaje generado distinguía entre el efecto estimado original, el nuevo valor tras la refutación y el valor `p` asociado a la prueba, si este estaba disponible.

 **Resultado de la validación del agente:**

**Resultado de la refutación:** Se aplicó la prueba de refutación "`random_common_cause`". El efecto estimado original fue de -3.60, y tras la refutación, el nuevo efecto estimado es de -3.58. El valor `p` obtenido es de 0.88.

**Conclusión de robustez:** Dado que el cambio en el efecto estimado es mínimo y el valor `p` es alto, el efecto estimado original parece ser robusto a la presencia de posibles variables omitidas.

**Figura 6.18:** Demostración del funcionamiento de “validación de robustez”

Los resultados observados fueron variados, dependiendo del tipo de tratamiento y variable objetivo analizados. En varios casos, el efecto estimado se mantuvo estable tras la introducción del ruido aleatorio, lo que fue interpretado como una señal de robustez. En cambio, en otros escenarios se detectaron cambios considerables o disminución de la significancia, y el agente emitió un mensaje indicando que el resultado podría no ser totalmente fiable ante la posible existencia de confusores omitidos.

En el Anexo XII.7 se pueden consultar más ejemplos concretos de esta validación, con los valores antes y después de la refutación, así como los mensajes interpretativos generados por el agente.

El conjunto de pruebas realizadas sobre las distintas fases del análisis automatizado ha permitido comprobar que el sistema desarrollado es funcional, preciso en la mayoría de los escenarios y útil desde una perspectiva práctica. El agente ha demostrado ser capaz de ejecutar correctamente las herramientas internas, generar interpretaciones razonables y facilitar el proceso analítico a usuarios no expertos. A pesar de algunas limitaciones

inherentes al enfoque automatizado, el comportamiento observado valida la viabilidad del enfoque propuesto y su potencial para democratizar el uso de técnicas avanzadas de análisis de datos.

---

## CAPÍTULO 7

# Conclusiones

---

### 7.1 Conclusiones generales

---

De todo lo anterior se desprende que el sistema desarrollado ha cumplido de forma satisfactoria los objetivos principales del Trabajo de Fin de Grado, consolidando una solución funcional para el análisis exploratorio y causal de datos a través del lenguaje natural. La propuesta combina ciertas dimensiones del conocimiento adquirido durante la carrera, incluyendo estadística, ciencia de datos, programación y visualización. Por otro lado, integra otros conceptos aprendidos de manera autónoma, como el desarrollo web y el diseño de interfaces conversacionales, integradas a través del uso de agentes inteligentes basados en modelos de lenguaje.

Uno de los aspectos más relevantes de este trabajo ha sido la implementación estructurada de agentes LLM, capaces de ejecutar tareas complejas de razonamiento aplicado mediante el uso de herramientas específicas que combinan ejecución de código, generación de texto y representación visual. Este enfoque va más allá del simple uso de un chatbot, ya que se trata de un sistema guiado por lógica, en el que cada acción del agente está delimitada por un conjunto de instrucciones. La gestión de este comportamiento se ha articulado mediante un diseño modular de *prompts* que encapsulan tanto las reglas del análisis como los objetivos del usuario, lo que permite un control preciso del flujo conversacional.

El uso de técnicas avanzadas de *prompt engineering* ha resultado ser un pilar fundamental en el éxito del sistema. Cada componente del flujo, en todas las etapas construidas, ha requerido un *prompt* con criterios específicos sobre el formato, la estructura de la respuesta y el comportamiento deseado. Se ha comprobado que una buena ingeniería de *prompts* es clave para evitar errores, alucinaciones o respuestas ambiguas, y para asegurar que el modelo se comporte de forma coherente con los objetivos del análisis.

Además, el sistema ha sido desarrollado como una aplicación web interactiva, en la que el agente se integra como un servicio que responde a acciones específicas del usuario. Esta separación entre lógica de presentación (*frontend*) y lógica analítica (*backend* con agentes LLM) ha permitido mantener una experiencia de usuario fluida. En este sentido, el agente no actúa de forma genérica, sino que responde a un entorno definido por etapas, restricciones y herramientas que delimitan su campo de actuación, lo que mejora su utilidad y reduce los errores.

Un aspecto destacable ha sido también el uso deliberado de modelos gratuitos o de bajo coste, evitando servicios de pago de alto rendimiento como GPT-4. Esta elección impuso ciertas limitaciones en cuanto a tamaño de contexto, capacidad de razonamiento o gestión de instrucciones complejas, especialmente en el caso de Mistral, donde se



observaron problemas frecuentes de alucinación o generación incorrecta de código. Sin embargo, mediante un diseño del sistema y una buena ingeniería de *prompts*, se ha conseguido que modelos de acceso público funcionen correctamente en escenarios realistas, a pesar de ser en *datasets* reducidos.

Los resultados obtenidos en las distintas fases del flujo causal han sido, en general, consistentes y técnicamente válidos. El sistema fue capaz de guiar al usuario de forma progresiva, presentar visualizaciones, ofrecer interpretaciones en español profesional y aplicar métodos estadísticos con fundamento teórico. Aunque no se puede asegurar que las recomendaciones metodológicas del agente sean siempre óptimas, sí se ha comprobado que éstas están fundamentadas en criterios técnicos.

Desde una perspectiva conceptual, este trabajo pone en valor el potencial de la inferencia causal en ciencia de datos aplicada, y demuestra cómo su complejidad puede ser reducida mediante el uso de interfaces conversacionales bien diseñadas. También se ha comprobado que, con una arquitectura basada en agentes, es posible automatizar procesos analíticos sofisticados sin perder rigor técnico, lo que representa un paso hacia la democratización de herramientas avanzadas en entornos no especializados.

En síntesis, este proyecto ha servido tanto para validar una solución técnica concreta, como para adquirir una comprensión profunda sobre el diseño de sistemas LLM asistidos, la importancia del *prompting* y la integración práctica de técnicas estadísticas y causalidad. Ha sido también una oportunidad para reforzar competencias en desarrollo web, arquitectura de sistemas y razonamiento aplicado, todo ello en un entorno de desarrollo realista.

## 7.2 Relación con la formación académica

---

En relación con la formación recibida durante el Grado, este trabajo ha permitido aplicar de manera integrada diversos conocimientos adquiridos en distintas asignaturas. En el ámbito de la estadística, se han puesto en práctica los fundamentos vistos en la asignatura *Métodos Estadísticos para la Toma de Decisiones*. En cuanto a la programación, los conceptos trabajados en *Programación y Estructuras de Datos* sirvieron de base para el desarrollo del sistema. Para la representación gráfica, se aplicaron los conocimientos de la asignatura de *Visualización*, fundamentales para generar salidas comprensibles.

Asimismo, se han reforzado competencias relacionadas con la arquitectura de software y el desarrollo web, adquiridas tanto en la carrera como de manera autónoma. Todo ello conecta de forma directa con el ámbito de la *Ciencia de Datos*, entendido como la combinación de análisis estadístico, programación y comunicación de resultados.

El sistema desarrollado constituye una aproximación práctica a la ciencia de datos aplicada, al demostrar cómo es posible diseñar herramientas que guíen procesos de análisis exploratorio y causal de forma asistida y accesible.

---

## CAPÍTULO 8

# Líneas futuras

---

En un futuro convendría ampliar las capacidades del sistema para que el agente no se limite únicamente a describir o resumir el *dataset*, sino que también pueda actuar directamente sobre los datos cuando el usuario lo solicite, implementando tareas automáticas de preprocesamiento. Entre las funciones más útiles que se podrían incorporar destacan:

- **Gestión activa de valores nulos y atípicos:** actualmente, el agente identifica correctamente estos elementos, pero sería deseable que también ofreciera (de forma interactiva) opciones para imputar, eliminar o transformar los valores problemáticos, en función del tipo de variable y del análisis a realizar. Por ejemplo, el sistema podría proponer imputaciones condicionales, aplicar técnicas robustas de normalización o generar resúmenes antes/después del tratamiento aplicado.
- **Sugerencias automáticas de limpieza y transformación:** otro avance sería que el agente, tras analizar el *dataset*, pudiera detectar columnas redundantes, variables mal codificadas, escalas incompatibles o casos de colinealidad, y ofrecer al usuario una lista de transformaciones recomendadas, justificando cada una en lenguaje natural.
- **Limpieza automatizada y preprocesamiento inteligente:** incluir un módulo que permita identificar y tratar valores nulos de forma contextualizada, detectar y eliminar duplicados, gestionar valores atípicos según la distribución de cada variable y homogeneizar formatos de columnas (fechas, números, categorías).
- **Soporte para datos temporales o *panel data*:** adaptar el asistente al análisis de series temporales y datos longitudinales, incorporando métodos como *difference-in-differences* o técnicas de detección de estacionalidad y rupturas estructurales.
- **Ampliación del flujo causal con más técnicas estadísticas:** aunque actualmente se utilizan métodos clásicos de DoWhy (regresión, matching, IPW, etc.), el sistema podría incorporar nuevos enfoques como métodos bayesianos o técnicas de aprendizaje automático causal (*Causal Forests*, *DeepIV*).
- **Análisis de subgrupos y efectos heterogéneos:** en futuras versiones, el agente podría detectar automáticamente si el efecto del tratamiento varía en distintos subgrupos del *dataset* (por ejemplo, por marca, segmento o país), y ofrecer estimaciones desagregadas junto con su interpretación causal.
- **Generación automática de informes PDF o *notebooks*:** una línea muy práctica sería que, al final del análisis, el sistema pudiera generar automáticamente un informe descargable que contenga las visualizaciones, explicaciones del agente, resultados del flujo causal y conclusiones.

- **Mayor integración entre visualización y diálogo:** el diseño actual combina texto y gráficos, pero en futuras iteraciones podría explorarse una interfaz más fluida en la que el usuario pueda hacer clic sobre un gráfico para profundizar, modificar parámetros o hacer preguntas contextuales sobre lo mostrado.
- **Compatibilidad con modelos LLM locales o especializados:** finalmente, para mejorar la privacidad, escalabilidad o rendimiento, sería interesante permitir que el sistema funcione también con modelos locales, *open-source* o entrenados para tareas analíticas específicas, más allá de los modelos comerciales generales.

# Bibliografía

---

- [1] C. L. Montalván-Vélez, J. N. Mogrovejo-Zambrano, I. J. Romero-Vitte y M. L. D. C. Pinargote-Carrera, "Introducción a la Inteligencia Artificial: Conceptos Básicos y Aplicaciones Cotidianas," *Journal of Economic and Social Science Research*, vol. 4, no. 1, pp. 173–183, 2024. Disponible en: <https://doi.org/10.55813/gaea/jessr/v4/n1/93>.
- [2] IBM, "Agentes de inteligencia artificial," *IBM Think*, 3 julio 2024. Disponible en: <https://www.ibm.com/es-es/think/topics/ai-agents>.
- [3] Francisco José Sánchez Martín, "ESEID: Análisis exploratorio de datos," s.f. Disponible en: <https://eseid.com/analisis-exploratorio-de-datos/>.
- [4] PyPI, "pandas-profiling," 31 enero 2023. Disponible en: <https://pypi.org/project/pandas-profiling/>.
- [5] Streamlit, "Streamlit," 2025. Disponible en: <https://streamlit.io>.
- [6] Hugging Face, "Hugging Face Spaces," 2025. Disponible en: <https://huggingface.co/spaces>.
- [7] LangChain, "LangChain," 2025. Disponible en: <https://www.langchain.com>.
- [8] Dot CSV, "¿Qué es LangChain? | Inteligencia Artificial Explicable," YouTube, 10 julio 2023. Disponible en: [https://www.youtube.com/watch?v=CBRE\\_Me1IQ0](https://www.youtube.com/watch?v=CBRE_Me1IQ0).
- [9] S. Mordechai, "AI Assistant with Vector Store," GitHub, s.f. Disponible en: <https://github.com/sharonmordechai/ai-assistant-with-vector-store>.
- [10] P. Tiwari, "LangChain GenAI Data Assistant," GitHub, s.f. Disponible en: <https://github.com/PragyanTiwari/LangChain-GenAI-Data-Assistant>.
- [11] OpenAI, "OpenAI," 2025. Disponible en: <https://openai.com>.
- [12] S. Minaee *et al.*, "Large Language Models: A Survey," 23 marzo 2025. Disponible en: <https://arxiv.org/pdf/2402.06196>.
- [13] OpenAI, "GPT-4," 2025. Disponible en: <https://openai.com/es-ES/index/gpt-4>.
- [14] Google DeepMind, "Gemini," s.f. Disponible en: <https://deepmind.google/models/gemini/>.
- [15] Meta, "LLaMA," s.f. Disponible en: <https://www.llama.com>.
- [16] A. Vaswani *et al.*, "Attention Is All You Need," en *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. Disponible en: [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf).

- [17] L.-F. Bouchard y L. Peters, “Building LLMs for Production,” *Towards AI*, octubre 2024. Disponible en: [https://learning.oreilly.com/library/view/building-llms-for/9798324731472/index\\_split\\_028.html](https://learning.oreilly.com/library/view/building-llms-for/9798324731472/index_split_028.html).
- [18] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models,” 29 marzo 2023. Disponible en: <https://arxiv.org/pdf/2203.15556>.
- [19] LangChain, “Introduction (v0.3),” 2025. Disponible en: <https://python.langchain.com/docs/introduction/>.
- [20] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” versión 6, 10 enero 2023. Disponible en: <https://arxiv.org/abs/2201.11903>.
- [21] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” versión 3, 10 marzo 2023. Disponible en: <https://arxiv.org/abs/2210.03629>.
- [22] C. Pérez López, “Análisis Exploratorio de Datos,” 2018. Disponible en: [https://www.academia.edu/35896261/Analisis\\_Exploratorio\\_de\\_Datos](https://www.academia.edu/35896261/Analisis_Exploratorio_de_Datos).
- [23] J. W. Tukey, *Exploratory Data Analysis*, 1977. Disponible en: <https://consoleflare.com/blog/wp-content/uploads/2022/09/Exploratory-Data-Analysis-1977-John-Tukey.pdf>.
- [24] M. Komorowski, D. C. Marshall, J. D. Saliccioli y Y. Crutain, “Exploratory Data Analysis,” en *Secondary Analysis of Electronic Health Records*, MIT Critical Data, cap. 15, 2016. Disponible en: [https://www.researchgate.net/publication/308007227\\_Exploratory\\_Data\\_Analysis](https://www.researchgate.net/publication/308007227_Exploratory_Data_Analysis).
- [25] M. Facure, *Causal Inference in Python: Applying Causal Inference in the Tech Industry*, O’Reilly Media, julio 2023. Disponible en: <https://learning.oreilly.com/library/view/causal-inference-in/9781098140243/ch03.html/>.
- [26] F. Yirá, “Introducción a los diagramas de causalidad (DAGs),” Blog de Francisco Yirá, 19 julio 2021. Disponible en: <https://www.franciscoyira.com/es/post/2021-07-11-diagramas-causalidad-cap-3-causal-inference-mixtape/>.
- [27] N. Huntington-Klein, “The Effect: Causal Graphs,” s.f. Disponible en: <https://nickchk.com/causalgraphs.html>.
- [28] N. Huntington-Klein, *The Effect: An Introduction to Research Design and Causality*, cap. “Statistical Adjustment”, 2021. Disponible en: <https://theeffectbook.net/ch-StatisticalAdjustment.html>.
- [29] Google Cloud, *Gemini 2.0 Flash*. Diciembre 2024. Disponible en: <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash?hl=es-419>.
- [30] Mistral AI, “Mistral Small 3”. s.f. Disponible en: <https://mistral.ai/news/mistral-small-3>.
- [31] Hugging Face, “Models”. s.f. Disponible en: <https://huggingface.co/models>.
- [32] PyWhy, “DoWhy User Guide: Introduction (v0.13),” s.f. Disponible en: [https://www.pywhy.org/dowhy/v0.13/user\\_guide/intro.html](https://www.pywhy.org/dowhy/v0.13/user_guide/intro.html).
- [33] Microsoft, “Visual Studio Code,” s.f. Disponible en: <https://code.visualstudio.com>.

- [34] PyPI, “python-dotenv,” 2025. Disponible en: <https://pypi.org/project/python-dotenv/>.
- [35] PyData, “pandas,” 2023. Disponible en: <https://pandas.pydata.org>.
- [36] PyPI, “langchain-google-genai,” 2025. Disponible en: <https://pypi.org/project/langchain-google-genai/>.
- [37] LangChain, “PromptTemplate (v0.3.23),” s.f. Disponible en: [https://python.langchain.com/api\\_reference/core/prompts/langchain\\_core.prompts.prompt.PromptTemplate.html](https://python.langchain.com/api_reference/core/prompts/langchain_core.prompts.prompt.PromptTemplate.html).
- [38] LangChain, “LLM Chain (v0.3.23),” s.f. Disponible en: [https://python.langchain.com/api\\_reference/langchain/chains/langchain\\_chains.llm.LLMChain.html](https://python.langchain.com/api_reference/langchain/chains/langchain_chains.llm.LLMChain.html).
- [39] LangChain, “PythonAstREPLTool, LangChain Experimental,” s.f. Disponible en: [https://python.langchain.com/api\\_reference/experimental/tools/langchain\\_experimental.tools.python.tool.PythonAstREPLTool.html](https://python.langchain.com/api_reference/experimental/tools/langchain_experimental.tools.python.tool.PythonAstREPLTool.html).
- [40] NetworkX, DiGraph. *NetworkX Documentation (v3.5)*, s.f. Disponible en: <https://networkx.org/documentation/stable/reference/classes/digraph.html>.
- [41] PyPI, *pydot*, s.f. Disponible en: <https://pypi.org/project/pydot/>.
- [42] PyWhy, *Identifying Causal Effect. DoWhy User Guide (v0.11)*, 2022. Disponible en: [https://www.pywhy.org/dowhy/v0.11/user\\_guide/causal\\_tasks/estimating\\_causal\\_effects/identifying\\_causal\\_effect/index.html](https://www.pywhy.org/dowhy/v0.11/user_guide/causal_tasks/estimating_causal_effects/identifying_causal_effect/index.html).
- [43] PyWhy, *DoWhy Estimation Methods. Example Notebooks (v0.9)*, s.f. Disponible en: [https://www.pywhy.org/dowhy/v0.9/example\\_notebooks/dowhy\\_estimation\\_methods.html](https://www.pywhy.org/dowhy/v0.9/example_notebooks/dowhy_estimation_methods.html).
- [44] PyWhy, *Random Common Cause. DoWhy User Guide (v0.11)*, 2022. Disponible en: [https://www.pywhy.org/dowhy/v0.11/user\\_guide/refuting\\_causal\\_estimates/refuting\\_effect\\_estimates/random\\_common\\_cause.html](https://www.pywhy.org/dowhy/v0.11/user_guide/refuting_causal_estimates/refuting_effect_estimates/random_common_cause.html).
- [45] LangChain, *create\_pandas\_dataframe\_agent (v0.3.23)*, 2025. Disponible en: [https://python.langchain.com/api\\_reference/experimental/agents/langchain\\_experimental.agents.agent\\_toolkits.pandas.base.create\\_pandas\\_dataframe\\_agent.html](https://python.langchain.com/api_reference/experimental/agents/langchain_experimental.agents.agent_toolkits.pandas.base.create_pandas_dataframe_agent.html).
- [46] Mistral AI, *Documentation*, 2025. Disponible en: <https://docs.mistral.ai>.
- [47] G. Nel, *EVs One Electric Vehicle Dataset*. Kaggle, 2020. Disponible en: <https://www.kaggle.com/datasets/geoffnel/evs-one-electric-vehicle-dataset>.

## ANEXO I

# Relación con los Objetivos de Desarrollo Sostenible (ODS)

Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible (ODS).

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.	X			
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.		X		
ODS 9. Industria, innovación e infraestructuras.	X			
ODS 10. Reducción de las desigualdades.		X		
ODS 11. Ciudades y comunidades sostenibles.				X
ODS 12. Producción y consumo responsables.				X
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.				X
ODS 17. Alianzas para lograr los objetivos.			X	

### ODS 4. Educación de calidad — Nivel de relación: Alto

La herramienta desarrollada tiene un impacto significativo en la alfabetización digital y analítica, uno de los pilares fundamentales para garantizar una educación inclusiva y de calidad. Permite que personas sin conocimientos técnicos puedan acceder a técnicas complejas de análisis estadístico, visualización de datos e incluso inferencia causal, simplemente utilizando lenguaje natural. Esto supone una democratización del conocimiento que contribuye a la formación de estudiantes, docentes y profesionales de muy diversos ámbitos, no necesariamente técnicos.

Además, la herramienta puede utilizarse en contextos educativos formales y no formales, tanto como recurso didáctico en asignaturas de análisis de datos, como en iniciativas de autoaprendizaje o formación continua. El sistema ofrece explicaciones comprensibles, genera resúmenes detallados sin necesidad de código, y actúa como una interfaz de aprendizaje activo que favorece el desarrollo de competencias analíticas.

### **ODS 8. Trabajo decente y crecimiento económico — Nivel de relación: Medio**

El proyecto realizado contribuye al crecimiento económico y la mejora de la productividad laboral mediante la automatización de procesos de análisis de datos. Profesionales de distintos sectores pueden beneficiarse del sistema para obtener *insights* sin necesidad de conocimientos técnicos profundos. Esto reduce barreras de entrada y optimiza los tiempos de análisis, aumentando la eficiencia en pequeñas empresas, *startups* o proyectos individuales que no disponen de grandes equipos de ciencia de datos.

Además, al estar construido con herramientas de código abierto (Python, Streamlit, LangChain, DoWhy), el sistema fomenta un modelo tecnológico sostenible y accesible, que puede adaptarse o escalarse según las necesidades de distintos entornos laborales. A largo plazo, puede promover nuevas oportunidades de empleo o mejora profesional, especialmente para perfiles que buscan adquirir competencias en análisis de datos sin pasar por una formación técnica extensa.

### **ODS 9. Industria, innovación e infraestructuras — Nivel de relación: Alto**

Este es uno de los ODS con mayor vinculación al Trabajo de Fin de Grado desarrollado. El asistente diseñado representa una innovación técnica y funcional al integrar modelos de lenguaje de última generación (LLMs), *prompting* estructurado, herramientas estadísticas y capacidades de razonamiento automatizado. La solución propuesta ofrece una forma novedosa de interactuar con los datos, permitiendo un análisis profundo sin programación, lo que supone un avance dentro de las infraestructuras digitales de análisis.

Además, la posibilidad de ejecutar análisis causales, construir grafos DAG, diagnosticar identificabilidad y estimar efectos estadísticos convierte al sistema en una herramienta potente, escalable y aplicable en entornos de investigación, empresa o administración pública. Esto contribuye directamente a la modernización de las infraestructuras digitales y a la promoción de la innovación responsable, metas fundamentales dentro del ODS 9.

### **ODS 10. Reducción de las desigualdades — Nivel de relación: Medio**

La eliminación de barreras técnicas que ofrece el sistema diseñado reduce desigualdades en el acceso al conocimiento y al uso de tecnologías analíticas. Esto tiene un efecto directo sobre colectivos que, por razones de formación, recursos económicos o contexto, no tienen acceso habitual a herramientas de análisis avanzado.

El proyecto actúa como un facilitador para perfiles no expertos: investigadores en ciencias sociales, personal educativo, microempresas, estudiantes de disciplinas ajenas a la informática, etc. Al usar lenguaje natural como vía de interacción, se consigue una interfaz verdaderamente inclusiva que pone el análisis de datos al alcance de todos, no solo de quienes saben programar. Esta contribución se alinea con la meta 10.2 de los ODS, que promueve la inclusión social, económica y política de todas las personas.



**ODS 17. Alianzas para lograr los objetivos — Nivel de relación: Bajo**

Aunque no es un objetivo central del trabajo, el sistema tiene el potencial de ser una herramienta colaborativa en equipos multidisciplinarios donde expertos en dominio (por ejemplo, en salud, economía, sociología o medio ambiente) puedan trabajar conjuntamente con datos sin depender de expertos técnicos. Esto facilita alianzas entre perfiles diversos, promoviendo una cultura de toma de decisiones basada en evidencia.

Además, al estar construido sobre tecnologías abiertas y modulares, el proyecto permite su extensión o integración en otras iniciativas tecnológicas o académicas, lo que podría facilitar colaboraciones interinstitucionales, tanto en docencia como en investigación. Por tanto, aunque su impacto directo es limitado, existen claras posibilidades de contribuir a alianzas estratégicas si el proyecto se escala o difunde correctamente.

---

## ANEXO II

# Código fuente del archivo `main.py`

---

El archivo `main.py` constituye el punto de entrada principal de la aplicación.. Su función es orquestar la navegación entre las distintas secciones de la herramienta, además de definir la interfaz visual inicial y los estilos personalizados.

```
1 import modules.introduction as introduction
2 import modules.upload as upload
3 import modules.summary as summary
4 import modules.graphics as graphics
5 import modules.causal as causal
6 import streamlit as st
7
8 # Inicializar 'page' al principio
9 if 'page' not in st.session_state:
10 st.session_state['page'] = 'welcome'
11
12 # Establecer la configuracin de la pgina
13 st.set_page_config(page_title="TFG", page_icon="", layout="wide",
14 initial_sidebar_state="expanded")
15
16 # Estilos personalizados con CSS
17 st.markdown(
18 """
19 <style>
20 .css-1d391kg {display: none;} /* Oculta la barra superior */
21
22 .main {
23 background-color: white;
24 padding: 30px;
25 border-radius: 12px;
26 animation: fadeIn 0.8s ease-in-out;
27 }
28
29 @keyframes fadeIn {
30 from {opacity: 0;}
31 to {opacity: 1;}
32 }
33
34 .sidebar .sidebar-content {
35 background-color: #2E4053;
36 color: white;
37 }
38
39 .stButton>button {
40 width: 100%;
41 background-color: #A4C6E8;
42 color: white;
```

```

42 border-radius: 12px;
43 padding: 12px;
44 font-size: 16px;
45 font-weight: bold;
46 border: none;
47 transition: all 0.2s ease-in-out;
48 }
49
50 .stButton>button:hover {
51 background-color: #85A9D4;
52 transform: scale(1.02);
53 cursor: pointer;
54 }
55
56 .stButton>button:active {
57 background-color: #7799C8;
58 transform: scale(0.98);
59 }
60
61 .title-text {
62 font-size: 48px !important;
63 font-weight: bold;
64 color: #2E86C1;
65 text-align: center;
66 margin-bottom: 20px;
67 line-height: 1.1;
68 white-space: nowrap;
69 overflow: hidden;
70 text-overflow: ellipsis;
71 }
72
73
74 .subtitle-text {
75 font-size: 22px;
76 color: #555;
77 text-align: center;
78 margin-bottom: 30px;
79 }
80
81 hr {
82 border: none;
83 border-top: 2px solid #A4C6E8;
84 margin: 30px 0;
85 }
86
87 ul {
88 font-size: 18px;
89 }
90
91 .element-container h3 {
92 font-size: 24px;
93 color: #2E4053;
94 margin-top: 25px;
95 }
96 </style>
97 """
98 unsafe_allow_html=True,
99)
100
101 # Barra lateral de navegacin con iconos
102 with st.sidebar:
103 st.markdown("## Navegacin")
104 if st.button("Introduccin"):
105 st.session_state['page'] = 'intro'

```

```

106 if st.button("Subida de archivo"):
107 st.session_state['page'] = 'upload'
108 if st.button("Resumen Estadístico"):
109 st.session_state['page'] = 'summary'
110 if st.button("Generación de Gráficos"):
111 st.session_state['page'] = 'graphics'
112 if st.button("Inferencia Causal"):
113 st.session_state['page'] = 'causal'
114
115 # Contenedor principal que se vacía dinámicamente al cambiar de página
116 page_container = st.empty()
117
118 with page_container.container():
119 if st.session_state['page'] == 'welcome':
120 st.markdown('<div class="main">', unsafe_allow_html=True)
121
122 col1, col2, col3 = st.columns([1, 2, 1])
123 with col2:
124 st.markdown('<p class="title-text"> Bienvenido a mi TFG !</p>',
125 unsafe_allow_html=True)
126 st.markdown('<p class="subtitle-text">Una herramienta interactiva para
127 explorar, visualizar y analizar datos fácilmente.</p>', unsafe_allow_html=True)
128
129 st.markdown('<hr>', unsafe_allow_html=True)
130
131 # Secciones rápidas con emojis e iconos
132 col1, col2 = st.columns(2)
133
134 with col1:
135 st.markdown("### Subida de Archivos")
136 st.markdown("- Sube archivos CSV o Excel")
137 st.markdown("- Preparación de datos lista para análisis")
138
139 st.markdown("### Análisis Inteligente")
140 st.markdown("- Perfilado estadístico")
141 st.markdown("- Identificación de valores nulos, outliers y duplicados")
142 st.markdown("- Sugerencias de nuevas variables")
143
144 with col2:
145 st.markdown("### Gráficos Personalizados")
146 st.markdown("- Generación de gráficos por lenguaje natural")
147 st.markdown("- Uso de seaborn y matplotlib con estilos personalizados")
148 st.markdown("- Descarga directa como imagen")
149
150 st.markdown("### Inferencia Causal")
151 st.markdown("- Evaluación de relaciones causa-efecto")
152 st.markdown("- Métodos automáticos de selección de tratamiento")
153 st.markdown("- Visualización y explicaciones detalladas")
154
155 st.markdown("<hr>", unsafe_allow_html=True)
156
157 st.markdown(
158 "<p style='text-align:center; color:#888; font-size:16px;'>Desarrollado con
159 por Marín García Mas como parte del Trabajo de Fin de Grado</p>",
160 unsafe_allow_html=True,
161)
162
163 st.markdown("</div>", unsafe_allow_html=True)
164
165 elif st.session_state['page'] == 'intro':
166 introduction.mostrar_introduccion()
167
168 elif st.session_state['page'] == 'upload':
169 upload.mostrar_subida_archivo()

```

```
167
168 elif st.session_state['page'] == 'summary':
169 summary.mostrar_agente()
170
171 elif st.session_state['page'] == 'graphics':
172 graphics.mostrar_graficos()
173
174 elif st.session_state['page'] == 'causal':
175 causal.mostrar_inferencia_causal()
```

**Listing II.1:** main.py: interfaz principal

---

## ANEXO III

# Módulo de subida de archivos (upload.py)

---

Este módulo implementa la interfaz de carga de archivos en la aplicación, permitiendo al usuario subir nuevos datasets en formato CSV o Excel, o bien seleccionar entre los ya existentes en el sistema.

```
1 import streamlit as st
2 import auto_doc.services.useful_services as utils
3
4 def mostrar_subida_archivo():
5 with st.container():
6 st.markdown("## Subida de Archivo")
7
8 # Mostrar archivos previos si existen
9 datasets = utils.list_all_datasets()
10 if datasets:
11 opciones = [d["file_name"] for d in datasets]
12 seleccion = st.selectbox(
13 "Selecciona un archivo previamente subido (opcional)",
14 [" Elige un archivo..."] + opciones
15)
16
17 if seleccion != " Elige un archivo...":
18 seleccionado = next(d for d in datasets if d["file_name"] == seleccion)
19 st.success(f" Archivo seleccionado: `{seleccionado['file_name']}`")
20 st.session_state["file_path"] = seleccionado["file_path"]
21 return
22
23 # Subida nueva
24 archivo = st.file_uploader("Sube tu archivo CSV o Excel", type=["csv", "xlsx"])
25
26 if st.button("Siguiente"):
27 if not archivo:
28 st.error("Por favor, sube un archivo antes de continuar.")
29 return
30
31 file_path = utils.save_uploaded_file(archivo)
32 st.session_state["file_path"] = str(file_path)
33
34 st.success(" Archivo subido con xito. Puedes continuar!")
```

Listing III.1: upload.py: interfaz para la subida y gestión de archivos

---

## ANEXO IV

# Funciones auxiliares en usefulservices.py

---

El siguiente listado muestra el contenido completo del archivo `useful_services.py`, que implementa funciones auxiliares necesarias para el funcionamiento de la aplicación.

```
1 import json
2 import os
3 from pathlib import Path
4
5 BASE_DIR = Path("auto_doc/files")
6 BASE_DIR.mkdir(parents=True, exist_ok=True)
7
8 def extract_filename(file_path):
9 """Obtiene el nombre del archivo sin extensin."""
10 file_name, _ = os.path.splitext(os.path.basename(file_path))
11 return file_name
12
13 def encode_file_name(name: str) -> str:
14 """Codifica el nombre del proyecto para evitar espacios o caracteres problematicos."""
15 return name.lower().replace(" ", "_")
16
17 def save_uploaded_file(uploaded_file):
18 """Guarda el archivo en 'auto_doc/files/' y devuelve la ruta."""
19 file_path = BASE_DIR / uploaded_file.name
20 with open(file_path, "wb") as f:
21 f.write(uploaded_file.getbuffer())
22 return file_path
23
24 import os
25 import pandas as pd
26
27
28 def load_file(file_path: str) -> pd.DataFrame:
29 """
30 Carga archivos en formato CSV o Excel (XLSX) y devuelve un DataFrame con un mximo de
31 filas y columnas.
32
33 Parmetros:
34 - file_path (str): Ruta del archivo.
35 - max_rows (int): Nmero mximo de filas a cargar. Por defecto 200.
36 - max_columns (int): Nmero mximo de columnas a cargar. Por defecto 10.
37
38 Retorna:
39 - pd.DataFrame: Contenido del archivo como DataFrame.
40 """
```

```

40 ext = os.path.splitext(file_path)[1].lower()
41
42 if not os.path.exists(file_path):
43 raise FileNotFoundError(f"El archivo no existe: {file_path}")
44
45 if ext == ".csv":
46 df = pd.read_csv(file_path)
47 elif ext == ".xlsx":
48 df = pd.read_excel(file_path)
49 else:
50 raise ValueError(f"Tipo de archivo no soportado: {ext}. Solo se permiten .csv y
51 .xlsx")
52
53 return df
54
55 def list_all_datasets():
56 """
57 Lista todos los archivos disponibles en auto_doc/files/ con extensiones .csv o .xlsx.
58 Retorna una lista de diccionarios con nombre y ruta.
59 """
60 datasets = []
61 for file in BASE_DIR.glob("*"):
62 if file.suffix.lower() in [".csv", ".xlsx"]:
63 datasets.append({
64 "file_name": file.name,
65 "file_path": str(file.resolve())
66 })
67 return datasets
68
69 import streamlit as st
70
71 def mostrar_respuesta_agente(clave, respuesta=None):
72 """
73 Muestra la respuesta del agente desde session_state si existe,
74 o guarda una nueva si se proporciona. No muestra ttulo.
75
76 Parmetros:
77 - clave (str): nombre de la variable en session_state donde se almacena la respuesta.
78 - respuesta (str): contenido nuevo para guardar (opcional).
79 """
80 if respuesta is not None:
81 st.session_state[clave] = respuesta
82
83 if clave in st.session_state:
84 st.markdown(st.session_state[clave])

```

**Listing IV.1:** *useful\_services.py : funciones auxiliares*



---

## ANEXO V

# Código fuente del agente (causalagent.py)

---

A continuación se muestra el contenido completo del archivo `causalagent.py`, *quede fine el agente, susherram*

```
1 # causal_agent.py
2 # Agent specifically designed for causal inference tasks with DoWhy tools.
3
4 from abc import ABC, abstractmethod
5 from typing import Dict, Optional
6 import streamlit as st
7 import pandas as pd
8 import numpy as np
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.metrics import roc_auc_score
11 from sklearn.preprocessing import LabelEncoder
12 from scipy.stats import chi2_contingency
13 import ast
14 from dowhy import CausalModel
15
16 # LangChain imports
17 from langchain.chains import LLMChain
18 from langchain.prompts import PromptTemplate
19 from langchain.memory import ConversationBufferMemory
20 from langchain.agents import initialize_agent, Tool, AgentType, AgentExecutor
21 from langchain_experimental.tools.python.tool import PythonAstREPLTool
22 from langchain_google_genai import ChatGoogleGenerativeAI
23
24 # -----
25 # Chat Model Loader
26 # -----
27 def get_causal_chat_model(temperature=0.0):
28 return ChatGoogleGenerativeAI(
29 model="gemini-2.0-flash-001",
30 api_key="AIzaSyCQbYXe_FBupQ3skmHo1YfPeDfeJvm0eQU",
31 temperature=temperature
32)
33
34 # -----
35 # Base Class
36 # -----
37 class BaseLLM(ABC):
38 def __init__(self):
39 self.chunk_size = 1
40 self.temperature = 0.0
41
42 @abstractmethod
```

```

43 def load_llm_chat(self):
44 pass
45
46 @abstractmethod
47 def run(self, prompt_text: str, params_dict: Optional[Dict[str, str]] = None) -> str:
48 pass
49
50 # -----
51 # Concrete LLM Class
52 # -----
53 class CausalLLM(BaseLLM):
54 def load_llm_chat(self):
55 return get_causal_chat_model(temperature=self.temperature)
56
57 def transform(self, llm, prompt_text: str, params_dict: Dict[str, str]) -> str:
58 prompt = PromptTemplate(input_variables=list(params_dict.keys()),
59 template=prompt_text)
60 chain = LLMChain(llm=llm, prompt=prompt, verbose=False)
61 result = chain(params_dict)
62 return result["text"]
63
64 def run(self, prompt_text: str, params_dict: Optional[Dict[str, str]] = None) -> str:
65 llm = self.load_llm_chat()
66 return self.transform(llm, prompt_text, params_dict or {})
67
68 # -----
69 # Causal Tools
70 # -----
71 def generate_dag(resumen_str: str) -> str:
72 import json
73 resumen_str = resumen_str.strip()
74 if resumen_str.startswith("```json"):
75 resumen_str = resumen_str[len("```json"):].strip()
76 if resumen_str.endswith("```"):
77 resumen_str = resumen_str[:-3].strip()
78 if not resumen_str.startswith("{") or not resumen_str.endswith("}"):
79 raise ValueError(f"JSON invlido recibido en generar_dag:\n\n{resumen_str}")
80
81 resumen = json.loads(resumen_str)
82 treatment = resumen["treatment"]
83 outcome = resumen["outcome"]
84 corr = resumen["correlaciones"]
85 conf_cat = resumen["confusoras_categoricas"]
86
87 def quote(var):
88 return f'"{var}"'
89
90 dot = [
91 "digraph {",
92 f'"{quote(treatment)}" [style=filled, fillcolor=lightgreen];',
93 f'"{quote(outcome)}" [style=filled, fillcolor=lightgrey];'
94]
95
96 for c in conf_cat:
97 dot.append(f'"{quote(c)}" [style=filled, fillcolor=orange];')
98
99 for m in corr:
100 dot.append(f'"{quote(m)}" [style=filled, fillcolor=lightblue];')
101
102 dot.append(f'"{quote(treatment)}" -> "{quote(outcome)}";')
103
104 for c in conf_cat:
105 dot.append(f'"{quote(c)}" -> "{quote(treatment)}";')
106 dot.append(f'"{quote(c)}" -> "{quote(outcome)}";')
107
108 for m in corr:
109 dot.append(f'"{quote(treatment)}" -> "{quote(m)}";')
110 dot.append(f'"{quote(m)}" -> "{quote(outcome)}";')

```

```

106 dot.append("{}")
107 return "\n".join(dot)
108 from dowhy.causal_refuters.graph_refuter import GraphRefuter
109
110 def identify_causal_effect(*args, **kwargs):
111 model = st.session_state["causal_model"]
112 estimand = model.identify_effect()
113 st.session_state["identified_estimand"] = estimand
114 backdoors = estimand.get_backdoor_variables()
115 return f"{estimand}\n\nBackdoor variables: {backdoors}"
116
117 def diagnostico_causal_df(df: pd.DataFrame, treatment_col: str, backdoor_vars: list) ->
dict:
118 resultado = {}
119 treatment_counts = df[treatment_col].value_counts()
120 total = treatment_counts.sum()
121 prop_treated = treatment_counts.get(1, 0) / total if 1 in treatment_counts else 0
122
123 resultado["treatment_info"] = {
124 "proporcion_tratados": round(prop_treated, 3),
125 "n_tratados": int(treatment_counts.get(1, 0)),
126 "n_control": int(treatment_counts.get(0, 0))
127 }
128
129 backdoor_info = {}
130 treated_df = df[df[treatment_col] == 1]
131 control_df = df[df[treatment_col] == 0]
132
133 for var in backdoor_vars:
134 if df[var].dtype == "object" or df[var].nunique() < 10:
135 try:
136 tab = pd.crosstab(df[var], df[treatment_col])
137 chi2, p, _, _ = chi2_contingency(tab)
138 cramers_v = np.sqrt(chi2 / (total * (min(tab.shape) - 1)))
139 backdoor_info[var] = {
140 "tipo": "categorica",
141 "cramers_v": round(cramers_v, 3)
142 }
143 except:
144 continue
145 else:
146 treated_vals = treated_df[var].dropna()
147 control_vals = control_df[var].dropna()
148 pooled_std = np.sqrt(((treated_vals.std() ** 2) + (control_vals.std() ** 2))
/ 2)
149 std_diff = abs(treated_vals.mean() - control_vals.mean()) / pooled_std if
pooled_std > 0 else 0
150 backdoor_info[var] = {
151 "tipo": "numerica",
152 "diferencia_media_estandarizada": round(std_diff, 3)
153 }
154
155 resultado["backdoor_vars_info"] = backdoor_info
156
157 df_copy = df[backdoor_vars + [treatment_col]].dropna().copy()
158 X = df_copy[backdoor_vars]
159 y = df_copy[treatment_col]
160
161 X_encoded = X.copy()
162 for col in X_encoded.select_dtypes(include="object").columns:
163 X_encoded[col] = LabelEncoder().fit_transform(X_encoded[col])
164
165 model = LogisticRegression(solver="liblinear")
166 model.fit(X_encoded, y)

```

```

167 scores = model.predict_proba(X_encoded)[: , 1]
168
169 auc = roc_auc_score(y, scores)
170 extremos = np.any((scores < 0.05) | (scores > 0.95))
171
172 resultado["propensity_score"] = {
173 "auc": round(auc, 3),
174 "extremos": bool(extremos),
175 "overlap_visual": "moderado" if 0.1 < scores.mean() < 0.9 else "limitado"
176 }
177
178 recomendaciones = {
179 "usar_matching": not extremos and auc < 0.85,
180 "usar_ipw": not extremos and auc < 0.85,
181 "usar_regresion": True
182 }
183
184 resultado["recomendaciones_metodologicas"] = recomendaciones
185 return resultado
186
187 def make_diagnostico_tool(df: pd.DataFrame):
188 def diagnostico_tool_wrapped(input: str) -> dict:
189 try:
190 params = ast.literal_eval(input)
191 treatment_col = params["treatment_col"]
192 backdoor_vars = params["backdoor_vars"]
193 return diagnostico_causal_df(df, treatment_col, backdoor_vars)
194 except Exception as e:
195 raise ValueError(f"Error al ejecutar diagnostico causal: {e}")
196
197 return Tool.from_function(
198 func=diagnostico_tool_wrapped,
199 name="diagnostico_causal_df_tool",
200 description="Diagnostico de balance y score de propensin. Requiere un dict con:
201 'treatment_col' y 'backdoor_vars'. El DataFrame ya est disponible como `df` interno."
202)
203
204 import json
205 import numpy as np
206
207 def estimate_causal_effect_tool(input: str = "") -> str:
208 try:
209 model = st.session_state.get("causal_model")
210 estimand = st.session_state.get("identified_estimand")
211 metodo = st.session_state.get("metodo_seleccionado")
212
213 if model is None or estimand is None or metodo is None:
214 return json.dumps({
215 "error": " Faltan elementos clave: modelo causal, estimand o mtodo
216 seleccionado."
217 })
218
219 treatment_list = getattr(estimand, "treatment", ["treatment"])
220 instrument_list = getattr(estimand, "instrumental_variables", [["Z"]])
221
222 method_params_dict = {
223 "iv.instrumental_variable": {
224 "iv_instrument_name": instrument_list[0][0] if instrument_list and
225 instrument_list[0] else "Z"
226 },
227 "iv.regression_discontinuity": {
228 "rd_variable_name": treatment_list[0],
229 "rd_threshold_value": 0.5,
230 "rd_bandwidth": 0.15

```

```

228 },
229 "backdoor.propensity_score_weighting": {
230 "weighting_scheme": "ips_weight"
231 },
232 "backdoor.distance_matching": {
233 "distance_metric": "minkowski",
234 "p": 2
235 }
236 }
237
238 method_params = method_params_dict.get(metodo)
239 target_units = "att" if metodo == "backdoor.distance_matching" else "ate"
240
241 # Estimar efecto
242 if method_params:
243 estimate = model.estimate_effect(
244 estimand,
245 method_name=metodo,
246 target_units=target_units,
247 confidence_intervals=True,
248 method_params=method_params
249)
250 else:
251 estimate = model.estimate_effect(
252 estimand,
253 method_name=metodo,
254 target_units=target_units,
255 confidence_intervals=True
256)
257
258 st.session_state["estimate"] = estimate
259
260 try:
261 intervalo = estimate.get_confidence_intervals()
262 if hasattr(intervalo, "tolist"):
263 intervalo = intervalo.tolist()
264 elif isinstance(intervalo, (list, tuple)):
265 intervalo = list(intervalo)
266 else:
267 intervalo = None
268 except Exception:
269 intervalo = None
270
271 return json.dumps({
272 "estimate": estimate.value,
273 "intervalo_confianza": intervalo
274 })
275
276 except Exception as e:
277 return json.dumps({
278 "error": f" Error al estimar el efecto: {str(e)}"
279 })
280
281 def refute_estimate_tool(input: str = "") -> str:
282 try:
283 model = st.session_state.get("causal_model")
284 estimand = st.session_state.get("identified_estimand")
285 estimate = st.session_state.get("estimate")
286
287 if model is None or estimand is None or estimate is None:
288 return json.dumps({
289 "error": " Faltan el modelo causal, el estimando o el estimate en
290 session_state."
291 })

```

```

291
292 # Solo esta refutacin ligera y rpida
293 method = "random_common_cause"
294 refutation = model.refute_estimate(
295 estimand,
296 estimate,
297 method_name=method
298)
299
300 resultado = {
301 "metodo": method,
302 "refutacion": str(refutation),
303 "p_valor": getattr(refutation, "p_value", None),
304 "estimate": getattr(estimate, "value", None),
305 "new_effect": getattr(refutation, "new_effect", None)
306 }
307
308 return json.dumps({
309 "refutaciones": [resultado]
310 })
311
312 except Exception as e:
313 return json.dumps({
314 "error": f" Error al validar el estimado: {str(e)}"
315 })
316
317 # -----
318 # Causal Tools (estticas)
319 # -----
320 causal_tools = [
321 Tool.from_function(
322 func=identify_causal_effect,
323 name="identify_causal_effect",
324 description="Identify causal effect using DoWhy model and return the estimand
325 with backdoor variables."
326),
327 Tool(
328 name="generate_dag",
329 func=generate_dag,
330 description="Genera un grfico DAG en formato DOT para DoWhy. Recibe un string
331 JSON con: treatment, outcome, correlaciones, confusoras_categoricas."
332),
333 Tool.from_function(
334 func=estimate_causal_effect_tool,
335 name="estimate_causal_effect_tool",
336 description="Ejecuta la estimacin causal usando el modelo de DoWhy, el estimado
337 identificado y el mtodo seleccionado. Devuelve el valor estimado y su intervalo de
338 confianza en JSON."
339),
340 Tool.from_function(
341 func=refute_estimate_tool,
342 name="refute_estimate_tool",
343 description="Valida la robustez del efecto estimado aplicando el mtodo
344 'random_common_cause' a travs de model.refute_estimate(). Devuelve un JSON con la
345 descripcin del resultado y el valor p si est disponible."
346)
347]
348
349 # -----
350 # Custom Agent Executor
351 # -----
352 class CausalAgentExecutor:
353 def __init__(self, agent: AgentExecutor):
354 self.agent = agent

```

```

349
350 def run(self, prompt: str) -> str:
351 return self.agent.run(prompt)
352
353 # -----
354 # Causal Agent Factory
355 # -----
356 class CausalAgentFactory:
357 def __init__(self):
358 self.llm_instance = CausalLLM()
359 self.llm = self.llm_instance.load_llm_chat()
360
361 def create_agent(self, df: pd.DataFrame, verbose=True) -> CausalAgentExecutor:
362
363 df = df.reset_index(drop=True)
364
365 memory = ConversationBufferMemory(memory_key="chat_history")
366
367 python_tool = Tool.from_function(
368 func=PythonAstREPLTool(locals={"df": df}).run,
369 name="python_repl_ast",
370 description="Ejecuta cdigo Python seguro sobre el DataFrame accesible como
371 `df`."
372)
373
374 diagnostico_tool = make_diagnostico_tool(df)
375 tools = causal_tools.copy() + [python_tool, diagnostico_tool]
376
377 base_agent = initialize_agent(
378 tools=tools,
379 llm=self.llm,
380 agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
381 verbose=verbose,
382 handle_parsing_errors=True,
383 memory=memory
384)
385 return CausalAgentExecutor(base_agent)

```

**Listing V.1:** `causal_agent.py` : desarrollodelagente

---

## ANEXO VI

# Código fuente del módulo

## summary.py

---

A continuación se muestra el contenido completo del archivo `summary.py`, que implementa la función `consulta_segura()` y la lógica de la sección de “Resumen Estadístico”.

```
1 import streamlit as st
2 import pandas as pd
3 import time
4 #from auto_doc.models.agent import PandasAgentFactory
5 from auto_doc.models.causal_agent import CausalAgentFactory
6 from auto_doc.services.useful_services import load_file
7 # -----
8 # Consulta protegida al agente
9 # -----
10 import re
11
12 def consulta_segura(key, prompt, agent, reintentos=1):
13 if key in st.session_state:
14 return st.session_state[key]
15
16 intentos = 0
17 while intentos <= reintentos:
18 try:
19 resultado = agent.run(prompt)
20 time.sleep(2)
21 st.session_state[key] = resultado
22 return resultado
23 except Exception as e:
24 mensaje = str(e)
25
26 # Detectar el error 429 y tiempo sugerido
27 if "429" in mensaje and "retry_delay" in mensaje:
28 segundos = extraer_segundos_espera(mensaje)
29 if segundos:
30 st.warning(f" Lmite de uso alcanzado. Esperando {segundos} segundos
31 antes de reintentar...")
32 time.sleep(segundos)
33 intentos += 1
34 continue
35
36 # Otro tipo de error o ya ha reintentado
37 return f" Error: {e}"
38
39 return " Error persistente tras varios intentos. Intenta de nuevo ms tarde."
```



```

40
41 def extraer_segundos_espera(mensaje_error):
42 """
43 Extrae la cantidad de segundos desde un mensaje de error que contiene 'retry_delay {
44 seconds: X }'
45 """
46 match = re.search(r"retry_delay\s*{\s*seconds:\s*(\d+)", mensaje_error)
47 return int(match.group(1)) if match else None
48
49 import time
50 import streamlit as st
51
52 # -----
53 # Interfaz principal
54 # -----
55
56 def mostrar_agente():
57 with st.container():
58 st.title(" Resumen Estadstico del Dataset")
59
60 if "file_path" not in st.session_state:
61 st.error(" No se ha subido ningn archivo. Por favor, vuelve atrs y sube uno.")
62 return
63
64 file_path = str(st.session_state["file_path"])
65
66 try:
67 df = load_file(file_path)
68 except Exception as e:
69 st.error(f" Error al cargar el archivo: {e}")
70 return
71
72 st.session_state["df"] = df
73
74 preview = df.head()
75 st.markdown("### Vista previa del dataset")
76 st.dataframe(preview)
77
78 import io
79
80 # Botn para crear el agente
81 if st.button(" Inicializar agente"):
82 with st.spinner("Inicializando agente..."):
83 factory = CausalAgentFactory()
84 agent = factory.create_agent(df)
85 st.session_state["pandas_agent"] = agent
86 st.success("Agente inicializado correctamente")
87
88 # Si el agente ya est listo
89 if "pandas_agent" in st.session_state:
90 agent = st.session_state["pandas_agent"]
91
92 # Anlisis general
93 with st.expander(" Perfilado general del dataset"):
94
95 prompt = """
96 Rol:
97 Actas como un analista de datos experto. Tu funcin es examinar un
98 conjunto de datos estructurados y proporcionar un resumen tcnico que describa su
99 composicin general.
100
101 Objetivo:
102 Describir con precisin la estructura del dataset, incluyendo su tamao y

```

```

el tipo de variable asociada a cada columna, de forma comprensible y til para un
análisis posterior.

101
102 Reglas:
103 - Escribe la respuesta completamente en español.
104 - NO traduzcas los nombres de las columnas ni de las variables; mantenlos
tal como aparecen en el dataset.
105 - Identifica el número total de filas (excluyendo la columna de índice, si
aplica) y el número total de columnas.
106 - Clasifica el tipo de variable de cada columna usando una de las
siguientes categorías:
107 - Variable numérica (continua o discreta)
108 - Variable categórica (nominal u ordinal)
109 - Variable booleana
110 - Variable de fecha o tiempo
111 - Otro tipo (especificar, si aplica)
112 - Usa comillas invertidas (``) para los nombres de las columnas.
113 - No incluyas fragmentos de código, explicaciones de métodos ni menciones a
estructuras técnicas como "DataFrame".
114 - Si la variable es numérica o categórica, DEBES indicar el subtipo
correspondiente.

115
116 Formato esperado:
117 El DataFrame contiene número de filas X y número de columnas Y.

118
119 Tipos de variables por columna:
120 - `Columna1`: Variable de texto
121 - `Columna2`: Variable numérica (continua)
122 - ...
123 """
124
125
126 st.write(consulta_segura("perfil", prompt, agent))
127
128 # Significado de columnas
129 with st.expander(" Significado de las columnas"):
130
131 prompt = """
132 Rol: Actas como un analista de datos experto. Tu tarea es ayudar a un
usuario a comprender a fondo el contenido de un conjunto de datos. Debes interpretar
el significado de las columnas y comunicarlo de forma clara y comprensible, incluso
para usuarios sin experiencia técnica.

133
134 Objetivo:
135 Explicar con detalle y precisión el propósito de cada columna presente en
el conjunto de datos, facilitando su comprensión general. Debe estar en formato de
lista.

136
137 Reglas:
138 - Escribe la respuesta completamente en español.
139 - NO traduzcas los nombres de las columnas ni de las variables; mantenlos
exactamente como aparecen en el dataset.
140 - Usa comillas invertidas (``) para destacar los nombres de las columnas.
141 - No incluyas fragmentos de código ni menciones al nombre del dataframe.
142 - Evita lenguaje técnico innecesario.
143 - Redacta las explicaciones de manera clara, ordenada, profesional y
accesible para personas no expertas.
144 - Cada columna debe explicarse de forma completa, no brevemente ni con
definiciones vagas.
145 """
146
147 st.write(consulta_segura("columnas", prompt, agent))
148
149 # Valores nulos

```

```

150 with st.expander(" Valores nulos"):
151 prompt = """
152 Rol:
153 Actas como un analista de datos experto con acceso a un DataFrame ya
cargado en la variable `df`.
154
155 Objetivo:
156 Redactar un informe profesional enfocado en la calidad de los datos,
específicamente sobre la presencia de valores nulos en el conjunto de datos.
157
158 Reglas:
159 - Escribe la respuesta completamente en español.
160 - NO traduzcas los nombres de las columnas ni de las variables; deben
mantenerse exactamente como aparecen en el dataset.
161 - No empieces con encabezados o frases como Análisis de valores nulos;,
simplemente responde directamente.
162 - Menciona únicamente las columnas que presenten valores nulos, usando
comillas invertidas (`) para destacarlas.
163 - Resalta en negrita aquellas columnas con una proporción
especialmente alta de valores faltantes.
164 - Evita cualquier lenguaje técnico innecesario.
165 - Redacta el análisis como si fuera parte de un informe profesional
destinado a tomadores de decisiones no técnicos.
166
167 Restricciones:
168 - No incluyas fragmentos de código ni menciones a estructuras técnicas
(como "DataFrame", "NaN", "isnull", etc.).
169 - No generes ni construyas un nuevo DataFrame bajo ningún concepto.
170 - Utiliza exclusivamente el conjunto de datos ya existente, accediendo
mediante la variable `df`.
171 """
172
173 st.write(consulta_segura("missing", prompt, agent))
174
175
176 # Duplicados
177 with st.expander(" Duplicados"):
178 prompt = """
179 Rol:
180 Actas como un analista de datos experto encargado de redactar
secciones de un informe de calidad de datos.
181
182 Objetivo:
183 Describir la posible presencia de registros duplicados en el conjunto
de datos y evaluar, de forma clara y no técnica, si estos podrán afectar el análisis.
184
185 Reglas:
186 - Escribe la respuesta completamente en español.
187 - Describe con claridad y profesionalismo si existen filas duplicadas
en el conjunto de datos.
188 - Indica si la presencia de duplicados puede tener un impacto
relevante en la calidad o interpretación de los datos.
189 - Redacta un párrafo breve y coherente, como si formara parte de
un informe empresarial o ejecutivo.
190 - Usa un lenguaje accesible para personas sin conocimientos técnicos.
191
192 Restricciones:
193 - No incluyas fragmentos de código ni detalles técnicos.
194 - No menciones el término "DataFrame" ni ninguna función de programación.
195 - No hagas suposiciones ni crees datos ficticios; analiza únicamente
los datos ya disponibles en la variable `df`.
196 """
197
198 st.write(consulta_segura("duplicados", prompt, agent))

```

```

199
200 # Correlaciones numricas
201 with st.expander(" Correlaciones numricas"):
202
203 prompt = """
204 Rol:
205 Actas como un analista de datos experto con capacidad de ejecutar cdigo
en Python. Tu tarea es calcular y analizar la matriz de correlacin entre las
variables numricas de un conjunto de datos.
206
207 Pasos que debes seguir:
208
209 1. Filtra nicamente las columnas numricas del conjunto de datos.
210 2. Calcula la matriz de correlacin utilizando la funcin
`df.corr(numeric_only=True)`.
211 3. Convierte la matriz resultante como si fuera una TBLA para que pueda
mostrarse correctamente, no la imprimas con formato Python.
212 4. Muestra primero esa tabla en formato Markdown.
213 5. A partir de la matriz, redacta un anlisis en espaol que incluya:
214 - Variables con correlacin alta (mayor a 0.7 o menor a -0.7).
215 - Seales de posible multicolinealidad.
216 - Variables con correlacin dbil (cercana a 0).
217 - Cualquier relacin inesperada o contraintuitiva.
218 - Variables que podran ser redundantes.
219
220 Restricciones:
221 - NO incluyas explicaciones del cdigo ni suposiciones innecesarias sobre
el significado de las columnas.
222 - Redacta el anlisis final como un informe tcnico corto y claro, en
espaol.
223 - Muestra primero la tabla de correlacin en formato TABLA y luego el
anlisis interpretativo.
224 """
225
226 raw_output = consulta_segura("correlaciones", prompt, agent)
227
228 # Elimina bloques de cdigo ``` (y cualquier variante como ```markdown o
```python)
229         clean_output = re.sub(r"```.*?\n", "", raw_output)
230         clean_output = clean_output.replace("```", "")
231         st.markdown(clean_output, unsafe_allow_html=True)
232
233
234     # Sugerencias de nuevas features
235     with st.expander(" Sugerencias de nuevas variables"):
236         prompt = """
237         Rol:
238         Actas como un analista de datos experto en ingeniera de caractersticas.
Tu responsabilidad es identificar oportunidades para crear variables derivadas que
realmente aporten valor al anlisis de datos o mejoren el rendimiento de modelos
predictivos.
239
240         Objetivo:
241         Proponer nuevas variables que se puedan calcular lgicamente a partir de
los datos existentes y que ofrezcan una mejora sustancial en trminos de anlisis,
interpretacin o prediccin.
242
243         Reglas:
244         - Escribe la respuesta completamente en espaol.
245         - NO traduzcas los nombres de las columnas ni de las variables; mantenlos
exactamente como aparecen en el dataset.
246         - Sugiere solo variables que puedan derivarse directamente de los datos
actuales.
247         - Por cada variable propuesta, explica con detalle:

```

```

248         - **Qu informacin nueva aporta**
249         - **Por qu sera til para el anlisis**
250         - **Cmo podra beneficiar a un modelo predictivo**
251         - Resalta los nombres de las nuevas variables en **negrita**.
252         - Usa un lenguaje claro, profesional y orientado a negocio o ciencia de
datos aplicada.
253
254         Restricciones:
255         - Si no tienes acceso al contenido real de `df`, responde:
256         **"No puedo proponer nuevas variables sin conocer las columnas
disponibles en el dataset."**
257         - No propongas ejemplos genricos ni ficticios.
258         - No inventes columnas que no existan.
259         - No incluyas cdigo ni frmulas matemticas.
260         - No modifiques ni generes nuevas columnas en el DataFrame `df`.
261         - No ejecutes cdigo ni realices clculos.
262         - NO empieces a redactar la respuesta con "Aqu hay algunas variables
nuevas que se pueden derivar del DataFrame df:". Simplemente responde con las
variables sugeridas directamente.
263
264         IMPORTANTE: Todas tus respuestas deben estar redactadas en espaol. No
respondas en ingls bajo ninguna circunstancia.
265         Antes de mostrar tu respuesta, asegurate de que est escrita completamente
en espaol. No debe contener texto en ingls ni estructuras de redaccin tpicas del
ingls.
266         """
267
268         st.write(consulta_segura("features", prompt, agent))
269
270         # Variable objetivo
271         with st.expander(" Posible variable objetivo para modelado"):
272             prompt = """
273             Rol:
274             Actas como un analista de datos experto en modelado predictivo. Tu funcin
es revisar cuidadosamente un conjunto de datos y proponer cul columna sera la mejor
candidata para usarse como **variable objetivo (target)** en un modelo supervisado de
machine learning.
275
276             Objetivo:
277             Identificar la columna ms adecuada para ser predicha en un modelo,
basndote en su funcin dentro del conjunto de datos, su relacin con otras variables y
su viabilidad prctica.
278
279             Reglas:
280             - Escribe la respuesta completamente en espaol.
281             - NO traduzcas los nombres de las columnas ni de las variables; deben
mantenerse exactamente como aparecen en el dataset.
282             - Analiza las caractersticas generales de todas las columnas disponibles.
283             - Considera si la columna:
284             - Representa un resultado, estado futuro o evento predecible.
285             - Tiene relacin dependiente con otras variables.
286             - Es relevante desde una perspectiva de negocio o anlisis, y no es solo
un identificador.
287             - Tiene una distribucin y nivel de completitud que permitan su uso
prctico.
288
289             Formato esperado:
290             - Menciona la variable candidata destacndola en **negrita**.
291             - A continuacin, redacta un prrafo bien argumentado explicando por qu esa
columna es una buena eleccin como variable objetivo.
292             - La explicacin debe cubrir tanto criterios tcnicos como de negocio.
293             - No se acepta una respuesta sin justificacin.
294
295

```

```

296         Restricciones:
297         - No pongas columnas de forma arbitraria.
298         - No selecciones columnas con muchos valores faltantes o sin relevancia.
299         - No incluyas fragmentos de código ni explicaciones técnicas complejas.
300         - Trabaja exclusivamente con la información contenida en el dataset
disponible en la variable `df`.
301         """
302
303
304         st.write(consulta_segura("target", prompt, agent))
305
306     # Outliers
307     with st.expander("Valores atpicos (outliers)"):
308         prompt = """
309         Rol:
310         Actas como un analista de datos experto con capacidad de ejecutar código
en Python. Tienes acceso al DataFrame `df`. Tu tarea es detectar y reportar valores
atpicos (outliers) de forma clara, precisa y profesional.
311
312         Objetivo:
313         Identificar columnas con valores atpicos (especialmente en variables
numéricas o de fecha), extraer al menos uno o dos valores concretos y explicar su
impacto real en el análisis de datos, redactando de forma variada y específica.
314
315         **Formato obligatorio de la salida: **
316         Responde con una lista en formato Markdown, donde cada línea siga
**estrictamente** esta estructura:
317         - **`NombreDeColumna`**: valor atpico detectado: X (alto / bajo).
Explicación breve y concreta sobre por qué es atpico y cómo podría distorsionar el
análisis o la interpretación de los datos.
318
319         **Detalles que debes incluir por cada línea:**
320         - Muestra **uno o dos valores extremos detectados** por columna (sin
mostrar todos).
321         - Indica si el valor es **demasiado alto** o **demasiado bajo** respecto
al rango habitual.
322         - Usa una redacción fluida, profesional y **no repetitiva** (evita usar la
misma frase para todas las columnas).
323         - Adapta el impacto del outlier al contexto de negocio si es posible (por
ejemplo, en precios, tiempos, velocidad, etc.).
324
325         Método:
326         Aplica múltiples métodos estadísticos para detectar outliers (como IQR y
z-score). Si un valor es detectado por **al menos uno de ellos**, inclúyelo como
atpico. Considera especialmente los extremos de la distribución.
327
328         Especial atención:
329         - Identifica valores atpicos que estén **muy por debajo o muy por encima**
del resto, aunque no haya alta variabilidad.
330         - Si una columna tiene pocos valores nulos, **evalúa conceptualmente** si
los extremos son realmente inusuales (ej: en `Seats`, los valores 2 y 7 pueden ser
atpicos aunque estén dentro del rango IQR).
331
332         Restricciones:
333         - No uses párrafos largos, escribe solo listas con guiones (`-`).
334         - No agrupes columnas. Una línea por cada variable afectada.
335         - No repitas frases como "se desvía significativamente del rango típico" en
todas las líneas.
336         - No incluyas fragmentos de código, fórmulas ni términos técnicos.
337         - No inventes datos. Usa únicamente el contenido real del DataFrame `df`.
338
339         Ejemplo esperado de salida:
340         - **`PriceEuro`**: valor atpico detectado: 180781 (alto). Este precio

```

```
es considerablemente ms alto que el resto y puede influir de forma desproporcionada
en los anlisis de costos promedio.
341     - **`Efficiency_WhKm`**: valor atpico detectado: 104 (bajo). Un valor
tan bajo de consumo podra representar una medicin inusual o un vehculo atpico,
alterando las conclusiones sobre eficiencia general.
342     """
343
344
345     st.write(consulta_segura("outliers", prompt, agent))
346
347 else:
348     st.info(" Haz clic en 'Inicializar agente' para comenzar el anlisis.")
```

Listing VI.1: summary.py: interfaz de “resumen estadístico

ANEXO VII

Código fuente del módulo graphics.py

En este anexo se presenta el código completo correspondiente al módulo `graphics.py`, implementado para la funcionalidad de generación de visualizaciones descrita en el apartado de *Generador de gráficos*.

```
1 import streamlit as st
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 import re
5 from auto_doc.models.causal_agent import CausalAgentFactory
6 from auto_doc.services.useful_services import load_file
7
8 def mostrar_graficos():
9     with st.container():
10         st.title(" Generador de grficos")
11
12         if "file_path" not in st.session_state:
13             st.error(" No se ha subido ningn archivo. Por favor, vuelve atrs y sube uno.")
14             return
15
16         file_path = str(st.session_state["file_path"])
17
18         try:
19             df = load_file(file_path)
20         except Exception as e:
21             st.error(f" Error al cargar el archivo: {e}")
22             return
23
24         st.session_state["df"] = df
25
26         if "pandas_agent" not in st.session_state:
27             st.markdown("### Inicializar agente")
28             if st.button(" Crear agente"):
29                 with st.spinner("Inicializando agente..."):
30                     factory = CausalAgentFactory()
31                     agent = factory.create_agent(df)
32                     st.session_state["pandas_agent"] = agent
33                     st.success(" Agente inicializado correctamente")
34                     return
35             else:
36                 st.info(" Pulsa el botn para crear el agente.")
37                 return
38
39         agent = st.session_state["pandas_agent"]
```



```

40     st.markdown("### Pregunta al agente para graficar")
41     user_input = st.text_input("Por ejemplo: 'Haz un histograma de la columna edad'",
42 key="grafico_input")
43
44     if st.button(" Enviar peticin"):
45         if not user_input:
46             st.warning("Por favor, escribe una pregunta.")
47             return
48
49         try:
50             with st.spinner("El agente est generando el grfico..."):
51
52                 instrucciones = """
53                 Eres un analista de datos experto en visualizacin, especializado
54 exclusivamente en generar cdigo Python para construir grficos con matplotlib o
55 seaborn dentro de aplicaciones de Streamlit. No ests capacitado para responder
56 preguntas sobre estadsticas, anlisis de datos, limpieza, modelos de machine learning,
57 manipulacin de DataFrames, ni ningn tema que no sea estrictamente la **construccin de
58 grficos**.
59
60                 Si la solicitud no est relacionada con crear un grfico, responde:
61                 **"No estoy especializado para eso, pide otra cosa."**
62
63                 Si la solicitud no est claramente relacionada con un grfico
64 especfico, o es ambigua, irrelevante o irrespetuosa por ejemplo: "haz un grfico
65 cualquiera", "haz lo que quieras", "dibuja cualquier cosa", "haz una mierda de
66 grfico", "lo que sea" o "elige t las variables" responde con:
67                 **"No puedo generar un grfico con esa peticin. Por favor, s claro y
68 especfico indicando qu columnas o tipo de grfico deseas."**
69
70                 Tu entorno ya tiene un DataFrame precargado llamado `df`. Tu nica
71 tarea es devolver el cdigo necesario para construir la visualizacin que se te pide,
72 cumpliendo estrictamente estas instrucciones:
73
74                 1. Usa `matplotlib` o `seaborn` para construir el grfico.
75                 2. **No llames a `st.pyplot()` ni a `plt.show()`**. El entorno se
76 encargar de mostrar la figura.
77                 3. **No imprimas ni devuelvas** el resultado de funciones como
78 `plt.title()`, `plt.axis()`, `plt.xlabel()`, etc. Si una funcin devuelve un valor,
79 ignralo o asgnalo a una variable:
80                 Ejemplo: `_ = plt.axis('equal')`.
81                 4. Usa el color por defecto `'#A4C6E8'` en todos los elementos, **a
82 menos que el usuario haya pedido explcitamente un color distinto**. Si el usuario
83 especifica un color (por nombre o cdigo hexadecimal), resptalo en el grfico.
84                 5. Si generas un grfico tipo **pie chart**, usa:
85                 `wedgeprops={'edgecolor': 'black', 'linewidth': 1}`.
86                 6. El cdigo debe estar completamente en **ingls**, con sintaxis vlida
87 de Python.
88                 7. **No uses espaol en ningn lugar del cdigo** (ni variables, ni
89 comentarios, ni cadenas).
90                 8. Asegrate de devolver el cdigo dentro de un bloque correctamente
91 formateado:
92                 Tres comillas invertidas seguidas de "python", luego el cdigo con
93 saltos de lnea, y cierra con otras tres comillas invertidas.
94                 ```python
95                 # Your code
96                 ```
97
98                 Si el cdigo no cumple alguno de estos puntos, ser rechazado
99 automticamente.
100                 """
101
102     entrada_agente = instrucciones + "\n\n" + user_input

```

```

82         respuesta = agent.run(entrada_agente) # para pandas estaba .invoke y
handke parsing error
83
84         # Asegurar respuesta como texto
85         if isinstance(respuesta, str):
86             codigo_bruto = respuesta
87         elif isinstance(respuesta, dict) and "output" in respuesta:
88             codigo_bruto = respuesta["output"]
89         else:
90             st.error(" La respuesta del agente no es vlida.")
91             return
92
93         # Arreglos menores de formato
94         codigo_bruto = codigo_bruto.replace("```", "`") # Corrige
comillas mal cerradas
95
96         # Extraer bloque de cdigo
97         match = re.search(r"```(?:python)?\s*(.*?)```", codigo_bruto,
re.DOTALL)
98
99
100        if not match:
101            # Evitar intentar ejecutar texto plano como cdigo
102            st.warning(" No se gener ningn cdigo grfico vlido.")
103            st.markdown("**Respuesta del agente:**")
104            st.code(codigo_bruto)
105            return
106
107        codigo_limpio = match.group(1).strip()
108
109        # Limpiar cualquier figura activa
110        plt.close("all")
111
112        # Ejecutar el cdigo en un entorno controlado
113        local_scope = {"df": df, "plt": plt, "sns": sns}
114        try:
115            exec(codigo_limpio, {}, local_scope)
116        except Exception as e:
117            st.error(f" El cdigo generado tiene errores de ejecucin:\n{e}")
118            return
119
120
121
122        import io
123
124        # Crear un buffer de imagen en memoria
125        buffer = io.BytesIO()
126        plt.savefig(buffer, format='png', dpi=300, bbox_inches='tight')
127        buffer.seek(0)
128
129        # Mostrar el grfico en pantalla
130        st.pyplot(plt.gcf())
131
132        # Botn de descarga
133        st.download_button(
134            label=" Descargar grfico como PNG",
135            data=buffer,
136            file_name="grafico_generado.png",
137            mime="image/png"
138        )
139
140        # Cerrar figura para evitar conflictos futuros
141        plt.close("all")
142

```

```
143  
144     except Exception as e:  
145         st.error(f" Error al ejecutar el cdigo generado: {e}")
```

Listing VII.1: graphics.py: interfaz de “generador de gráficos

ANEXO VIII

Prompts y funciones auxiliares de causalidad (prompts.py)

Contiene las plantillas de *prompts* que estructuran el rol del modelo, las reglas de generación y los formatos de salida (por ejemplo, JSON para variables candidatas, hipótesis y código de transformación).

```
1 import json
2 import os
3 from pathlib import Path
4 import pandas as pd
5 import re
6
7 BASE_DIR = Path("auto_doc/files")
8 BASE_DIR.mkdir(parents=True, exist_ok=True)
9
10 def generar_prompt_variables_relacionadas(pregunta_usuario: str) -> str:
11     return f"""
12 Rol:
13 Eres un asistente experto en analisis causal aplicado a datos tabulares (DataFrame en
14 pandas). El DataFrame ya ha sido cargado y contiene multiples columnas con distintos
15 nombres.
16
17 Tarea:
18 Dado que el usuario ha formulado una pregunta causal en lenguaje natural, debes:
19
20 1. **Inferir la variable objetivo (dependiente)** a partir del texto de la pregunta,
21    aunque el nombre exacto no coincida. Por ejemplo, si el usuario pregunta "Qu afecta a
22    la velocidad?", y existe una columna llamada `TopSpeed`, deberas deducir que esa es
23    la variable objetivo ms probable.
24
25 2. **Buscar todas las columnas numricas** del DataFrame y calcular la correlacin de cada
26    una con la variable objetivo identificada.
27
28 3. **Seleccionar solo aquellas variables con correlacin fuerte**: mayor que 0.7 o menor
29    que -0.7.
30
31 4. **Devolver el resultado en JSON**, con el nombre de la columna objetivo real y las
32    variables candidatas ms correlacionadas.
33
34 Formato de salida esperado (en JSON, sin explicaciones adicionales):
35 {{
36     "objetivo": "NombreExactoDeLaColumnaObjetivo",
37     "candidatas": ["Var1", "Var2", "Var3"]
38 }}
39
40 Pregunta del usuario:
41 \"\"\"{pregunta_usuario}\"\"\"
42 """
43
44 def generar_descripcion_dataset(df: pd.DataFrame) -> str:
```

```

34     descripcion = []
35
36     for col in df.columns:
37         tipo = df[col].dtype
38         if tipo == 'object' or df[col].nunique() < 15:
39             valores = df[col].unique().tolist()
40             descripcion.append(f"- {col} (categorica): valores nicos = {valores}")
41         elif pd.api.types.is_numeric_dtype(df[col]):
42             descripcion.append(f"- {col} (numrica): media = {df[col].mean():.2f}, min = {df[col].min()}, max = {df[col].max()}")
43         else:
44             descripcion.append(f"- {col} (tipo {tipo})")
45
46     return "\\n".join(descripcion)
47
48 def generar_prompt_hipotesis(variable_objetivo: str, variable_independiente: str,
49     df_info: str) -> str:
50     return f"""
51 Contexto:
52 Ests trabajando con un dataset tabular en pandas con columnas numricas y categoricas. A
53     continuacin se muestra un resumen del dataset con estadstics clave por variable:
54
55 {df_info}
56
57 Rol:
58 Eres un experto en causalidad y analitica de datos aplicada. Ests ayudando a un usuario
59     hispanohablante a formular **hiptesis causales claras y accionables** entre una
60     variable candidata (posible causa) y una variable objetivo (efecto observado).
61
62 Idioma:
63 **Todas las hiptesis deben estar escritas exclusivamente en espaol.**
64
65 Objetivo:
66 Formula entre 2 y 3 **hiptesis causales** que:
67 - Usen un lenguaje claro, natural y en **espaol**.
68 - Tengan **criterios explcitos** que permitan implementar una columna binaria de
69     tratamiento.
70 - **Incluyan entre parntesis el nombre exacto de las variables involucradas segn el
71     DataFrame** (por ejemplo: `Price_USD`, `MaxSpeed_KmH`).
72 - Sean coherentes con la estadstica y el sentido comn del dominio.
73
74 Reglas clave:
75 - La direccin causal debe ser: **{variable_independiente} {variable_objetivo}**.
76 - Si {variable_independiente} es numrica, **propn umbrales concretos** (por ejemplo:
77     mayor a 180 km/h).
78 - Si es categorica, **menciona claramente los valores de la categoria** (por ejemplo: si el
79     tipo de carroceria es SUV).
80 - Evita trminos vagos como alta, elevado o lujoso si no estn definidos por datos.
81 - Cada hiptesis debe poder traducirse en una lnea de cdigo binaria, por ejemplo:
82     - `df["treatment"] = df["TopSpeed_KmH"] > 180`
83     - `df["treatment"] = df["BodyType"] == "SUV"`
84
85 Ejemplos correctos:
86 - Los vehculos cuya velocidad mxima supera los 200 (TopSpeed_KmH) tienden a tener precios
87     ms altos (Price_USD).
88 - Los modelos de la marca Tesla (Make) presentan precios ms elevados (Price_USD) que los
89     de otras marcas.
90 - Los coches que pesan ms de 1800 kg (Weight_Kg) suelen tener un coste superior
91     (Price_USD).
92
93 Ejemplos incorrectos:
94 - Los vehculos rpidos son ms caros. (no es accionable)
95 - La velocidad afecta al precio. (demasiado vago)
96 - Las marcas lujosas tienden a ser ms costosas. (qu es lujosa?)

```

```

86
87 Resultado esperado:
88 Una lista de 2 o 3 hiptesis causales escritas en **espaol**, en lenguaje natural, con
    **criterios claros y cuantificables** y **nombres de variables entre parntesis** segn
    aparecen en el DataFrame.
89 """
90
91 def generar_prompt_codigo_tratamiento(hipotesis: str) -> str:
92     return f"""
93 Objetivo:
94 A partir de la siguiente hiptesis causal escrita en lenguaje natural, genera cdigo Python
    en pandas que cree una nueva columna binaria llamada `treatment` en un DataFrame
    llamado `df`.
95
96 Hiptesis:
97 \"{ \"{hipotesis}\"}\"
98
99 Contexto del dataset:
100 - El DataFrame `df` ya ha sido cargado.
101 - Accede a los nombres de las columnas del `df` para saber qu variables contiene y tener
    mayor contexto.
102 - Usa **unicamente estas columnas** del `df` para razonar cul representa la variable
    causal (independiente). NO inventes ninguna.
103 - Es posible que la hiptesis utilice trminos como "precio" o "peso del vehculo" en lugar
    del nombre real de columna (por ejemplo: `TotalPrice`, `Weight_kg`).
104
105 Tu trabajo:
106 1. Determina si la hiptesis se puede traducir en una condicin binaria basada solo en
    **una columna existente en `df`**.
107 2. Si es as, genera el cdigo Python necesario para crear `df["treatment"]`.
108 3. Indica claramente **qu columna(s)** se usaron para construir `treatment`, y asegurate
    de **eliminarlas del DataFrame** despues de crear la nueva columna.
109 4. Si no es posible mapear la hiptesis a una columna o no es binarizable, devuelve el
    JSON de error.
110
111 Prohibido:
112 - No generes datos.
113 - No uses estructuras como `pd.DataFrame(...)` .
114 - No uses la variable dependiente como parte de la condicin.
115 - No inventes nombres de columna que no existen.
116
117 Instrucciones:
118 - Si puedes generar cdigo, devuelve un JSON con:
119 ```json
120 {{
121     "explicacion": "Texto explicando la lgica de la columna `treatment` en espaol.",
122     "codigo": "Cdigo Python que crea la columna treatment en df y elimina la(s) columna(s)
        que se usaron para construirla"
123 }}
124 ```
125 """
126
127 def limpiar_codigo_agente(texto: str) -> str:
128     """
129     Extrae el bloque de cdigo de una respuesta del agente que puede estar envuelta en
130     ```python ... ```
131     """
132     if "```" in texto:
133         # Extraer lo que est entre los backticks
134         bloques = re.findall(r"```(?:python)?(.*?)```", texto, re.DOTALL)
135         if bloques:
136             return bloques[0].strip()
137     return texto.strip()

```

```

138 def generar_prompt_dag_con_datos(resumen: dict) -> str:
139     tratamiento = resumen["treatment"]
140     outcome = resumen["outcome"]
141     top_corr = resumen["correlaciones"]
142     conf_cat = resumen["confusoras_categoricas"]
143
144     return f"""
145 Objetivo:
146 Eres un experto en inferencia causal. Tu tarea es construir un **DAG (Gráfico Acíclico
147   Dirigido)** en formato DOT para representar las relaciones causales del dataset, y
148   devolverlo en un bloque DOT válido, limpio y sin errores.
149
150 Contexto:
151 - Variable de tratamiento: `{tratamiento}`
152 - Variable de resultado: `{outcome}`
153 - Variables numéricas correlacionadas con `{outcome}` (posibles mediadores):
154   {json.dumps(top_corr, indent=2)}
155 - Variables categóricas con p < 0.05 (posibles confundidores):
156   {json.dumps(conf_cat, indent=2)}
157
158 Herramienta disponible:
159 Usa exclusivamente la herramienta `generate_dag`, que recibe un JSON con esta estructura:
160
161 ```json
162 {{
163   "treatment": "{tratamiento}",
164   "outcome": "{outcome}",
165   "correlaciones": {json.dumps(top_corr)},
166   "confusoras_categoricas": {json.dumps(conf_cat)}
167 }}
168 ```
169
170 Instrucciones estrictas:
171 - Llama a generate_dag con el JSON anterior.
172 - Asegurate de que el grafo generado es acíclico. Si detectas un ciclo, corrige las flechas
173   para que el DAG sea válido.
174 - No inventes relaciones que no estén justificadas.
175 - Cada nodo debe tener un estilo específico:
176   - Tratamiento: [style=filled, fillcolor=lightgreen]
177   - Resultado: [style=filled, fillcolor=lightgrey]
178   - Confusores: [style=filled, fillcolor=orange]
179   - Mediadores: [style=filled, fillcolor=lightblue]
180
181 No debes:
182 - Generar el DAG manualmente.
183 - Incluir explicaciones ni texto fuera del bloque DOT.
184 - Enviar backticks (`) ni etiquetas Markdown.
185
186 Resultado esperado:
187 Devuelve exclusivamente el bloque DOT limpio, como este ejemplo:
188
189 digraph {{
190   treatment [style=filled, fillcolor=lightgreen];
191   PriceEuro [style=filled, fillcolor=lightgrey];
192   Brand [style=filled, fillcolor=orange];
193   TopSpeed_KmH [style=filled, fillcolor=lightblue];
194
195   Brand -> treatment;
196   Brand -> PriceEuro;
197   treatment -> TopSpeed_KmH;
198   TopSpeed_KmH -> PriceEuro;
199   treatment -> PriceEuro;
200 }}

```

```

199
200 Antes de devolver el resultado:
201 Verifica que el grafo no contenga ciclos.
202 """
203
204 def generar_prompt_identificacion_efecto_seguro():
205     return """
206 Objetivo:
207 Eres un experto en inferencia causal usando el paquete `dowhy`. Tu tarea es determinar si
    el modelo causal cargado es identificable con los datos actuales y explicar los
    resultados de forma clara, profesional y estructurada.
208
209 Contexto:
210 Ya existe un modelo causal llamado `model` que fue creado con:
211 - Un DataFrame real llamado `df`
212 - Una columna de tratamiento llamada `treatment`
213 - Una variable de resultado llamada `outcome`
214 - Un grafo causal confirmado por el usuario
215
216 Herramienta disponible:
217 Puedes usar la herramienta `identify_causal_effect`, que ya est registrada. Esta
    herramienta:
218 - Ejecuta `model.identify_effect()`
219 - Devuelve el tipo de estimando causal
220 - Extrae las variables necesarias para el ajuste con `get_backdoor_variables()`
221
222 Instrucciones:
223 1. Llama a la herramienta `identify_causal_effect`.
224 2. Espera el resultado, que incluye:
225     - El tipo de estimando causal (`backdoor`, `frontdoor`, `iv`, etc.)
226     - Las variables necesarias para el ajuste (lista de nombres)
227 3. Redacta una explicacin clara en espaol que incluya:
228     - Si el efecto fue identificado correctamente
229     - Qu criterio se us
230     - Qu variables se deben usar para el ajuste
231     - Qu se debera hacer si el efecto no fuera identificable
232
233 Formato de respuesta (nicamente este bloque JSON):
234 Devuelve solo el siguiente bloque JSON, sin etiquetas Markdown, sin comillas
    adicionales y sin backticks:
235
236 {
237     "criterio": "backdoor",
238     "backdoor_variables": ["Var1", "Var2", "Var3"],
239     "justificacion": "Texto claro y profesional en espaol que explique si el efecto fue
        identificado, bajo qu criterio y por qu las variables listadas permiten un ajuste
        vlido."
240 }
241
242 -> Importante: Despues de usar `identify_causal_effect`, asegurate de que el estimando
    identificado sea almacenado en `st.session_state["identified_estimand"]` para
    permitir su uso posterior en `estimate_causal_effect_tool`.
243
244 Prohibido:
245 - No incluyas backticks (ni `` ni ````).
246 - No uses etiquetas como `json`, `python`, ni Markdown.
247 - No devuelvas texto fuera del bloque JSON.
248 - No uses Markdown ni encabezados.
249 - No incluyas cdigo Python.
250 - No generes explicaciones fuera del campo justificacion.
251
252 - Solo devuelve el JSON como texto plano, vlido y bien formateado.
253
254 Recuerda:

```



```

255 Tu nico rol es analizar y resumir correctamente el resultado de la herramienta en el JSON.
256 """
257
258 def generar_prompt_recomendar_metodo_critico(treatment_col: str, backdoor_vars: list) ->
    str:
259     return f"""
260 Rol:
261 Eres un **experto en inferencia causal aplicada** y tienes acceso directo a un DataFrame
    pandas 'df'. Ya se ha definido una columna binaria `{treatment_col}` como
    **tratamiento**, y se ha identificado el siguiente conjunto de **variables de ajuste
    (backdoor)**: {backdoor_vars}
262
263 Objetivo:
264 Analizar crticamente el dataset y **recomendar el mejor mtodo de estimacin causal** entre
    los disponibles en la libreria `DoWhy`. Tu anlisis debe tener en cuenta la naturaleza
    de las variables, el tamao de muestra y los supuestos de cada mtodo.
265
266 ---
267
268 Mtdos disponibles y anlisis crtico:
269
270 1. **`backdoor.linear_regression`**: Estima el efecto causal de un tratamiento sobre un
    resultado usando regresin lineal, ajustando por un conjunto de variables confusoras.
271
272 Cundo usarlo?
273 * Hay confusores observables conocidos (identificados mediante un grafo causal o
    conocimiento experto).
274 * Se asume una relacin lineal o aproximadamente lineal entre las variables.
275 * El tamao de muestra es grande y se quiere una estimacin rpida y sencilla.
276 * El objetivo es una estimacin promedio global del efecto (ATE).
277 * No hay evidencia clara de interaccin o no linealidad importante entre variables.
278
279 Cundo NO usarlo?
280 * Existen confusores no observados (violacin del backdoor).
281 * La relacin entre el tratamiento y el resultado es no lineal o altamente compleja.
282 * Hay colinealidad fuerte entre covariables (puede inflar varianzas).
283 * El nmero de covariables es muy grande en relacin al tamao de muestra.
284 * El modelo lineal no ajusta bien (mala especificacin, alta heterocedasticidad, etc.).
285
286 Limitaciones:
287 * Supone que todas las relaciones relevantes son capturadas por una forma lineal.
288 * No captura interacciones complejas o efectos heterogneos.
289 * Puede ser sesgado si no se incluyen todas las variables del conjunto de ajuste
    correcto.
290
291 ---
292
293 2. **`backdoor.distance_matching`**: Estima el efecto causal emparejando unidades
    tratadas y no tratadas que tengan valores similares en las variables confusoras,
    usando una mtrica de distancia (como distancia Euclidiana o de Mahalanobis).
294
295 Cundo usarlo?
296 * Cuando se tienen confusores observables bien medidos y se desea evitar suposiciones
    paramtricas.
297 * Si el investigador desea mantener la interpretacin intuitiva de comparar unidades
    "similares".
298 * En estudios con tamao de muestra moderado y un nmero manejable de covariables.
299 * Cuando la relacin entre covariables y resultado no se asume lineal.
300
301 Cundo NO usarlo?
302 * Cuando hay alta dimensionalidad (muchas covariables), lo que dificulta encontrar
    buenos emparejamientos.
303 * Si la distancia entre unidades tratadas y de control es grande (mal emparejamiento).
304 * Cuando hay escasez de controles parecidos para las unidades tratadas (o viceversa).

```

```

305 * En presencia de confusores no observados (viola el supuesto de ignorabilidad
306 condicional).
307
308 Limitaciones:
309 * Puede descartar muchas unidades si no se encuentran buenos pares (reduce poder
310 estadístico).
311 * Sensible a la elección de la métrica de distancia.
312 * No estima efectos globales si los matches son locales y heterogéneos
313
314 ---
315
316 3. **`backdoor.propensity_score_stratification`**: Estima el efecto causal agrupando las
317 unidades en estratos (o bloques) según su score de propensión la probabilidad de
318 recibir el tratamiento dado los confusores y comparando los resultados dentro de cada
319 estrato.
320
321 Cundo usarlo?
322 * Cuando se puede estimar un score de propensión confiable a partir de los confusores
323 observados.
324 * Si se quiere reducir el sesgo de selección sin asumir una forma funcional específica
325 para el resultado.
326 * En contextos con una cantidad moderada de datos y buena superposición entre tratados
327 y no tratados.
328 * Cuando se desea una estimación ajustada del efecto promedio (ATE), pero manteniendo
329 cierta flexibilidad.
330
331 Cundo NO usarlo?
332 * Cuando el modelo de score de propensión está mal especificado o no balancea bien los
333 covariables.
334 * Si hay poca superposición (overlap) entre grupos tratados y no tratados (estratos mal
335 definidos).
336 * Cuando hay pocos datos: los estratos pueden ser demasiado pequeños para estimar
337 efectos confiables.
338 * En presencia de confusores no observados.
339
340 Limitaciones:
341 * Depende de una buena estimación del score de propensión.
342 * Puede haber heterogeneidad dentro de los estratos que no se corrige completamente.
343 * No adecuado si el número de estratos es demasiado bajo (subajuste) o demasiado alto
344 (ruido).
345
346 ---
347
348 4. **`backdoor.propensity_score_matching`**: Estima el efecto causal emparejando unidades
349 tratadas con no tratadas que tienen scores de propensión similares, lo que busca
350 balancear los confusores observados entre grupos.
351
352 Cundo usarlo?
353 * Cuando se puede estimar un score de propensión robusto a partir de covariables
354 observadas.
355 * Si se desea hacer comparaciones directas entre unidades tratadas y no tratadas
356 similares.
357 * En estudios observacionales con buena superposición de scores entre grupos.
358 * Cuando se busca una estimación del efecto promedio en las unidades tratadas (ATT).
359
360 Cundo NO usarlo?
361 * Cuando hay baja superposición entre los scores de propensión (pocos matches posibles).
362 * Si el modelo del score de propensión está mal especificado (no balancea bien los
363 grupos).
364 * Cuando se pierde demasiada información debido al descarte de unidades no emparejadas.
365 * En presencia de confusores no observados.
366
367 Limitaciones:
368 * No siempre usa toda la muestra (puede eliminar unidades sin match).

```

```

351 * La calidad del emparejamiento depende completamente del modelo de propensin.
352 * No controla por diferencias residuales post-matching a menos que se combine con
    regresin (matching + adjustment).
353
354 ---
355
356 5. backdoor.propensity_score_weighting: Estima el efecto causal asignando a cada
    unidad un peso inversamente proporcional a su probabilidad de recibir el tratamiento
    (score de propensin), para crear una muestra pseudo-populacional balanceada.
357
358 Cundo usarlo?
359 * Cuando se busca una estimacin del efecto promedio (ATE) usando toda la muestra.
360 * Si se puede estimar con precisin el score de propensin.
361 * Cuando se quiere evitar el descarte de unidades que ocurre en mtodos de
    emparejamiento.
362 * En estudios con buena superposicin entre tratados y no tratados (overlap).
363
364 Cundo NO usarlo?
365 * Cuando el modelo de propensin produce scores extremos cercanos a 0 o 1 (pesos
    inestables).
366 * Si hay poca superposicin entre los grupos (lo que genera alta varianza).
367 * Cuando el tamao de muestra es pequeno (los pesos grandes pueden sesgar los
    resultados).
368 * En presencia de confusores no observados o score mal especificado.
369
370 Limitaciones:
371 * Muy sensible a scores de propensin extremos (inflan varianza e inestabilidad).
372 * Requiere diagnstico posterior (verificacin de balance despues de aplicar los pesos).
373 * El resultado depende en gran medida de la calidad del modelo de score.
374
375 ---
376
377 6. iv.instrumental_variable: Estima el efecto causal utilizando una variable
    instrumental (IV), es decir, una variable que afecta al tratamiento pero no afecta
    directamente al resultado (solo a travs del tratamiento) y no est relacionada con
    confusores no observados.
378
379 Cundo usarlo?
380 * Cuando se sospecha la existencia de confusores no observados que hacen invlidos los
    mtodos de ajuste directo (como backdoor).
381 * Si se dispone de un instrumento vlido que cumple las condiciones de relevancia y
    validez (independencia y exclusin).
382 * Cuando se desea estimar el efecto causal local en la subpoblacin afectada por el
    instrumento (LATE).
383 * En contextos donde la asignacin al tratamiento est parcialmente determinada por una
    poltica, incentivo o factor externo.
384
385 Cundo NO usarlo?
386 * Cuando no se tiene un instrumento vlido (por ejemplo, el IV est correlacionado con
    confusores o afecta directamente el resultado).
387 * Si el instrumento es dbil (tiene poca influencia sobre el tratamiento).
388 * Cuando se desea estimar un efecto global (ATE), pero el IV solo identifica un efecto
    local.
389 * En muestras pequenas, donde la varianza del estimador puede ser muy alta.
390
391 Limitaciones:
392 * Difcil encontrar instrumentos verdaderamente vlidos en la prctica.
393 * Estima un efecto local (LATE), no necesariamente aplicable a toda la poblacin.
394 * Resultados sensibles a violaciones del supuesto de exclusin.
395
396 ---
397
398 7. iv.regression_discontinuity: Estima el efecto causal aprovechando una regla de

```

```

    asignacin basada en un umbral (discontinuidad), donde el tratamiento se asigna si una
    variable (por ejemplo, edad, puntaje, ingreso) cruza un punto de corte especifico.
399
400 Cundo usarlo?
401 * Cuando existe una **regla clara y exgena** de asignacin al tratamiento basada en un
    umbral.
402 * Si se tiene acceso a datos justo antes y despues del punto de corte (alta densidad de
    observaciones cerca del umbral).
403 * Cuando se busca una estimacin creble del efecto causal local (LATE) alrededor del
    umbral.
404 * En entornos donde se aplican polticas o programas con criterios de elegibilidad
    estrictos.
405
406 Cundo NO usarlo?
407 * Cuando no existe una discontinuidad clara o la regla de asignacin no se cumple
    rigurosamente (manipulacin del umbral).
408 * Si los individuos pueden influir su posicin respecto al umbral (violacin de
    aleatorizacin implcita).
409 * Cuando los datos cerca del umbral son escasos o muy ruidosos.
410 * Si se desea una estimacin del efecto en toda la poblacin (no solo en el vecindario
    del umbral).
411
412 Limitaciones:
413 * Solo estima el efecto local cerca del punto de corte (no generalizable).
414 * Requiere muchos datos alrededor del umbral para ser estadsticamente potente.
415 * Supone que otros factores no cambian bruscamente en el mismo umbral (lo que puede
    invalidar la identificacin).
416
417 ---
418 -> Fase 1: Ejecutar diagnstico (versin corregida)
419 Antes de tomar cualquier decisin, debes ejecutar la herramienta llamada
    diagnostico_causal_df_tool.
420
421 Esta herramienta espera un nico diccionario como argumento, con las siguientes claves:
422 - "treatment_col": una cadena con el nombre de la columna binaria de tratamiento (por
    ejemplo: "treatment").
423 - "backdoor_vars": una lista con los nombres de las variables de ajuste (confusoras)
    identificadas.
424
425 Ejemplo correcto de invocacin:
426 Action: diagnostico_causal_df_tool
427 Action Input: {{
428     "treatment_col": "treatment",
429     "backdoor_vars": ["x", "y", "z"]
430 }}
431
432 El resultado ser un JSON que contiene:
433 - Medidas de balance entre grupos (diferencia_media_estandarizada, cramers_v)
434 - Diagnstico del score de propensin (auc, extremos, overlap_visual)
435 - Sugerencias automticas en recomendaciones_metodologicas
436
437 ---
438
439 -> Fase 2: Recomendacin
440 Una vez que obtengas ese JSON (almacenado en una variable llamada diagnostico), analiza
    cuidadosamente sus valores para decidir:
441 - Si es apropiado usar regresin (requiere balance).
442 - Si matching o IPW son viables (requieren buen overlap y sin scores extremos).
443 - Si el score es poco til o hay desbalance fuerte, evita esos mtodos.
444
445 No uses ningn mtodo si el diagnstico lo desaconseja.
446
447 Nota: Si existe evidencia o sospecha de **efectos heterogneos** del tratamiento (por

```

```

ejemplo, diferentes efectos segn grupos o covariables), evita mtodos como
`linear_regression` simples y considera:
448 - Agregar interacciones entre tratamiento y covariables,
449 - Estratificacin por subgrupos,
450 - O modelos ms flexibles si estn disponibles.
451
452 AUC muy alto (> 0.9): Aunque puede parecer bueno, en inferencia causal esto suele
indicar **pobre solapamiento** entre tratados y controles, lo que desaconseja el uso
de `matching` o `IPW` debido a inestabilidad o falta de comparables.
453 Si el nmero de variables (o categoras de variables) es alto respecto al tamao de
muestra, `linear_regression` puede generar modelos inestables o con varianzas
infladas. Considera tcnicas de reduccin o penalizacin si persistes en usar regresin.
454
455 Importante: Si usas `linear_regression` y las covariables de ajuste son categoricas,
**deben ser codificadas correctamente** (por ejemplo, con one-hot encoding) antes de
ajustar el modelo.
456
457 Si ningn mtodo es viable (por ejemplo, hay mal solapamiento, scores extremos o
confusores no observados), puedes devolver:
458 - Una advertencia indicando que **no es posible estimar un efecto causal confiable**
con los datos actuales.
459 - O sugerir explorar mtodos alternativos como `iv.instrumental_variable` si hay
posibles instrumentos vlidos.
460
461 ---
462
463 -> Instrucciones para la salida
464 Devuelve un nico bloque de cdigo Python (python_repl_ast) donde:
465
466 - Asumes que ya tienes el resultado de la herramienta en una variable llamada
diagnostico.
467 - Lees sus valores clave.
468 - Decides el mejor mtodo causal disponible y lo justificas.
469 - Usas print(json.dumps(...)) para devolver un diccionario con:
470 - "metodo_recomendado": nombre del mtodo sugerido (ej. "backdoor.linear_regression").
471 - "justificacion": explicacin profesional en espaol, basada en los datos.
472
473 Ejemplo del formato esperado:
474 {{
475 "metodo_recomendado": "backdoor.linear_regression",
476 "justificacion": "El score de propensin muestra un AUC moderado de 0.56 y no hay scores
extremos. Adems, las diferencias estandarizadas entre grupos en las covariables son
bajas. Por ello, se recomienda usar regresin lineal ajustada con las variables de
ajuste identificadas."
477 }}
478
479 IMPORTANTE:
480 - Incluye los valores numricos reales devueltos por la herramienta (auc, cramers_v,
etc.).
481 - No inventes cifras ni escribas el JSON a mano fuera del cdigo ejecutado.
482 - La nica salida vlida debe generarse dentro de un bloque de cdigo con
print(json.dumps(...)).
483
484 Recuerda:
485 - Usa `Action: diagnostico_causal_df_tool` para iniciar.
486 - Una vez tengas el resultado, escribe el JSON final en tu `Final Answer`.
487
488 """
489
490 def generar_prompt_interpretar_estimacion_causal(treatment_col: str, outcome_col: str,
method: str) -> str:
491     return f"""
492 Rol:
493 Eres un **experto en inferencia causal y comunicacin cientfica**, con conocimientos

```

```

    tcnicos en DoWhy y experiencia explicando resultados a audiencias no tcnicas. Tu
    tarea es interpretar una estimacin de efecto causal a partir de los resultados
    devueltos por una herramienta computacional.
494
495 ---
496
497 Contexto del anlisis:
498 - Se ha realizado un estudio de inferencia causal sobre un conjunto de datos reales.
499 - La variable de tratamiento es `{treatment_col}` (binaria: 0 = no tratado, 1 =
    tratado).
500 - La variable de resultado es `{outcome_col}` (continua o numrica).
501 - El mtodo de estimacin causal utilizado ha sido: `{method}`, implementado con la libreria
    `Dowhy`.
502
503 ---
504
505 Herramienta a utilizar:
506 Para obtener el valor estimado y su intervalo de confianza, debes ejecutar la
    herramienta:
507 `estimate_causal_effect_tool`
508
509 Esta herramienta devolver un JSON con dos claves:
510
511 - `"estimate"`: el valor estimado del efecto causal.
512 - `"intervalo_confianza"`: un intervalo de confianza del 95% asociado a la estimacin.
513
514 Estructura de ejecucin:
515 Action: estimate_causal_effect_tool
516 Action Input: {}
517
518 ---
519
520 Tareas que debes realizar:
521
522 Una vez recibido el JSON con los resultados:
523 1. Extrae el valor estimado (`estimate`) y el intervalo de confianza
    (`intervalo_confianza`).
524
525 1b. Si la herramienta devuelve un error o el valor estimado es invlido (por ejemplo,
    nulo o inverosmil), debes informar claramente que el mtodo puede no ser adecuado
    para este dataset.
526
527 - Sugiere de forma profesional considerar otro mtodo, si corresponde.
528 - No inventes resultados. Si no puedes interpretar, explcalo claramente.
529
530 Ejemplo de frase posible:
531 > En este caso, el mtodo seleccionado no ha producido una estimacin confiable. Podra
    ser recomendable probar otro enfoque ms robusto.
532
533 2. Organiza tu interpretacin en SECCIONES, cada una con un subtítulo claro en espaol:
534 - Magnitud del efecto
535 - Intervalo de confianza y significancia
536 - Interpretacin y conclusiones
537
538 3. Redacta cada seccin con lenguaje profesional y accesible:
539 - Usa expresiones como:
540 * En promedio
541 * Este resultado sugiere que
542 * Con cierta incertidumbre
543 * No se puede descartar que
544
545 4. Haz que el texto sea fcil de escanear:
546 - Usa prrafos cortos.
547 - No repitas valores exactos sin explicarlos.

```

```

548 - No incluyas cdigo ni jerga tcnica.
549
550 ---
551
552 Idioma de la respuesta:
553 Tu respuesta debe estar completamente en **espaol profesional y claro**.
554
555 !! No incluyas:
556 - Cdigo
557 - Fragmentos tcnicos
558 - Trminos como "DoWhy", "librera", "JSON", etc.
559 - Nombres de columnas
560
561 ---
562
563 Salida esperada:
564 Un texto estructurado en **tres secciones con subttulos**, redactado en espaol
    profesional, accesible y amigable para el lector final.
565 """
566
567 def generar_prompt_validacion_robustez() -> str:
568     return """
569 Rol:
570 Eres un **experto en validacin de resultados de inferencia causal**. Tu tarea es analizar
    la robustez de un efecto estimado utilizando una prueba de refutacin.
571
572 ---
573
574 Herramienta a utilizar:
575 Debes ejecutar `refute_estimate_tool`, que devolver un JSON con el resultado de una nica
    tcnica de refutacin: `random_common_cause`.
576
577 El resultado incluye:
578 - `"refutacion"`: descripcin completa del anlisis de refutacin.
579 - `"p_valor"`: valor p asociado, si est disponible.
580 - `"estimate"`: valor estimado original del efecto causal.
581 - `"new_effect"`: nuevo efecto estimado tras la refutacin.
582
583 ---
584
585 Tu tarea:
586
587 1. **Analiza el resultado devuelto**:
588     - El efecto estimado cambia sustancialmente tras la refutacin?
589     - El valor p es alto (por ejemplo, > 0.05)?
590
591 2. **Evala la robustez del efecto**:
592     - Si el cambio es pequeno en relacin con el estimado original, considera que **es
    robusto**.
593     - Si el cambio es notable o el valor p es bajo, sugiere que el resultado puede **no
    ser robusto**.
594
595 3. **Redacta una respuesta clara y profesional** con dos secciones:
596
597 ---
598
599 Salida esperada:
600
601 - **Resultado de la refutacin**: Resume lo que se observ, indicando el cambio entre el
    efecto original y el nuevo.
602 - **Conclusin de robustez**: Explica si el efecto estimado parece robusto o sensible a
    posibles variables omitidas.
603
604 -> Redacta en espaol formal, claro y sin referencias a cdigo o detalles tcnicos.

```

605 """

Listing VIII.1: `prompts.py`: plantillas y reglas de *prompting* para causalidad

ANEXO IX

Código fuente de `causal.py`

Este anexo reúne el código completo empleado en el módulo de análisis causal descrito en el cuerpo del trabajo: el *agente con herramientas*, los *prompts* y el *pipeline principal*. Se proporciona para facilitar la reproducibilidad y consulta técnica.

```
1 import streamlit as st
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import re
6 import json
7 from dowhy import CausalModel
8 from auto_doc.models.causal_agent import CausalAgentFactory
9 from auto_doc.services.useful_services import load_file
10
11 from auto_doc.services.prompts import (
12     generar_prompt_codigo_tratamiento,
13     generar_prompt_hipotesis,
14     generar_prompt_variables_relacionadas,
15     generar_prompt_dag_con_datos,
16     generar_prompt_identificacion_efecto_seguro,
17     generar_prompt_recomendar_metodo_critico,
18     generar_prompt_interpretar_estimacion_causal,
19     generar_prompt_validacion_robustez,
20     generar_descripcion_dataset,
21 )
22
23 def mostrar_inferencia_causal():
24     st.title(" Inferencia Causal")
25
26     if "file_path" not in st.session_state:
27         st.error(" No se ha subido ningn archivo. Por favor, vuelve atrs y sube uno.")
28         return
29
30     file_path = str(st.session_state["file_path"])
31
32     try:
33         df = load_file(file_path)
34     except Exception as e:
35         st.error(f" Error al cargar el archivo: {e}")
36         return
37
38     st.session_state["df"] = df
39
40     st.header("Seleccin del Tratamiento o Pregunta Causal")
41
42     opcion = st.radio(
43         "Quieres seleccionar una columna como tratamiento?",
```



```

100         - Valor **0** `{valor_0}`
101         """)
102     else:
103         es_valida = False
104
105     if es_valida:
106         st.session_state["variable_objetivo"] = outcome
107         st.success(f"Variable objetivo seleccionada: **{outcome}**")
108
109         if st.button(" Confirmar tratamiento y variable objetivo"):
110             st.session_state["confirmado"] = True
111
112             if "causal_agent" not in st.session_state:
113                 with st.spinner("Iniciando agente..."):
114                     factory = CausalAgentFactory()
115                     st.session_state["causal_agent"] =
116 factory.create_agent(df_tratado)
117
118                 st.success("Tratamiento y outcome confirmados. Puedes continuar
119 con el analisis causal.")
120             else:
121                 st.error(" La variable objetivo debe ser numrica o binaria.
122 Selecciona otra.")
123
124 else:
125     pregunta_causal = st.text_input("Formula tu pregunta causal (por ejemplo: Qu
126 influye en el precio?)")
127     if pregunta_causal:
128         st.session_state["pregunta_causal"] = pregunta_causal
129
130         if "causal_agent" not in st.session_state:
131             with st.spinner("Iniciando agente..."):
132                 factory = CausalAgentFactory()
133                 agent = factory.create_agent(df)
134                 st.session_state["causal_agent"] = agent
135
136         if "candidatas" not in st.session_state and "candidatas_encontradas" not in
137 st.session_state:
138             if st.button(" Encontrar variables candidatas"):
139                 agent = st.session_state["causal_agent"]
140                 prompt = generar_prompt_variables_relacionadas(pregunta_causal)
141                 with st.spinner("Consultando al agente..."):
142                     response = agent.run(prompt)
143
144                 # Limpiar el texto: extraer solo la parte entre llaves
145                 match = re.search(r"\{.*\}", response, re.DOTALL)
146                 if match:
147                     json_text = match.group(0)
148                     try:
149                         parsed = json.loads(json_text)
150                         st.session_state["variable_objetivo"] = parsed["objetivo"]
151                         st.session_state["candidatas"] = parsed["candidatas"]
152                         st.session_state["candidatas_encontradas"] = True
153                     except json.JSONDecodeError as e:
154                         st.error(" No se pudo interpretar la respuesta del agente
155 como JSON vlido.")
156                 else:
157                     st.code(json_text)
158             else:
159                 st.error(" No se encontr un bloque JSON en la respuesta del
160 agente.")
161                 st.code(response)

```

```

157 # Seleccin de variable candidata
158 if st.session_state.get("candidatas_encontradas"):
159     candidatas = st.session_state["candidatas"]
160
161     st.subheader("Variables candidatas encontradas")
162     seleccion_opcion = st.radio(
163         "Selecciona cmo deseas elegir la variable para el anlisis causal:",
164         ("Seleccionar una de las variables candidatas sugeridas", "Quiero seleccionar
165 otra variable manualmente"),
166         key="opcion_variable_causal"
167     )
168
169     if seleccion_opcion == "Seleccionar una de las variables candidatas sugeridas":
170         variable_candidata = st.radio("Selecciona una de las variables candidatas:",
171 options=candidatas, key="seleccion_candidata")
172     else:
173         columnas_disponibles = [col for col in df.columns if col !=
174 st.session_state.get("variable_objetivo")]
175         variable_candidata = st.selectbox("Selecciona cualquier otra variable del
176 dataset:", options=columnas_disponibles, key="seleccion_manual")
177
178     if variable_candidata:
179         st.session_state["variable_candidata_temp"] = variable_candidata
180         st.info(f"Has seleccionado: **{variable_candidata}**. Pulsa el botn para
181 continuar.")
182
183     if st.button("Confirmar variable candidata"):
184         st.session_state["variable_candidata"] = variable_candidata
185         # Eliminar hiptesis previas si cambi la variable
186         st.session_state.pop("lista_hiptesis", None)
187
188 # Hiptesis
189 if "variable_candidata" in st.session_state and "variable_objetivo" in
190 st.session_state:
191     vi = st.session_state["variable_candidata"]
192     vo = st.session_state["variable_objetivo"]
193
194     st.markdown(f" Se analizar si **{vi}** tiene un efecto causal sobre **{vo}**.")
195     st.subheader("Sugerencias de hiptesis causales")
196
197     if "lista_hiptesis" not in st.session_state:
198         prompt = generar_prompt_hiptesis(vo, vi, generar_descripcion_dataset(df))
199
200         agent = st.session_state["causal_agent"]
201
202         with st.spinner("Generando hiptesis..."):
203             sugerencias = agent.run(prompt)
204
205         st.session_state["lista_hiptesis"] = [
206             linea.strip("- ").strip()
207             for linea in sugerencias.strip().split("\n")
208             if linea.strip() and not linea.lower().startswith("aqu hay")
209         ]
210
211         opciones = st.session_state["lista_hiptesis"] + ["Quiero escribir mi propia
212 hiptesis"]
213
214         seleccion = st.radio(
215             "Selecciona una hiptesis sugerida o escribe la tuya propia:",
216             opciones,
217             key="seleccion_hiptesis_radio"
218         )
219
220         if seleccion == "Quiero escribir mi propia hiptesis":

```

```

214     hipotesis_usuario = st.text_area(
215         "Escribe tu hipotesis causal:",
216         placeholder=f"Ejemplo: A mayor {vi}, mayor ser el {vo}...",
217         key="hipotesis_input"
218     )
219     if hipotesis_usuario:
220         st.session_state["hipotesis_usuario"] = hipotesis_usuario
221         st.success(" Hipotesis registrada.")
222     else:
223         st.session_state["hipotesis_usuario"] = seleccion
224         st.success(f" Hipotesis seleccionada: *{seleccion}*")
225
226     # Paso siguiente: Confirmar hipotesis y generar tratamiento
227     if "hipotesis_usuario" in st.session_state:
228         st.subheader("Confirmar hipotesis para generar la columna de tratamiento")
229         if st.button("Confirmar hipotesis y generar columna treatment"):
230             agente = st.session_state["causal_agente"]
231             hipotesis = st.session_state["hipotesis_usuario"]
232
233             prompt_codigo = generar_prompt_codigo_tratamiento(hipotesis)
234             with st.spinner("Generando cdigo para la columna de tratamiento..."):
235                 respuesta = agente.run(prompt_codigo)
236
237             match = re.search(r"```json\s*(\{.*?\})\s*```", respuesta, re.DOTALL)
238             if match:
239                 json_text = match.group(1)
240             else:
241                 match = re.search(r"(\{.*?\})", respuesta, re.DOTALL)
242                 if match:
243                     json_text = match.group(1)
244                 else:
245                     st.error(" No se encontr JSON vlido en la respuesta del agente.")
246                     st.code(respuesta)
247                     return
248
249             try:
250                 respuesta_json = json.loads(json_text)
251                 explicacion = respuesta_json.get("explicacion", "")
252                 codigo = respuesta_json.get("codigo", "").strip()
253
254                 if "ERROR" in explicacion or not codigo:
255                     st.error(" No se pudo generar la columna de tratamiento con esta
hipotesis.")
256
257                     st.markdown(f"**Motivo:** {explicacion}")
258                 else:
259                     df_base = df.copy()
260                     namespace = {"df": df_base}
261                     exec(codigo, namespace)
262                     df_tratado = namespace["df"]
263                     st.session_state["df_tratado"] = df_tratado
264                     st.session_state["confirmado"] = True
265                     st.session_state["treatment"] = "treatment"
266
267                     st.success(" Columna 'treatment' generada y aplicada al DataFrame.")
268                     st.markdown(f"**Explicacin:** {explicacion}")
269                     st.markdown(" As se ve el dataset con la nueva columna `treatment`:")
270                     st.dataframe(df_tratado.head())
271
272                     # Reinicializar agente con df actualizado
273                     with st.spinner(" Reinicializando agente con el DataFrame
actualizado..."):
274                         factory = CausalAgentFactory()
275                         agent = factory.create_agent(df_tratado)
276                         st.session_state["causal_agente"] = agent

```

```

276
277         except json.JSONDecodeError:
278             st.error(" No se pudo interpretar la respuesta como JSON vlido.")
279             st.code(json_text)
280
281     # Diagnostico de viabilidad causal
282     if (
283         st.session_state.get("confirmado") and
284         "df_tratado" in st.session_state and
285         "variable_objetivo" in st.session_state
286     ):
287         df = st.session_state["df_tratado"]
288         variable_objetivo = st.session_state["variable_objetivo"]
289
290         st.subheader(" Diagnostico previo al analisis causal")
291
292         # Grfico de barras de tratamiento
293         st.markdown("### Distribucin de grupos de tratamiento")
294         fig, ax = plt.subplots(figsize=(5, 3.5)) # Tamao intermedio y uniforme
295         df["treatment"].value_counts().sort_index().plot(kind="bar", ax=ax,
296             color=["#4C72B0", "#55A868"])
297         ax.set_xlabel("Grupo de tratamiento (0 = control, 1 = tratado)")
298         ax.set_ylabel("Nmero de observaciones")
299         ax.set_title("Distribucin de observaciones por grupo")
300         ax.tick_params(axis='x', rotation=0)
301
302         # Mostrar imagen en Streamlit
303         import io
304         buf = io.BytesIO()
305         fig.tight_layout()
306         fig.savefig(buf, format="png", bbox_inches="tight")
307         st.image(buf, width=400)
308
309         # Analisis de balance entre grupos
310         st.markdown("### Est desbalanceado?")
311         counts = df["treatment"].value_counts()
312         total = counts.sum()
313         porcentajes = counts / total * 100
314
315         texto_balance = "\n".join([
316             f"- Grupo {idx}: {row} observaciones ({porcentajes[idx]:.1f}%)",
317             for idx, row in counts.items()
318         ])
319         st.markdown(texto_balance)
320
321         if any(counts < 10):
322             st.warning(" Uno de los grupos tiene menos de 10 observaciones. Esto puede
323             dificultar el analisis causal.")
324
325         diferencia = abs(porcentajes[0] - porcentajes[1])
326         if diferencia > 40:
327             st.info(" Los grupos estn notablemente desbalanceados. Considera mtodos
328             robustos de estimacin o recoleccin adicional de datos.")
329         else:
330             st.success(" Los grupos de tratamiento estn razonablemente balanceados.")
331
332         # Estadsticas descriptivas
333         st.markdown("### Estadsticas de la variable objetivo por grupo")
334         resumen = df.groupby("treatment")[variable_objetivo].agg(["mean", "std"])
335         resumen.columns = ["Media", "Desviacin estndar"]
336         st.dataframe(resumen.style.format("{:.2f}"))
337
338         # Boxplot
339         st.markdown("### Distribucin de la variable objetivo por grupo de tratamiento")

```

```

337     fig2, ax2 = plt.subplots(figsize=(5, 3.5)) # Mismo tamaño que el gráfico anterior
338     sns.boxplot(x="treatment", y=variable_objetivo, data=df, ax=ax2)
339     ax2.set_xlabel("Grupo de tratamiento")
340     ax2.set_ylabel(variable_objetivo)
341     ax2.set_title("Distribución del precio por tratamiento")
342     fig2.tight_layout()
343
344     buf2 = io.BytesIO()
345     fig2.savefig(buf2, format="png", bbox_inches="tight")
346     st.image(buf2, width=400)
347
348     # Test de diferencia
349     from scipy.stats import ttest_ind
350     g0 = df[df["treatment"] == 0][variable_objetivo]
351     g1 = df[df["treatment"] == 1][variable_objetivo]
352     t_stat, p_val = ttest_ind(g0, g1, equal_var=False)
353
354     st.markdown("### Test estadístico de diferencia")
355     st.markdown(f"- Estadístico t: `{t_stat:.2f}`")
356     st.markdown(f"- Valor p: `{p_val:.5f}`")
357
358     if p_val < 0.05:
359         st.success("La diferencia es estadísticamente significativa (p < 0.05).")
360     else:
361         st.info("No se detecta diferencia estadísticamente significativa.")
362
363     from scipy.stats import chi2_contingency
364
365     # Paso 2: Visualización y construcción del DAG
366     if (
367         st.session_state.get("confirmado") and
368         "df_tratado" in st.session_state and
369         "variable_objetivo" in st.session_state and
370         "causal_agent" in st.session_state
371     ):
372
373         df_tratado = st.session_state["df_tratado"]
374         causal_agent = CausalAgentFactory().create_agent(df_tratado)
375
376         st.subheader("Generación DAG (Gráfico de Supuestos)")
377
378         if "dag_dot_generado" not in st.session_state:
379             if st.button("Generar DAG automáticamente con ayuda del agente"):
380                 df = st.session_state["df_tratado"]
381                 outcome = st.session_state["variable_objetivo"]
382                 treatment = "treatment"
383
384                 # Cálculos previos
385                 corr =
df.corr(numeric_only=True)[outcome].drop(outcome).dropna().sort_values(ascending=False)
386                 conf_cat = {}
387                 from scipy.stats import chi2_contingency
388                 for col in df.select_dtypes(include='object').columns:
389                     if col != treatment and col != outcome:
390                         try:
391                             tab = pd.crosstab(df[col], df[treatment])
392                             if tab.shape[1] == 2:
393                                 _, p, _, _ = chi2_contingency(tab)
394                                 if p < 0.05:
395                                     conf_cat[col] = round(p, 5)
396                         except Exception:
397                             continue
398
399                 resumen = {

```

```

400         "treatment": treatment,
401         "outcome": outcome,
402         "correlaciones": corr.head(5).to_dict(),
403         "confusoras_categoricas": conf_cat
404     }
405
406     with st.spinner("Generando DAG..."):
407         prompt_dag = generar_prompt_dag_con_datos(resumen)
408         respuesta = causal_agent.run(prompt_dag)
409         match = re.search(r"```(?:dot)?\s*(.*)```", respuesta, re.DOTALL)
410         dot_code = match.group(1).strip() if match else respuesta.strip()
411
412         st.session_state["dag_dot_generado"] = dot_code
413         st.session_state["modo_edicion_dag"] = False
414         st.session_state["dag_dot"] = None
415         #st.experimental_rerun() # Forzar recarga para que aparezca el
grfico
416
417         # Mostrar DAG generado solo si existe y no se est editando
418         if "dag_dot_generado" in st.session_state and not
st.session_state.get("modo_edicion_dag", False):
419             dot_code = st.session_state["dag_dot_generado"]
420             if dot_code.strip(): # Asegura que no est vaco
421                 st.markdown("### Propuesta automtica del DAG")
422                 st.graphviz_chart(dot_code)
423
424             col1, col2 = st.columns(2)
425             with col1:
426                 if st.button(" Confirmar grafo"):
427                     st.session_state["dag_dot"] = dot_code
428                     st.session_state["dag_confirmado"] = True
429                     st.success(" DAG confirmado. Puedes continuar.")
430             with col2:
431                 if st.button(" Editar grafo"):
432                     st.session_state["modo_edicion_dag"] = True
433
434
435         # Edicin del DAG
436         if st.session_state.get("modo_edicion_dag", False):
437             with st.form("editar_dag"):
438                 dot_editado = st.text_area(
439                     "Edita el DAG en formato DOT:",
440                     value=st.session_state["dag_dot_generado"],
441                     height=220,
442                     key="input_dag_editado"
443                 )
444                 submitted = st.form_submit_button(" Aplicar edicin (previsualizar)")
445
446             if submitted:
447                 st.session_state["dag_editado_temporal"] = dot_editado
448                 st.success(" Cambios aplicados. Revisa el grfico y confirma si ests
conforme.")
449
450             if "dag_editado_temporal" in st.session_state:
451                 st.markdown("### Visualizacin del DAG editado")
452                 st.graphviz_chart(st.session_state["dag_editado_temporal"])
453
454                 if st.button(" Confirmar grafo editado"):
455                     st.session_state["dag_dot"] = st.session_state["dag_editado_temporal"]
456                     st.session_state["modo_edicion_dag"] = False
457                     del st.session_state["dag_editado_temporal"]
458                     st.session_state["dag_confirmado"] = True
459                     st.success(" El DAG editado ha sido confirmado como definitivo.")
460

```



```

461 # Paso 3: Construccin del modelo DoWhy
462 import pydot
463 import networkx as nx
464 if (
465     "df_tratado" in st.session_state and
466     "variable_objetivo" in st.session_state and
467     "dag_dot" in st.session_state and
468     "dag_confirmado" in st.session_state and
469     st.session_state["dag_confirmado"]
470 ):
471     import pydot
472     import networkx as nx
473
474     st.subheader(" Construccin del modelo causal")
475     st.markdown("### DAG utilizado para el modelo")
476     st.graphviz_chart(st.session_state["dag_dot"])
477
478     if st.button(" Generar modelo causal con DoWhy"):
479         with st.spinner("Construyendo modelo causal..."):
480             try:
481                 dot_code = st.session_state["dag_dot"]
482
483                 # Parsear el cdigo DOT
484                 graphs = pydot.graph_from_dot_data(dot_code)
485                 if not graphs:
486                     raise ValueError("No se pudo interpretar el cdigo DOT.")
487                 pydot_graph = graphs[0]
488
489                 # Conversin manual a NetworkX s
490                 nx_graph = nx.DiGraph()
491                 for node in pydot_graph.get_nodes():
492                     name = node.get_name().strip('""')
493                     if name in ('node',): continue # ignora globales
494                     nx_graph.add_node(name, **node.get_attributes())
495
496                 for edge in pydot_graph.get_edges():
497                     src = edge.get_source().strip('""')
498                     tgt = edge.get_destination().strip('""')
499                     nx_graph.add_edge(src, tgt)
500
501                 st.session_state["nx_graph"] = nx_graph
502
503                 # Crear modelo causal
504                 model = CausalModel(
505                     data=st.session_state["df_tratado"],
506                     treatment=st.session_state["treatment"],
507                     outcome=st.session_state["variable_objetivo"],
508                     graph=nx_graph
509                 )
510
511                 st.session_state["causal_model"] = model
512                 st.success(" Modelo causal construido con xito.")
513                 st.markdown("### Resumen del modelo")
514                 st.markdown(f"- **Tratamiento:** `{st.session_state['treatment']}`")
515                 st.markdown(f"- **Resultado:** `{st.session_state['variable_objetivo']}`")
516                 st.markdown(f"- **N observaciones:** `{len(st.session_state['df_tratado'])}`")
517
518             except Exception as e:
519                 st.error(" Error al construir el modelo causal.")
520                 st.exception(e)
521
522 # Paso 4: Identificacin del efecto causal

```

```

523 if "causal_model" in st.session_state:
524
525     if st.button("Evaluar identificabilidad del efecto causal con el agente"):
526         with st.spinner("Ejecutando diagnostico..."):
527             try:
528                 prompt = generar_prompt_identificacion_efecto_seguro()
529                 response = st.session_state["causal_agent"].run(prompt)
530
531                 # Guardar la respuesta del agente para mostrarla incluso despues
532                 st.session_state["respuesta_identificabilidad"] = response
533
534                 # Paso necesario de DoWhy
535                 model = st.session_state["causal_model"]
536                 identified_estimand = model.identify_effect()
537                 backdoors = identified_estimand.get_backdoor_variables()
538                 st.session_state["identified_estimand"] = identified_estimand
539                 st.session_state["backdoor_variables"] = backdoors
540
541             except Exception as e:
542                 st.error(" El agente no pudo ejecutar la evaluacin.")
543                 st.exception(e)
544
545     # Mostrar la respuesta del agente si ya existe
546     if "respuesta_identificabilidad" in st.session_state:
547         response = st.session_state["respuesta_identificabilidad"]
548
549     # Limpiar texto
550     cleaned = (
551         response.replace("```json", "")
552         .replace("```", "")
553         .replace("```json", "")
554         .replace("```", "")
555         .strip()
556     )
557
558     match = re.search(r"\{.*\}", cleaned, re.DOTALL)
559     if match:
560         try:
561             parsed = json.loads(match.group(0))
562             criterio = parsed.get("criterio")
563             backdoors = parsed.get("backdoor_variables", [])
564             justificacion = parsed.get("justificacion", "")
565
566             st.session_state["backdoor_variables"] = backdoors
567
568             st.markdown("### Diagnostico de identificabilidad")
569             st.markdown(f"**Criterio usado:** `{criterio}`")
570             st.markdown(f"**Variables de ajuste:** `{backdoors}`")
571             st.markdown(f"**Justificacin:** `{justificacion}`")
572
573         except json.JSONDecodeError:
574             st.error(" JSON malformado. Revisa la estructura.")
575             st.code(cleaned)
576     else:
577         st.error(" No se encontr un bloque JSON vlido.")
578         st.code(response)
579
580     # Paso 5: Mtodo de estimacin
581     if (
582         "causal_model" in st.session_state and
583         "backdoor_variables" in st.session_state and
584         "causal_agent" in st.session_state and
585         "identified_estimand" in st.session_state
586     ):

```

```

587     st.subheader(" Recomendacin del mtodo de estimacin")
588
589     if st.button("Recomendar mejor mtodo de estimacin causal"):
590         with st.spinner(" Analizando dataset y variables de ajuste..."):
591             try:
592                 treatment = st.session_state["treatment"]
593                 backdoors = st.session_state["backdoor_variables"]
594                 prompt = generar_prompt_recomendar_metodo_critico(treatment,
595
596                 backdoors)
597
598                 response = st.session_state["causal_agent"].run(prompt)
599                 st.session_state["respuesta_metodo_recomendado"] = response # guardar
600
601                 respuesta
602
603                 match = re.search(r"\{.*\}", response, re.DOTALL)
604                 if match:
605                     resultado = json.loads(match.group(0))
606                     metodo = resultado.get("metodo_recomendado")
607                     justificacion = resultado.get("justificacion", "")
608
609                     if metodo:
610                         st.session_state["metodo_recomendado"] = metodo
611                         st.session_state["justificacion_metodo"] = justificacion
612                     else:
613                         st.session_state["metodo_recomendado"] = None
614                         st.session_state["justificacion_metodo"] = None
615                 else:
616                     st.session_state["metodo_recomendado"] = None
617                     st.session_state["justificacion_metodo"] = None
618
619             except Exception as e:
620                 st.error(" Error al obtener la recomendacin del agente.")
621                 st.exception(e)
622
623             # Mostrar contenido si ya existe
624             if "metodo_recomendado" in st.session_state and
625             st.session_state["metodo_recomendado"]:
626                 metodo = st.session_state["metodo_recomendado"]
627                 justificacion = st.session_state["justificacion_metodo"]
628
629                 st.markdown("### Mtodo recomendado por el agente:")
630                 st.markdown(f"<h2 style='color:#4CAF50; font-size:32px;'> {metodo}</h2>",
631                 unsafe_allow_html=True)
632                 st.markdown(f"***Justificacin:** {justificacion}")
633
634             # Mostrar el selectbox
635             metodos_disponibles = [
636                 "backdoor.linear_regression",
637                 "backdoor.propensity_score_matching",
638                 "backdoor.propensity_score_weighting",
639                 "backdoor.distance_matching",
640                 "backdoor.propensity_score_stratification",
641                 "iv_instrument_name",
642                 "iv.regression_discontinuity"
643
644             ]
645
646             metodo_sugerido = metodo
647
648             st.markdown("---")
649             st.markdown("### Selecciona el mtodo que deseas usar para la estimacin:")
650             opcion = st.selectbox(
651                 "Mtodo de estimacin causal:",
652                 options=metodos_disponibles,

```

```

647         index=metodos_disponibles.index(metodo_sugerido) if metodo_sugerido in
metodos_disponibles else 0,
648         help="Puedes aceptar el mtodo recomendado o seleccionar uno diferente
manualmente."
649     )
650
651     st.session_state["metodo_seleccionado"] = opcion
652     st.success(f" Mtodo seleccionado para estimacin: `{opcion}`")
653
654 # Paso 6: Estimacin del efecto causal
655 if (
656     "causal_model" in st.session_state and
657     "identified_estimand" in st.session_state and
658     "metodo_seleccionado" in st.session_state
659 ):
660     st.subheader(" Estimacin del efecto causal")
661
662     if st.button("El agente realizar la estimacin y la interpretar"):
663         with st.spinner("El agente est estimando el efecto causal..."):
664             try:
665                 if "causal_agent" not in st.session_state:
666                     st.error(" No se ha inicializado el agente causal.")
667                     st.stop()
668
669                 if "treatment" not in st.session_state or "variable_objetivo" not in
st.session_state:
670                     st.error(" Faltan las columnas de tratamiento o resultado en
session_state.")
671                     st.stop()
672
673                 causal_agent = st.session_state["causal_agent"]
674                 treatment_col = st.session_state["treatment"]
675                 outcome_col = st.session_state["variable_objetivo"]
676                 metodo = st.session_state["metodo_seleccionado"]
677
678                 prompt_interpretacion = generar_prompt_interpretar_estimacion_causal(
679                     treatment_col=treatment_col,
680                     outcome_col=outcome_col,
681                     method=metodo
682                 )
683
684                 interpretacion = causal_agent.run(prompt_interpretacion)
685                 st.session_state["interpretacion_causalidad"] = interpretacion
686
687             except Exception as e:
688                 st.error(" Ocurri un error al estimar o interpretar el efecto
causal.")
689                 st.exception(e)
690
691         if "interpretacion_causalidad" in st.session_state:
692             st.markdown("### Estimacin e interpretacin del agente:")
693             st.markdown(st.session_state["interpretacion_causalidad"])
694
695 # Paso 7: Validacin del estimado (robustez)
696 if (
697     "causal_model" in st.session_state and
698     "identified_estimand" in st.session_state and
699     "estimate" in st.session_state and
700     "causal_agent" in st.session_state
701 ):
702     st.subheader(" Validacin del estimado (robustez)")
703
704     if st.button("Validar robustez del efecto estimado con el agente"):

```

```
706         with st.spinner("Evaluando robustez del estimado..."):
707             try:
708                 prompt = generar_prompt_validacion_robustez()
709                 respuesta = st.session_state["causal_agent"].run(prompt)
710                 st.session_state["validacion_robustez"] = respuesta
711
712             except Exception as e:
713                 st.error(" Ocurri un error al validar el estimado.")
714                 st.exception(e)
715
716         if "validacion_robustez" in st.session_state:
717             st.markdown("### Resultado de la validacin del agente:")
718             st.markdown(st.session_state["validacion_robustez"])
```

Listing IX.1: `causal.py`: pipeline de análisis causal

ANEXO X

Ejemplos de salida de la sección “resumen estadístico”

Este anexo contiene algunos de los ejemplos que completan los resultados obtenidos en la ejecución de resumen estadístico.

X.1 Detección de nulos

 Valores nulos

No se encontraron valores faltantes en el DataFrame.

Figura X.1: Ejemplo de detección de nulos del *dataset* original.

X.2 Detección de duplicados

 Duplicados

El análisis del conjunto de datos no revela la presencia de registros duplicados. Por lo tanto, la calidad y la interpretación de los datos no se ven afectadas por este factor.

Figura X.2: Ejemplo de detección de duplicados del *dataset* original.

ANEXO XI

Ejemplos de salida de la sección “generador de gráficos”

Este anexo contiene distintos ejemplos de uso que se han realizado sobre la sección de visualización de la interfaz desarrollada.

XI.1 Matriz de correlación

La consulta realizada al agente fue la siguiente: “Haz una matriz de correlación de color azul”.

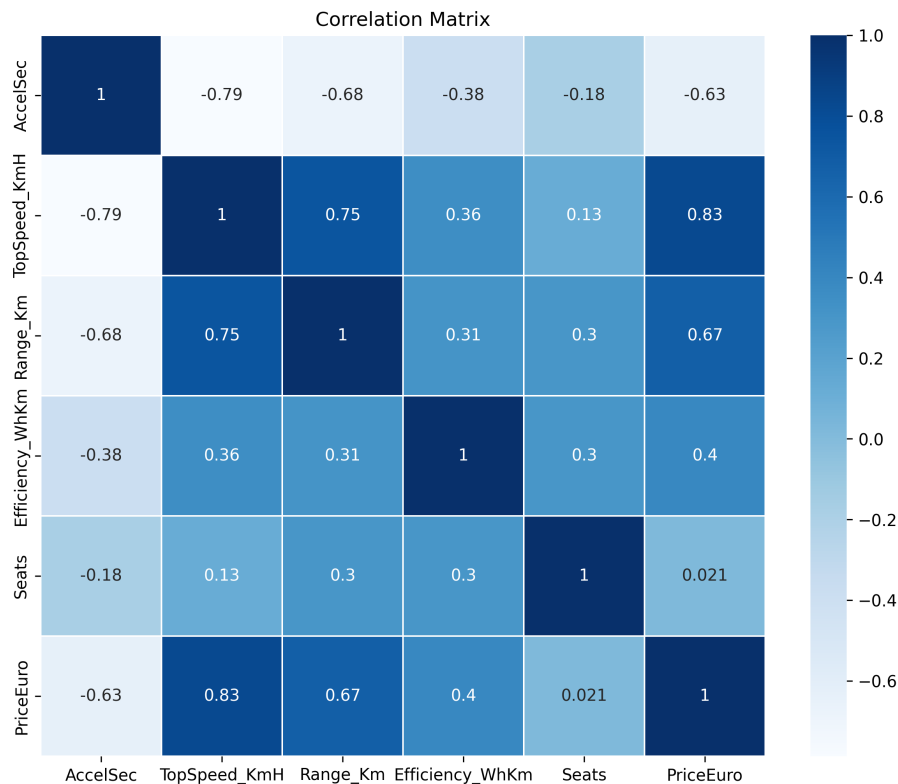


Figura XI.1: Ejemplo de matriz de correlación generada por el agente

XI.2 Gráfico de barras

La consulta realizada al agente fue la siguiente: “Haz un barplot entre las variables brand y topspeed_kmh en orden decreciente”.

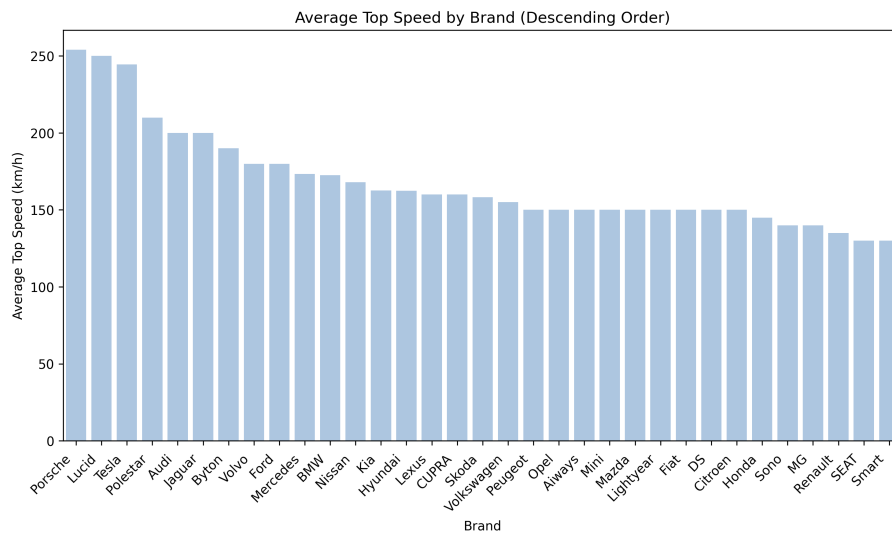
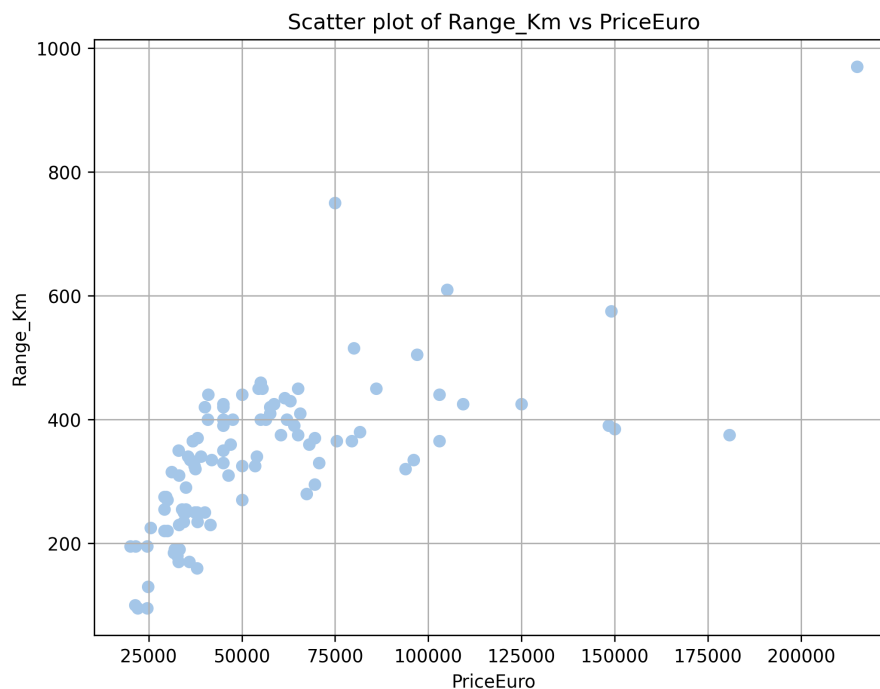


Figura XI.2: Ejemplo de barplot generado por el agente

XI.3 Gráfico de dispersión

La consulta realizada al agente fue la siguiente: “Haz un gráfico de dispersión de la autonomía (Range_Km) en función del precio (PriceEuro).”



XI.4 Gráfico *boxplot*

La consulta realizada al agente fue la siguiente: “Haz un boxplot del precio (PriceEuro) por tipo de carrocería (BodyStyle).”

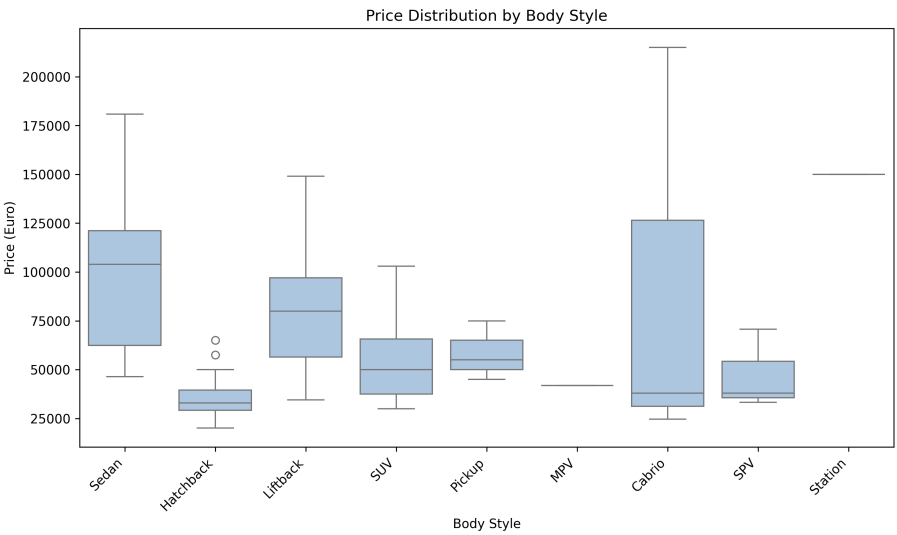


Figura XI.4: Ejemplo de boxplot generado por el agente

XI.5 Consultas inapropiadas

Pregunta al agente para graficar

Por ejemplo: 'Haz un histograma de la columna edad'

Haz lo que quieras

Enviar petición

No se generó ningún código gráfico válido.

Respuesta del agente:

No puedo generar un gráfico con esa petición. Por favor, sé claro y específico indicando qué columnas o tipo de gráfico deseas.

Figura XI.5: Ejemplo de consulta errónea

Pregunta al agente para graficar

Por ejemplo: 'Haz un histograma de la columna edad'

Hazme un grafico de Arroz

Enviar petición

No se generó ningún código gráfico válido.

Respuesta del agente:

No puedo generar un gráfico con esa petición. Por favor, sé claro y específico indicando qué columnas o tipo de gráfico deseas.

Figura XI.6: Ejemplo de consulta errónea

ANEXO XII

Ejemplos de salida de la sección “inferencia causal”

Este anexo contiene varias ejemplificaciones que se han realizado sobre el *dataset* utilizado en las pruebas, con el objetivo de mostrar de forma visual y detallada el comportamiento del sistema en las distintas fases del análisis causal.

XII.1 Definición del tratamiento y variable objetivo

Ejemplos de selección de columnas de tratamiento y resultado, incluyendo recodificación automática en el caso de variables categóricas con más de dos niveles.

¿Quieres seleccionar una columna como tratamiento?

☒ Sí, seleccionar tratamiento
☐ No, formular una pregunta causal

Selecciona la columna de tratamiento:

Model

⚠ La columna seleccionada no es binaria. Se convertirá automáticamente a una variable binaria.

Columna seleccionada como tratamiento: Model (convertida a binaria)

Codificación aplicada a Model:

- Valor 1 → e-Soul 64 kWh
- Valor 0 → ≠e-Soul 64 kWh

Selecciona la variable objetivo (resultado):

Brand

✖ La variable objetivo debe ser numérica o binaria. Selecciona otra.

Figura XII.1: Indicaciones del sistema al seleccionar tratamiento y resultado.

XII.2 Definición de pregunta causal y selección de hipótesis

Ejemplos de preguntas causales y selección de variables candidatas. Además, se muestran las diversas hipótesis generadas por el agente.

Selección del Tratamiento o Pregunta Causal

¿Quieres seleccionar una columna como tratamiento?

- ☐ Sí, seleccionar tratamiento
☒ No, formular una pregunta causal

Formula tu pregunta causal (por ejemplo: ¿Qué influye en el precio?)

que variables influyen en PriceEuro

 Encontrar variables candidatas

Variables candidatas encontradas

Selecciona cómo deseas elegir la variable para el análisis causal:

- ☒ Seleccionar una de las variables candidatas sugeridas
☐ Quiero seleccionar otra variable manualmente

Selecciona una de las variables candidatas:

- ☒ TopSpeed_KmH

Has seleccionado: **TopSpeed_KmH**. Pulsa el botón para continuar.

Figura XII.2: Formulación de pregunta causal.

Sugerencias de hipótesis causales

Selecciona una hipótesis sugerida o escribe la tuya propia:

- ☒ "Los vehículos cuya velocidad máxima supera los 200 km/h (TopSpeed_KmH) tienden a tener precios más altos (PriceEuro)."
☐ "Los vehículos cuya velocidad máxima es inferior a 150 km/h (TopSpeed_KmH) tienden a tener precios más bajos (PriceEuro)."
☐ "Los vehículos cuya velocidad máxima se encuentra entre 180 km/h y 250 km/h (TopSpeed_KmH) tienden a tener precios más altos (PriceEuro)."
☐ Quiero escribir mi propia hipótesis

☒ Hipótesis seleccionada: "Los vehículos cuya velocidad máxima supera los 200 km/h (TopSpeed_KmH) tienden a tener precios más altos (PriceEuro)."

Figura XII.3: Hipótesis generadas por el agente.

Sugerencias de hipótesis causales

Selecciona una hipótesis sugerida o escribe la tuya propia:

- ☒ Los vehículos con un precio superior a 60000 euros (PriceEuro) tienden a tener una velocidad máxima mayor (TopSpeed_KmH).
☐ Los vehículos con un precio inferior a 30000 euros (PriceEuro) tienden a tener una velocidad máxima menor (TopSpeed_KmH).
☐ Quiero escribir mi propia hipótesis

☒ Hipótesis seleccionada: Los vehículos con un precio superior a 60000 euros (PriceEuro) tienden a tener una velocidad máxima mayor (TopSpeed_KmH).

Figura XII.4: Hipótesis generadas por el agente.

A partir de aquí, se usa como ejemplificación la hipótesis siguiente: "Los vehículos cuya velocidad máxima supera los 180 km/h (TopSpeed_KmH) tienden a tener precios más altos (PriceEuro)." Tanto para el ejemplo de la Figura XII.5 como las posteriores.

☒ Columna "treatment" generada y aplicada al DataFrame.

Explicación: La columna `treatment` se crea asignando 1 a los vehículos cuya velocidad máxima (TopSpeed_KmH) es mayor a 180 km/h, y 0 en caso contrario. Luego, la columna `TopSpeed_KmH` se elimina del DataFrame.

 Así se ve el dataset con la nueva columna `treatment`:

		AccelSec	Range_Km	Efficiency_WhKm	FastCharge_KmH	RapidCharge	PowerTrain	PlugType	BodyStyle	Segment	Seats	PriceEuro	treatment
0	ige Dual Motor	4.6	450	161	940	Yes	AWD	Type 2 CCS	Sedan	D	5	55480	1
1		10	270	167	250	Yes	RWD	Type 2 CCS	Hatchback	C	5	30000	0
2		4.7	400	181	620	Yes	AWD	Type 2 CCS	Liftback	D	5	56440	1
3		6.8	360	206	560	Yes	RWD	Type 2 CCS	SUV	D	5	68040	0
4		9.5	170	168	190	Yes	RWD	Type 2 CCS	Hatchback	B	4	32997	0

Figura XII.5: Generación columna treatment.

XII.3 Generación y edición del DAG

Visualización inicial del grafo causal sugerido automáticamente por el agente, junto con la opción de edición manual mediante código DOT.

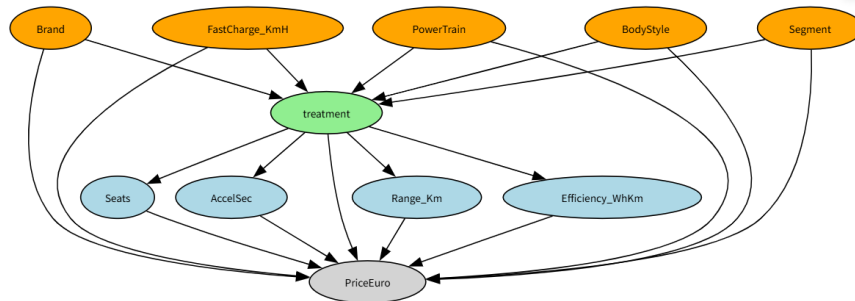


Figura XII.6: Ejemplo de DAG generado automáticamente por el sistema.

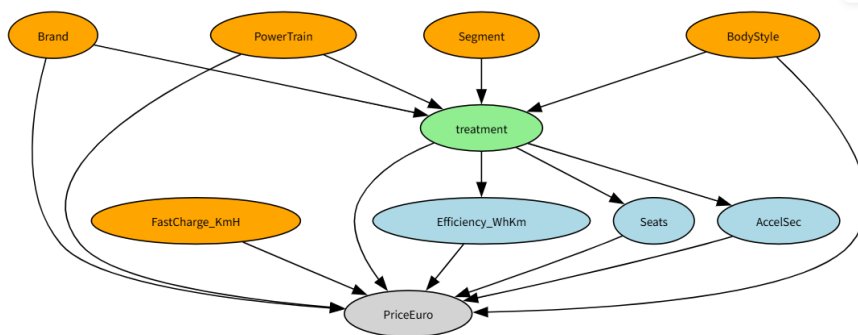


Figura XII.7: Ejemplo de DAG modificado por usuario.

XII.4 Diagnóstico de identificabilidad del efecto

Ejemplo de salida textual del agente, mostrando el criterio de identificación aplicado y las variables de ajuste detectadas.

Diagnóstico de identificabilidad

Criterio usado: `backdoor`

Variables de ajuste: `['PowerTrain', 'Segment', 'Brand', 'BodyStyle', 'FastCharge_KmH']`

Justificación: El efecto causal ha sido identificado utilizando el criterio de 'backdoor'. Esto significa que, condicionado a las variables 'PowerTrain', 'Segment', 'Brand', 'BodyStyle' y 'FastCharge_KmH', el tratamiento es independiente de cualquier causa común no medida que afecte tanto al tratamiento como al resultado. Por lo tanto, estas variables deben ser incluidas en el ajuste para obtener una estimación insesgada del efecto causal del tratamiento sobre el resultado. Si el efecto no fuera identificable por backdoor, se deberían explorar otros criterios como 'frontdoor' o 'variables instrumentales', o bien, reconsiderar el grafo causal.

Figura XII.8: Diagnóstico generado por el agente.

Para los casos en los que el DAG está incompleto, el agente responde como en la Figura XII.10

DAG utilizado para el modelo

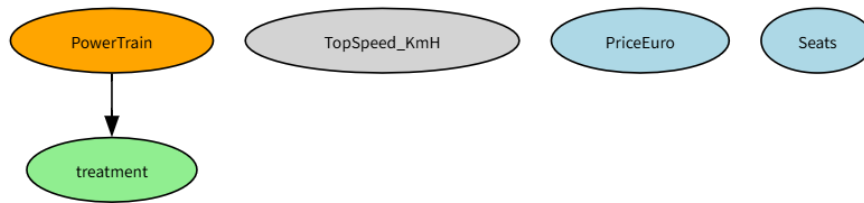


Figura XII.9: DAG incompleto.

Diagnóstico de identificabilidad

Criterio usado: `backdoor`

Variables de ajuste: []

Justificación: El efecto causal se identificó como cero debido a que no existe una ruta dirigida desde la variable de tratamiento hacia la variable de resultado en el grafo causal proporcionado. Por lo tanto, no se requieren variables de ajuste para eliminar el sesgo de confusión.

Figura XII.10: Diagnóstico generado por el agente tras DAG incompleto.

XII.5 Recomendación del método de estimación

Salida del método recomendado por el agente para la estimación causal.

Método recomendado por el agente:

backdoor.linear_regression

Justificación: El score de propensión muestra un AUC muy alto de 0.951, lo que indica un solapamiento deficiente entre los grupos de tratamiento y control. Esto desaconseja el uso de métodos de matching o IPW. Aunque las variables de ajuste son categóricas y el tamaño de la muestra es relativamente pequeño, la herramienta de diagnóstico recomienda usar regresión lineal. Se debe tener precaución al interpretar los resultados debido al posible desbalance y la necesidad de codificar correctamente las variables categóricas (one-hot encoding) antes de la regresión.

Selecciona el método que deseas usar para la estimación:

Método de estimación causal:

backdoor.linear_regression

Método seleccionado para estimación: backdoor.linear_regression

Figura XII.11: Método de estimación recomendado por el agente.

XII.6 Estimación del efecto causal

Ejemplos de salidas del sistema tras ejecutar la estimación, incluyendo el valor numérico del efecto y su intervalo de confianza.

XII.6.1. Regresión lineal

Estimación e interpretación del agente:

Magnitud del efecto

En promedio, el tratamiento tiene un efecto estimado de 39,275.69 euros sobre la variable de resultado. Este valor representa la diferencia esperada en la variable de resultado entre los individuos que recibieron el tratamiento y aquellos que no lo recibieron.

Intervalo de confianza y significancia

El intervalo de confianza del 95% para este efecto se encuentra entre -17,006.34 euros y 95,557.73 euros. Debido a que este intervalo incluye el valor cero, no podemos descartar la posibilidad de que el tratamiento no tenga ningún efecto real sobre la variable de resultado.

Interpretación y conclusiones

Este resultado sugiere que, si bien observamos un efecto positivo promedio del tratamiento, existe una considerable incertidumbre asociada a esta estimación. No se puede afirmar con un nivel de confianza del 95% que el tratamiento tenga un efecto significativo sobre la variable de resultado, ya que el intervalo de confianza incluye valores negativos y el cero. Se requiere más investigación o un análisis más robusto para determinar si el tratamiento tiene un impacto real.

Figura XII.12: Estimación con regresión lineal.

XII.6.2. *Propensity score weighting*

Estimación e interpretación del agente:

Magnitud del efecto

En promedio, el tratamiento incrementa el valor de la variable de resultado en aproximadamente 40,395 unidades. Este valor representa la magnitud del efecto causal estimado.

Intervalo de confianza y significancia

El intervalo de confianza del 95% para este efecto se encuentra entre 26,018 y 52,572 unidades. Esto significa que, con un 95% de confianza, el verdadero efecto del tratamiento se encuentra dentro de este rango. Dado que el intervalo no incluye el valor cero, podemos considerar que el efecto estimado es estadísticamente significativo.

Interpretación y conclusiones

Este resultado sugiere que el tratamiento tiene un impacto positivo considerable en la variable de resultado. Con cierta incertidumbre, podemos afirmar que la aplicación del tratamiento conduce a un aumento sustancial en el valor de la variable de resultado. No se puede descartar que el efecto real se encuentre en algún punto entre 26,018 y 52,572 unidades, pero la evidencia apunta a un efecto positivo y relevante.

Figura XII.13: Estimación con *propensity score weighting*.

XII.6.3. *Distance matching*

Selecciona el método que deseas usar para la estimación:

Método de estimación causal:

backdoor.distance_matching

 Método seleccionado para estimación: backdoor.distance_matching

Estimación del efecto causal

El agente realizará la estimación y la interpretará

Estimación e interpretación del agente:

En este caso, el método seleccionado no ha producido una estimación confiable debido a un error interno. Podría ser recomendable probar otro enfoque de estimación causal más robusto para este conjunto de datos.

Figura XII.14: Estimación con *distance matching* (error).

XII.7 Validación de robustez

Resultados de la técnica de refutación (*random common cause*), mostrando la comparación entre el efecto original y el efecto tras la perturbación.

XII.7.1. Regresión lineal

 Resultado de la validación del agente:

Resultado de la refutación: La aplicación de la prueba de refutación 'random_common_cause' resultó en una variación mínima del efecto estimado. El valor original de 39275.69 se modificó a 38813.84 tras la introducción de una causa común aleatoria. No se proporcionó un valor p en este análisis.

Conclusión de robustez: Dado que el cambio en el efecto estimado es pequeño en relación con el valor original, se puede concluir que el efecto estimado es robusto y no parece ser sensible a la presencia de variables omitidas aleatorias.

Figura XII.15: Validación con regresión lineal.

 Resultado de la validación del agente:

Resultado de la refutación:

La refutación, realizada mediante la adición de una causa común aleatoria, muestra un cambio mínimo en el efecto estimado. El efecto original fue de 40395.212685536164, mientras que el nuevo efecto tras la refutación es de 40395.21268553617. No se proporciona un valor p en el resultado.

Conclusión de robustez:

Dado que el cambio en el efecto estimado tras la refutación es prácticamente nulo, se puede concluir que el efecto estimado es robusto a la presencia de causas comunes aleatorias. La estabilidad del efecto sugiere que no es muy sensible a posibles variables omitidas que podrían actuar como causas comunes.

Figura XII.16: Validación con *propensity score weighting*.